

DESIGN Output - Stage 1

Task: Generated by LLM
Agent: design
Stage: 1
Timestamp: 2026-09-26T22:30:25.886962
Model: deepseek-v4.1-flash
Provider: opencode-go

Token Usage

- **Input Tokens:** 114440
- **Output Tokens:** 137557
- **Cached Tokens:** 62080
- **Reasoning Tokens:** 60186
- **Total Tokens:** 251997
- **Cost:** \$0.099700
- **Cost per 1K input:** \$0.00015
- **Cost per 1K output:** \$0.0006

Output

F-1: Intelligent Task Prioritization

Requirements

Functional Requirements (global ids — owned range FR-1..FR-25)

ID	Title	Summary
FR-1	Automatic priority scoring	The system assigns every active task a numeric priority score derived from the user's prioritization profile.
FR-2	Ranked priority queue	The system exposes an ordered "Today" queue of active tasks sorted by descending priority score.
FR-3	Ranking factors	Scoring uses at minimum: due date/time, explicit user importance, estimated effort, task age, and dependency state.
FR-4	Factor weighting controls	The user can adjust the relative weight of each ranking factor.
FR-5	Explainability	The user can view, per task, a plain-language explanation of why it has its current rank.
FR-6	Manual override	The user can pin a task to the top of the queue or force it into a specific rank position.
FR-7	Override scope and expiry	Overrides can be indefinite or time-boxed (until a date/time or until task completion).
FR-8	Override visibility and removal	Pinned/overridden tasks are visually marked and the override can be removed by the user at any time.
FR-9	Deadline urgency escalation	Tasks approaching or past their due date escalate in score as the deadline nears.
FR-10	Blocked task handling	Tasks blocked by an incomplete dependency are demoted and labelled as blocked.

| FR-11 | Dependency unblocking | When a blocking task completes, dependent tasks are re-scored and may re-enter the active queue. |

| FR-12 | Recompute triggers | Scores are recomputed on task create/update/complete/delete, due-date change, dependency change, override change, profile change, and on a periodic schedule. |

| FR-13 | Deterministic ordering | Ties are broken deterministically so the queue order is stable across identical recomputes. |

| FR-14 | Priority tiers | Each task is bucketed into a tier (Critical / High / Medium / Low) derived from its score, with tier thresholds configurable. |

| FR-15 | Queue filtering and segmentation | The queue can be filtered (e.g., by project, tag, due window, effort, blocked state) without altering underlying scores. |

| FR-16 | Deferred/hidden tasks | Tasks explicitly deferred by the user are excluded from the active queue but retain their score. |

| FR-17 | Stale score invalidation | A score older than its validity window is flagged stale and refreshed before being presented as authoritative. |

| FR-18 | Bulk recalculation | The user can trigger a full recalculation of all active task scores. |

| FR-19 | Score history | The system retains a bounded history of score changes per task for auditing and trend display. |

| FR-20 | Drift feedback signal | The system accepts lightweight user feedback (e.g., "this is not important") and applies it as a bounded adjustment. |

| FR-21 | Cross-device consistency | The priority queue and overrides are consistent for the same account across devices after sync. |

| FR-22 | Offline override capture | Overrides created while offline are queued locally and reconciled on reconnect using last-write-wins with conflict surfacing. |

| FR-23 | Explanation provenance | Every explanation lists the factors and weights that produced the score at the time it was generated. |

| FR-24 | Queue size limits | The active queue is paginated/limited to a configurable maximum with the remainder accessible via continuation. |

| FR-25 | Priority settings reset | The user can reset weighting and tier thresholds to system defaults. |

****Non-Functional Requirements (owned range NFR-1..NFR-15)****

| ID | Category | Requirement |

|---|---|---|

| NFR-1 | Performance | Priority queue retrieval returns first page within 300 ms at p95 for accounts with up to 5,000 active tasks. |

| NFR-2 | Performance | Single-task score recomputation completes within 100 ms at p95. |

| NFR-3 | Performance | Full recalculation of 5,000 active tasks completes within 10 s and runs asynchronously. |

| NFR-4 | Scalability | Scoring must remain correct and within NFR-1 as task count grows linearly to 50,000 active tasks per account. |

| NFR-5 | Availability | Priority queue read endpoints must sustain 99.9% monthly availability; ranking degrades to last-known-good ordering rather than failing. |

| NFR-6 | Security | Only the owning account (or an explicitly delegated collaborator) may read or mutate scores, weights, and overrides. |

| NFR-7 | Security | Override and weighting changes are auditable with actor, timestamp, and before/after values. |

| NFR-8 | Data residency | Priority data, overrides, and score history are stored in the user's configured data region and are not replicated outside it. |

| NFR-9 | Data retention | Score history retains at most 90 days or 500 entries per task, whichever is smaller; overrides persist until removed. |

| NFR-10 | Deployment/environment | Scoring runs identically in local, staging, and production environments; behaviour is controlled by configuration, not environment-specific branches. |

| NFR-11 | Deployment/environment | Weight-profile schema changes are backward compatible for at least one prior version to allow rolling deploys. |

NFR-12	Reliability	Recompute operations are idempotent; repeated runs with unchanged inputs produce identical scores.
NFR-13	Observability	Score computation emits metrics for latency, recompute volume, override rate, and stale-score count.
NFR-14	Accessibility	Rank, tier, blocked state, and override markers are conveyed by text/label in addition to colour, and the queue is fully keyboard navigable.
NFR-15	Privacy	Ranking factors must not require content inspection beyond task metadata already stored; no task body text is used unless the user opts in.

****User Stories (owned range US-1..US-15)****

ID	Story
US-1	As a busy professional, I want my tasks automatically ordered by what matters most, so that I can start working without deciding what to do first.
US-2	As a user, I want to see why a task is ranked where it is, so that I can trust the ordering.
US-3	As a user, I want to pin a task to the top, so that I can override the system when I know better.
US-4	As a user, I want blocked tasks pushed down, so that I am not nudged toward work I cannot start.
US-5	As a user, I want to tune how much deadlines versus importance matter, so that the ranking matches how I actually work.
US-6	As a user, I want overdue tasks to surface aggressively, so that nothing silently slips.
US-7	As a user, I want to filter the queue without changing scores, so that I can focus on one project at a time.
US-8	As a user, I want to defer a task out of the active queue, so that it stops competing for attention.
US-9	As a user, I want my pins and settings to survive app restarts and device changes, so that my setup is portable.
US-10	As a user, I want to record a pin while offline, so that I am not blocked by connectivity.
US-11	As a user, I want to reset my weighting to defaults, so that I can recover from a bad configuration.
US-12	As a user, I want to force a full recalculation, so that I can correct a queue that looks stale.
US-13	As a user, I want to tell the system a ranking is wrong, so that it adapts over time.
US-14	As a user, I want to remove an override, so that the task returns to automatic ranking.
US-15	As a user, I want a large task list paginated, so that the app stays responsive.

Behaviour

****Scoring model (conceptual, technology-agnostic)****

Each active task receives a score computed from a bounded set of normalised factors, combined using the user's weighting profile, then adjusted by bounded modifiers.

DM ID	Concept	Description
DM-1	Task	A unit of work with due date/time (optional), importance (user-set), estimated effort, created-at, status, project/tags, and dependency links.
DM-2	PriorityProfile	Per-user configuration: factor weights, tier thresholds, urgency horizon, staleness window, feedback adjustment cap, default override duration.
DM-3	PriorityFactor	A named contributor to a score (e.g., deadline urgency, importance, effort fit, age, dependency state) with its weight and contribution value.

DM-4	PriorityScore	Computed value for a task: numeric score, tier, ordered factor contributions, computed-at timestamp, model/profile version, staleness flag.
DM-5	PriorityOverride	User- or system-created ordering instruction: type (pin / fixed-rank / defer), scope, expiry, creator, reason (optional), active flag.
DM-6	PriorityAuditEvent	Immutable record of a score or override change: actor, timestamp, before/after values, trigger.
DM-7	RankedQueue	The ordered, paginated projection of active tasks after scoring, overrides, deferral, and filtering.
DM-8	FeedbackSignal	A bounded user reaction to a ranking ("too low"/"too high"/"not important") with a decay policy.

****Behaviour flow****

1. A task is created or modified → the system marks it for scoring (FR-12).
2. The scorer normalises each factor to a comparable 0–1 range and multiplies by the profile weight for that factor (FR-1, FR-3, FR-4).
3. Modifiers apply in a fixed, documented order: dependency/blocked demotion (FR-10), urgency escalation as the deadline nears (FR-9), stale-score invalidation (FR-17), bounded feedback adjustment (FR-20).
4. A numeric score and a tier are produced and persisted with the factor breakdown and profile version (FR-14, FR-23).
5. Overrides are applied on top of the score at queue-assembly time, not baked into the score, so removing an override restores automatic ordering exactly (FR-6, FR-8).
6. The ranked queue is assembled: active, undeferred tasks sorted by override precedence, then score descending, then deterministic tie-breakers (FR-2, FR-13, FR-16).
7. Filters and pagination are applied to the assembled queue without mutating stored scores (FR-15, FR-24).
8. Explanations are generated from the stored factor breakdown of the score that produced the current rank (FR-5, FR-23).

****Ordering precedence (highest to lowest)****

1. Active `fixed-rank` overrides, ascending by target rank.
2. Active `pin` overrides, ordered among themselves by score descending, then tie-breakers.
3. Automatic score descending, then tie-breakers.

****Deterministic tie-breakers (FR-13), applied in order****

1. Earlier due date/time wins; tasks with no due date sort after tasks with one.
2. Higher user importance wins.
3. Older `created-at` wins.
4. Lexicographic ascending task identifier wins (guarantees total order).

****Recompute triggers and scheduling (FR-12, FR-18)****

- Event-driven: task create/update/complete/delete, due-date change, dependency link change, override change, profile change, feedback signal.
- Time-driven: periodic sweep (default every 15 minutes) to catch time-based urgency drift.
- User-driven: explicit full recalculation.
- Sweeps and full recalculations run asynchronously and are idempotent (NFR-3, NFR-12).

****Cross-device and offline behaviour (FR-21, FR-22)****

- Overrides and profile changes carry a client timestamp and a per-record version.
- On reconnect, server reconciles: highest version wins; if two different active overrides target the same task with equal versions, both are preserved in conflict state and the user is asked to choose; until resolved the task is shown with a conflict marker and uses automatic scoring.

Business rules

| ID | Rule |

|---|---|

| BR-1 | Only tasks in an active (not completed, not archived, not deleted) status participate in scoring and the queue. |

| BR-2 | Completing or deleting a task removes it from the queue immediately and triggers recompute of its dependents. |

| BR-3 | A task cannot hold both a `pin` and a `fixed-rank` override simultaneously; the newer instruction replaces the older and the superseded one is recorded in the audit log. |

| BR-4 | A `fixed-rank` override that targets a rank beyond the queue size places the task at the end of the queue. |

| BR-5 | If two tasks are assigned the same `fixed-rank`, the later-created override takes the lower (worse) position and the earlier one shifts up by one. |

| BR-6 | A task with at least one incomplete dependency is demoted below all unblocked tasks with an equal or lower tier and is labelled "Blocked". |

| BR-7 | A blocked task may still be pinned by the user; the user's pin outranks the blocked demotion. |

| BR-8 | An overdue task is placed in the Critical tier regardless of computed score, unless the user has deferred it. |

| BR-9 | Deferred tasks are excluded from the active queue but keep their score and history; the deferral expires at its configured time or never, per BR-10. |

| BR-10 | A deferral without an expiry remains in force until the user removes it or the task is completed. |

| BR-11 | Score history is append-only from the user's perspective; entries are removed only by retention policy (NFR-9), never edited. |

| BR-12 | Feedback adjustments are capped by `feedback adjustment cap` in the profile and decay over time so that repeated feedback cannot permanently dominate ranking. |

| BR-13 | Weights are relative, not absolute: the system normalises weights so the total influence stays constant regardless of the raw numbers entered. |

| BR-14 | Tier thresholds must be strictly descending (Critical > High > Medium > Low); an invalid configuration is rejected, not silently coerced. |

| BR-15 | The queue is always scoped to a single account; no cross-account ranking exists. |

| BR-16 | A stale score is never presented as authoritative without a staleness indicator; the queue may serve last-known-good values while refresh is in flight. |

| BR-17 | A task aged beyond the configured "age ceiling" stops accruing additional age-based score, preventing indefinite drift upward. |

| BR-18 | Removing an override must restore the exact ordering the task would have had under current scores with no override present. |

Validation

Validation applies to all inputs that affect scoring. Violations return a structured error (see API behaviour) and never partially apply.

- Weights: each factor weight must be a finite number in the inclusive range 0–100; at least one weight must be greater than 0. Non-numeric, negative, or out-of-range values are rejected.
- Tier thresholds: finite numbers, strictly descending, and each within the score scale's bounds.
- Urgency horizon: a positive duration, minimum 1 hour, maximum 365 days.
- Staleness window: a positive duration, minimum 60 seconds, maximum 24 hours.
- Feedback adjustment cap: a percentage in the inclusive range 0–50.
- Override type: must be one of `pin`, `fixed-rank`, `defer`.

- `fixed-rank` target: integer ≥ 1 .
- Override expiry: an absolute timestamp strictly in the future at the moment of acceptance; relative expiries are converted server-side.
- Deferral without expiry: allowed only if the profile permits indefinite deferral; otherwise an expiry is required.
- Feedback signal value: must be one of the enumerated signals; free-text reasons are optional and length-capped.
- Task fields used in scoring: due date/time, if present, must parse as a valid instant; importance must be within its defined scale; estimated effort must be > 0 when provided.
- Pagination: page size within documented minimum/maximum; cursor must be well-formed and not expired.
- Dependency links: a task cannot depend on itself and cycles among dependencies are rejected.

Edge cases

ID	Case	Required behaviour
EC-1	User has zero active tasks	Queue returns an empty result with an explicit empty-state payload; no error.
EC-2	All active tasks are deferred or blocked	Queue returns the items it has; if none remain, the response indicates only deferred/blocked items exist and surfaces them in a separate count.
EC-3	Task has no due date	Deadline urgency contributes nothing; the task is ranked on remaining factors only.
EC-4	Task has no estimated effort	Effort factor is treated as neutral (contributes zero), not as zero effort.
EC-5	Due date changed to the past	Task becomes overdue per BR-8 and is recomputed immediately.
EC-6	Due date removed	Urgency contribution drops to zero; the change is reflected in the next recompute and in the explanation.
EC-7	Clock skew between client and server	Server time is authoritative for urgency and expiry; client-supplied timestamps are used only for conflict resolution ordering, not for urgency.
EC-8	Two clients pin different tasks within the same second	Both pins are honoured; relative order among pins falls back to score, then tie-breakers.
EC-9	Same task pinned on two devices offline	Reconciles to a single pin (last-write-wins by version); no duplicate override records.
EC-10	Override expires while the queue is being read	The read returns a consistent snapshot; the expiry takes effect at the next read or recompute, not mid-response.
EC-11	Task is completed while pinned	Completion wins; the task leaves the queue and the override is marked inactive, retained in history.
EC-12	Dependency chain longer than the configured depth	Cycle detection and depth cap prevent unbounded traversal; the task is labelled Blocked with the chain depth reported.
EC-13	All weights set to zero	Rejected by validation; the previously saved profile remains in force.
EC-14	Account exceeds the queue maximum	Queue is truncated per FR-24 with a continuation cursor and a count of remaining items.
EC-15	Full recalculation requested while one is already running	The second request is coalesced into the running job; the user is told a recalculation is already in progress.
EC-16	Stale scores served during an outage of the scoring path	Queue serves last-known-good ordering with a staleness indicator (NFR-5, BR-16); no error page.
EC-17	Profile version newer than the scorer supports	The system falls back to the last supported profile version, flags the fallback, and logs an alert.

EC-18	Task deleted between queue assembly and explanation fetch	Explanation request returns a not-found outcome with a clear message, not an internal error.
EC-19	Very large task count (50,000 active)	Ranking honours NFR-4; pagination and asynchronous recompute keep responses within NFR-1.
EC-20	User submits contradictory feedback repeatedly in a short window	Adjustment is capped (BR-12) and decays; no unbounded score movement.

Error handling

ID	Condition	Handling
EH-1	Invalid profile (weights/thresholds/horizons)	Reject with a 422-style validation outcome listing every offending field; no partial save.
EH-2	Invalid override payload	Reject with the same structured validation outcome; existing overrides unchanged.
EH-3	Override targets a task not owned by the account	Reject without revealing task existence; no state change.
EH-4	Recompute job fails partway	Job is retryable and idempotent; previously written scores remain valid; failures are recorded in the audit log and surfaced in observability (NFR-13).
EH-5	Scoring dependency (e.g., dependency graph store) unavailable	Serve last-known-good queue with staleness indicator; queue reads never 5xx solely because scoring is unavailable.
EH-6	Queue cursor expired or invalid	Return a defined "cursor expired" outcome instructing the client to restart pagination from the first page.
EH-7	Optimistic-concurrency conflict on profile update	Reject the second write with a conflict outcome including the current version; client must re-read and retry.
EH-8	Rate limit exceeded on recompute or feedback endpoints	Return a throttling outcome with a retry-after hint; no work is queued.
EH-9	Unexpected internal error in scoring	Task retains its last valid score; the error is logged with a correlation id and never blocks queue reads.
EH-10	Explanation requested for a task with no stored breakdown	Recompute on demand; if unavailable, return an explicit "explanation unavailable" outcome rather than an empty or misleading one.

Acceptance criteria

ID	Criterion
AC-1	Given a set of active tasks with distinct due dates and importances, when the queue is fetched, then tasks are ordered by descending score with the documented tie-breakers applied.
AC-2	Given an overdue task, when the queue is fetched, then the task appears in the Critical tier unless deferred.
AC-3	Given a task with an incomplete dependency, when the queue is fetched, then it appears below unblocked tasks of equal or lower tier and is labelled Blocked.
AC-4	Given the blocking task is completed, when recompute finishes, then the previously blocked task is re-scored and appears in its automatic position.
AC-5	Given a user pins a task, when the queue is fetched, then the pinned task appears above all non-pinned tasks and is visually marked as pinned.

| AC-6 | Given a pin is removed, when the queue is fetched, then the task's position exactly matches its position under current scores with no override. |

| AC-7 | Given a user requests an explanation for a task, then the response lists every factor, its weight, its contribution, the profile version, and the computation timestamp. |

| AC-8 | Given a user changes a factor weight, when the profile save succeeds, then all affected scores are recomputed and the new ordering is served. |

| AC-9 | Given all weights are set to zero, when the user attempts to save, then the save is rejected with a field-level error and the previous profile remains in force. |

| AC-10 | Given a task is deferred with an expiry, then it is absent from the active queue before expiry and present after the expiry time following the next recompute. |

| AC-11 | Given a task is completed, then it leaves the queue immediately and dependents are recomputed. |

| AC-12 | Given an account with 5,000 active tasks, then first-page queue retrieval meets the p95 latency target in NFR-1. |

| AC-13 | Given an account with 50,000 active tasks, then ranking is correct and pagination returns the full ordered set via cursors. |

| AC-14 | Given a full recalculation of 5,000 tasks is triggered, then it completes asynchronously within NFR-3 and produces scores identical to an incremental recompute of the same inputs. |

| AC-15 | Given two recomputes run with identical inputs, then the resulting scores and queue order are byte-identical (NFR-12). |

| AC-16 | Given an override is created offline, when connectivity returns, then the override is applied and visible on all of the user's devices. |

| AC-17 | Given conflicting offline overrides on the same task, then a conflict state is surfaced and the task uses automatic scoring until resolved. |

| AC-18 | Given scoring is unavailable, then the queue still returns the last-known-good order with a staleness indicator and HTTP success. |

| AC-19 | Given the user resets settings, then weights and tier thresholds return to documented defaults and scores are recomputed. |

| AC-20 | Given a non-owner attempts to read or mutate another account's scores, weights, or overrides, then the request is rejected without data leakage. |

| AC-21 | Given the queue is navigated using keyboard only, then every item, explanation trigger, and override control is reachable and labelled (NFR-14). |

| AC-22 | Given tier and override state, then they are distinguishable without relying on colour alone. |

| AC-23 | Given repeated contradictory feedback within a short window, then the cumulative adjustment never exceeds the configured cap. |

| AC-24 | Given a task aged beyond the age ceiling, then its age contribution stops increasing. |

| AC-25 | Given a user filters the queue by project, then the underlying stored scores are unchanged and clearing the filter restores the original order. |

API behaviour

All endpoints are account-scoped and require authentication. All responses use a stable envelope: `data`, `meta` (pagination, profile version, staleness), and `errors` (array of field-level problems). Writes are idempotent where noted.

ID	Method & path	Purpose	Key inputs	Success	Notable failures
API-1	GET /priority/queue	Fetch the ranked active queue	`cursor`, `limit`, `filter[project]`, `filter[tag]`, `filter[dueWindow]`, `filter[blocked]`, `filter[tier]`	200 with ordered items (task id, score, tier, blocked flag, override marker, stale flag) plus `meta.nextCursor` and `meta.profileVersion`	EH-6, EH-5, EH-8

| API-2 | `GET /priority/tasks/{taskId}/score` | Fetch a task's current score and factor breakdown | `taskId` | 200 with score, tier, factors, `computedAt`, `profileVersion`, `stale` | EH-10, EH-3 |

| API-3 | `GET /priority/tasks/{taskId}/explanation` | Fetch the plain-language explanation of current rank | `taskId`, `locale` | 200 with summary text plus the structured factor list and the rank it explains | EH-10, EH-3, EC-18 |

| API-4 | `POST /priority/tasks/{taskId}/overrides` | Create a pin, fixed-rank, or defer override | `type`, `targetRank` (fixed-rank only), `expiresAt` (optional), `reason` (optional), `clientVersion` | 201 with the created override and the resulting queue position | EH-2, EH-3, EH-7, EC-8 |

| API-5 | `DELETE /priority/tasks/{taskId}/overrides/{overrideId}` | Remove an override | `overrideId` | 204; subsequent queue reads reflect automatic ordering (AC-6) | EH-3, EH-6 |

| API-6 | `GET /priority/tasks/{taskId}/overrides` | List overrides for a task, including inactive history | `taskId`, `includeInactive` | 200 with override records and audit metadata | EH-3 |

| API-7 | `GET /priority/profile` | Read the user's weighting profile | — | 200 with weights, tier thresholds, horizons, staleness window, feedback cap, defaults for comparison | — |

| API-8 | `PUT /priority/profile` | Replace the weighting profile | Full profile body + `version` | 200 with normalised profile and `version` incremented; recompute scheduled | EH-1, EH-7 |

| API-9 | `POST /priority/profile/reset` | Reset weights and thresholds to defaults | optional `scope` (weights only / all) | 200 with default profile and recompute scheduled | EH-7 |

| API-10 | `POST /priority/recompute` | Trigger full recalculation | `scope` (account / project / task), `dryRun` (optional) | 202 with job id and status URL; coalesced if already running (EC-15) | EH-4, EH-8, EC-15 |

| API-11 | `GET /priority/recompute/{jobId}` | Poll recalculation status | `jobId` | 200 with state ('queued'/'running'/'succeeded'/'failed'), counts, and errors summary | EH-4 |

| API-12 | `POST /priority/tasks/{taskId}/feedback` | Submit a bounded ranking-feedback signal | `signal` (enum), `reason` (optional) | 200 with the adjusted score and the applied (capped) delta | EH-2, EH-8, EC-20 |

| API-13 | `POST /priority/queue/reorder` | Persist an explicit user drag-reorder as fixed-rank overrides | ordered list of task ids + `clientVersion` | 200 with the created/replaced overrides and the resulting order | EH-2, EH-7, BR-5 |

| API-14 | `GET /priority/tasks/{taskId}/history` | Read bounded score history | `taskId`, `limit`, `cursor` | 200 with entries (score, tier, trigger, actor, timestamp) within retention limits | EH-6, NFR-9 |

****Cross-cutting API rules****

- Read endpoints never fail solely because scoring is unavailable; they degrade per EH-5 and set `meta.stale = true`.
- Write endpoints validate fully before mutating; validation failures return field-level `errors` and change nothing (EH-1, EH-2).
- Override and profile writes require a client-supplied `version` for optimistic concurrency; mismatches return a conflict outcome (EH-7).
- `POST /priority/recompute` and `POST /priority/tasks/{taskId}/feedback` accept an idempotency key; replays return the original outcome.
- Pagination cursors are opaque, time-bounded, and account-scoped; expiry is explicit (EH-6).
- All list endpoints are stable-sorted: identical request parameters against unchanged state return identical ordering.

Tests

| ID | Type | What it proves |

| ---|---|---|

| T-1 | Unit | Factor normalisation and weighted combination for boundary inputs (0, max, missing). |

| T-2 | Unit | Tie-breaker chain produces a total order for fully equal scores. |

T-3	Unit	Weight normalisation keeps total influence constant across raw scales (BR-13).
T-4	Unit	Override precedence: fixed-rank beats pin beats automatic.
T-5	Unit	Removing an override restores the pre-override position exactly (AC-6, BR-18).
T-6	Unit	Feedback adjustment is capped and decays (BR-12, EC-20).
T-7	Unit	Age ceiling stops further age contribution (BR-17, AC-24).
T-8	Unit	Validation rejects zero-weight, non-descending thresholds, and past expiries (EH-1, EH-2).
T-9	Integration	Event-driven recompute fires on every trigger listed in FR-12.
T-10	Integration	Dependency completion re-scores dependents and unblocks them (AC-4).
T-11	Integration	Cycle and self-dependency in the dependency graph are rejected.
T-12	Integration	Full recalculation output equals incremental recompute output for identical inputs (AC-14, NFR-12).
T-13	Integration	Idempotency keys on recompute and feedback produce single effects.
T-14	Integration	Optimistic-concurrency conflict returns conflict and preserves stored profile (EH-7).
T-15	Performance	Queue first page within p95 target at 5,000 active tasks (NFR-1).
T-16	Performance	Single-task recompute within p95 target (NFR-2).
T-17	Performance	Full recalculation of 5,000 tasks within the NFR-3 budget.
T-18	Scale	Correct ranking and pagination at 50,000 active tasks (NFR-4, EC-19).
T-19	Resilience	Scoring unavailability yields last-known-good queue with staleness flag (EH-5, AC-18).
T-20	Resilience	Mid-job failure leaves prior scores valid and the job retryable (EH-4).
T-21	Security	Cross-account read and write attempts are rejected (NFR-6, AC-20).
T-22	Security	Override and weighting changes produce audit records with actor and before/after values (NFR-7).
T-23	Accessibility	Keyboard-only navigation and non-colour-dependent state labels (NFR-14, AC-21, AC-22).
T-24	Sync	Offline override capture and reconnect reconciliation, including conflicts (AC-16, AC-17).
T-25	Retention	Score history respects the 90-day / 500-entry bound (NFR-9).

Open questions

ID	Question	Why it needs the user, not the designer
OQ-1	What should the default weighting profile be (relative emphasis of deadline vs. importance vs. effort vs. age)?	This encodes the product's opinion about how users *should* work; the designer must not decide it unilaterally.
OQ-2	Should overdue tasks always occupy the Critical tier, or only up to a configurable number so that a large backlog does not flatten the tier's meaning?	Changes visible behaviour and prioritisation philosophy.
OQ-3	How much of the factor breakdown should be exposed to the user — full weights and contributions, or a simplified natural-language summary only?	Trade-off between trust/transparency and cognitive load; a product decision.
OQ-4	Is indefinite deferral allowed, or must every deferral have an expiry?	Affects whether tasks can be permanently hidden from ranking.
OQ-5	What is the intended queue maximum before pagination, and is a finite "Today" queue (e.g., top 10) the primary surface?	Determines the primary UX surface and the meaning of rank.
OQ-6	Should feedback signals persist indefinitely or expire entirely after the decay window?	Affects long-term ranking personalisation and audit expectations.

| OQ-7 | Confirm the data region(s) in which priority data must reside, and whether any cross-region replication is permitted. | Regulatory/product constraint that the designer cannot assume. |

Priority

****must-have**** — F-1 is the core differentiator of the product: the "Today" queue's value depends entirely on intelligent ordering. Within F-1, the must-have subset is FR-1 through FR-16 plus FR-12, FR-13, FR-17, FR-21, FR-23 (scoring, ranked queue, factors, weighting, explainability, override with expiry and removal, urgency escalation, blocked handling, dependency unblocking, recompute triggers, deterministic ordering, tiers, filtering, deferral, staleness, cross-device consistency, explanation provenance), along with NFR-1 through NFR-9, NFR-12, NFR-14, and US-1 through US-11 and US-14.

****should-have**** — FR-18 through FR-20, FR-22, FR-24, FR-25 (bulk recalculation, score history, drift feedback, offline override capture, queue limits, settings reset), NFR-13, NFR-15, and US-12, US-13, US-15.

****nice-to-have**** — Trend visualisation derived from DM-6/DM-4 history and any additional ranking factors beyond the five baseline factors in FR-3.

F-2: Quick Add

> ****Feature id:**** F-2 · ****Owned global id ranges:**** FR-26..FR-50, NFR-16..NFR-30, US-16..US-30 · ****Local ids**** (BR-, EC-, AC-, API-) are scoped to this feature and numbered sequentially from 1.

Requirements

****Functional Requirements (global ids — owned range FR-26..FR-50)****

| ID | Name | Description |

|---|---|---|

| FR-26 | Single-input capture | A single free-text line is the only mandatory input; one confirm action (Enter or "Add") creates a task. Reachable from every primary screen without opening a multi-field form. |

| FR-27 | Natural-language date/time parsing | The parser recognizes natural-language date/time expressions ("tomorrow 5pm", "next Monday", "in 2 hours", "2026-01-05 14:00") and binds the resolved value to the task's due date/time. |

| FR-28 | Priority & importance parsing | Recognizes inline priority markers (!, !, "high", "urgent", "p1") and maps them to the task's explicit-importance field consumed by F-1 prioritization. |

| FR-29 | Tag / project / context parsing | Recognizes `#tag` and `@context` tokens and attaches them to the task as tags/project; unrecognized tokens remain part of the title. |

| FR-30 | Parsed-field preview | Interpreted fields (title, due date/time, importance, tags, project) are rendered as distinct chips before/at save, so the user sees what the parser decided. |

| FR-31 | Inline correction of parsed fields | Any parsed chip can be edited or removed before saving without leaving the Quick Add input. |

| FR-32 | Defaults application | Fields not supplied or parsed inherit the user's configured defaults (default project, default due = none, default priority = none). |

| FR-33 | Rapid capture mode | After a successful save the input clears and stays focused so tasks can be added consecutively; Escape exits the mode. |

| FR-34 | Undo last capture | The most recent capture can be undone within a defined window, deleting the created task and restoring the original input text. |

FR-35	Offline capture & deferred sync	With no network, captures persist locally and queue; a pending-count badge is shown; queued items sync exactly once on reconnect with no data loss.
FR-36	Duplicate detection	On save, a near-duplicate of an open task's title triggers a non-blocking warning offering "Add anyway" or "Open existing".
FR-37	Keyboard-first & accessible operation	Fully operable by keyboard alone; screen readers announce parsed fields, save confirmation, and errors.
FR-38	Save confirmation	A brief toast confirms creation with an inline "Undo" action and an "Open task" link.

****Non-Functional Requirements (global ids — owned range NFR-16..NFR-30)****

ID	Category	Target
NFR-16	Performance — invocation	Input becomes visible and focused in <100 ms from shortcut/click.
NFR-17	Performance — parsing	Preview returned in <50 ms for inputs ≤200 characters.
NFR-18	Performance — save	Local task creation reflected in UI in <100 ms (optimistic); server ack must never block the input.
NFR-19	Scalability / availability	Sustains ≥20 captures/min per user without UI degradation; capture remains usable while offline.
NFR-20	Security	Input sanitized; task text is not sent to third-party parsers; parsing stays inside the app trust boundary; transport is TLS.
NFR-21	Data / residency	Captured text and parsed fields are stored in the user's configured data region; the on-device offline queue is encrypted at rest.
NFR-22	Deployment / environment	Works across supported browsers/OSes with no native-only APIs; degrades gracefully where notifications are unavailable.

****User Stories (global ids — owned range US-16..US-30)****

ID	Story
US-16	As a busy professional, I want to capture a task in one line, so that I don't lose the thought while switching contexts.
US-17	As a frequent typist, I want natural-language dates and priorities recognized, so that I can capture without filling a form.
US-18	As a mobile user, I want to capture tasks offline, so that a flaky connection never costs me a task.
US-19	As a careful user, I want to review and correct what the app parsed, so that a misread date never silently mis-schedules my work.
US-20	As a user, I want to undo a mistaken capture, so that a typo or wrong keystroke is recoverable instantly.

Behaviour

1. ****Invocation**** (FR-26): the user opens Quick Add via a global shortcut, a persistent "+" affordance, or the empty-state CTA. The field is focused and ready for input on open.
2. ****Typing**** (FR-27, FR-28, FR-29): as text is entered, the parser resolves date/time, importance markers, `#tags`, and `@contexts`. The title is the residual text after consumed tokens are removed.
3. ****Preview**** (FR-30): resolved fields appear as editable chips next to the input. Chips update live as the text changes.
4. ****Correction**** (FR-31): the user may click/backspace a chip to edit or remove it; corrections are reflected in the saved task and never mutate the visible title.
5. ****Confirm**** (FR-26): Enter or "Add" persists the task. Defaults fill any gaps (FR-32). A duplicate check runs before persistence (FR-36).

6. ****Feedback**** (FR-38): a toast confirms creation with "Undo" and "Open". The input clears and refocuses for the next task (FR-33).
7. ****Offline**** (FR-35): when disconnected, the task is created locally, marked pending, and flushed on reconnect via an idempotent queue.
8. ****Undo**** (FR-34): within the undo window, "Undo" or the shortcut removes the created task and restores the prior input text.

Business rules

ID	Rule
BR-1	A task is created only on explicit confirmation (Enter / "Add"); typed-but-unconfirmed text is never persisted.
BR-2	The task title is the input text minus tokens consumed by parsing (dates, importance markers, `#tags`, `@contexts`).
BR-3	If parsing would consume the entire input leaving no title, the raw text becomes the title and no field is parsed (no empty tasks).
BR-4	A parsed due date in the past is not auto-applied; it is surfaced for confirmation instead of silently scheduling in the past.
BR-5	Explicitly parsed values override configured defaults; defaults apply only when nothing is parsed or supplied.
BR-6	The importance marker maps to the explicit-importance input consumed by F-1; Quick Add must not set the pin/override state owned by F-1.

Validation

ID	Rule
BR-7	Title length is 1–500 characters after trimming; leading/trailing whitespace is trimmed.
BR-8	Line breaks are disallowed (single-line field); non-printable/control characters are stripped.
BR-9	Tags must match `#[p{L}p{N}_-]+`, ≤50 chars each, ≤20 tags per task.
BR-10	`@context` tokens must resolve to an existing project; a new project is created only with explicit user confirmation.
BR-11	Due date/time must be within the supported range (≥1970, ≤100 years ahead).
BR-12	Input longer than 500 characters is rejected inline before save and never silently truncated.

Edge cases

ID	Case & handling
EC-1	Ambiguous date ("next Friday" on a Friday): nearest future occurrence is chosen and shown as a chip for confirmation.
EC-2	Multiple date expressions: only the first is parsed; the remainder stays in the title.
EC-3	Time zone / DST boundary or user travel: due time is stored in the user's configured zone at capture time and displayed locally.
EC-4	Empty input + Enter: no-op, no error, no task.
EC-5	Metadata-only input (e.g., `#urgent`): falls back per BR-3.
EC-6	Repeated Enter presses: debounced to create exactly one task per confirmation.
EC-7	Offline queue at storage cap: new captures still accepted; user warned when local storage is exhausted.
EC-8	Intentionally identical captures: allowed after the duplicate warning (FR-36).

Error handling

| ID | Failure & handling |

|---|---|

| EC-9 | Parse engine/service failure: fall back to raw-text title + defaults; the task is still creatable and never blocked. |

| EC-10 | Save/network failure: task is retained in the offline queue and shown as "will sync"; it is never lost or double-created. |

| EC-11 | Sync conflict (task edited/deleted elsewhere): keep the captured copy and flag it for user resolution; never silently drop. |

| EC-12 | Undo after sync: undo issues a delete; if the task was modified post-sync, prompt before deleting. |

| EC-13 | Validation failure (BR-7..BR-12): inline error near the field, input preserved, save blocked until corrected. |

Acceptance criteria

| ID | Given / When / Then |

|---|---|

| AC-1 | ****Given**** the Quick Add field is focused, ****when**** the user types `Call Sam tomorrow 4pm` and presses Enter, ****then**** a task titled "Call Sam" with due date = tomorrow 16:00 (user zone) is created. |

| AC-2 | ****Given**** the user types a date expression, ****when**** parsing completes, ****then**** a due-date chip appears showing the resolved date/time. |

| AC-3 | ****Given**** the input contains `!!`, ****when**** saved, ****then**** the task's explicit importance is set to high and the chip reflects it. |

| AC-4 | ****Given**** the input contains `#billing`, ****when**** saved, ****then**** the tag "billing" is attached and shown as a chip. |

| AC-5 | ****Given**** a parsed chip is shown, ****when**** the user edits it, ****then**** the saved task reflects the corrected value and the title is unchanged. |

| AC-6 | ****Given**** no due date is supplied, ****when**** the task is saved, ****then**** the configured default due value is applied. |

| AC-7 | ****Given**** a task was just saved, ****when**** the toast appears, ****then**** the input is empty and focused for the next entry. |

| AC-8 | ****Given**** a task was just saved, ****when**** the user triggers Undo within the window, ****then**** the task is removed and the original input text is restored. |

| AC-9 | ****Given**** the device is offline, ****when**** the user captures a task, ****then**** it appears immediately with a pending-sync badge and is created exactly once after reconnect. |

| AC-10 | ****Given**** an open task with the same normalized title exists, ****when**** the user saves, ****then**** a non-blocking duplicate warning offers "Add anyway"/"Open existing" and no task is created until a choice is made. |

| AC-11 | ****Given**** keyboard-only use, ****when**** the user invokes Quick Add and saves, ****then**** every step is operable without a pointer and focus order is logical. |

| AC-12 | ****Given**** a task is created, ****when**** save succeeds, ****then**** a toast confirms creation and offers Undo and "Open". |

API behaviour

| ID | Endpoint / behaviour |

|---|---|

| API-1 | `POST /tasks` — creates a task from a Quick Add capture. Body: `{ rawText, title, dueAt?, importance?, tags[], projectId?, clientKey }`. Returns the created task (id, normalized fields). |

| API-2 | `POST /tasks/parse` (optional; may be client-side) — resolves free text to `{ title, dueAt, importance, tags[], projectId, confidence }` without persisting. |

| API-3 | `POST /tasks/sync` — flushes the offline queue as a batch with per-item idempotency keys; response reports accepted/duplicate/rejected per item. |

| API-4 | `DELETE /tasks/{id}` — used by Undo; supports an `If-Unmodified-Since` guard to avoid deleting a post-sync-modified task (EC-12). |

| API-5 | Idempotency — every create carries a client-generated `clientKey`; on a duplicate key the server returns the already-created task (HTTP 200) instead of a second task. |

| API-6 | Status codes — `400` validation (BR-7..BR-12), `409` near-duplicate/conflict, `422` unparseable-but-storable (falls back per EC-9), `429` rate limit, `5xx` with offline-queue fallback (EC-10). |

Priority

Must-have. Quick Add is the primary low-friction capture path that feeds the F-1 prioritization queue; without it the product's core loop (capture → prioritize → act) cannot function, and NFR-16..NFR-22 all gate on invocability and offline durability.

F-3: Focus Mode

> **Feature id:** F-3 · **Owned global id ranges:** FR-51..FR-75, NFR-31..NFR-45, US-31..US-45 · **Local ids** (BR-, V-, EC-, EH-, AC-, API-, DM-, T-, OQ-) are scoped to this feature and numbered sequentially from 1. References to F-1 / F-2 ids are plain text only; this section never re-defines them.

Requirements

Functional Requirements (global ids — owned range FR-51..FR-75)

ID	Name	Description
FR-51	Enter focus on a task	The user can start a focus session bound to a single existing task from the task detail view, the "Today" queue (F-1), or a quick-add result (F-2).
FR-52	Default session length	Sessions run for a configurable default duration (built-in default 25 minutes) expressed in whole minutes.
FR-53	Custom session length	Before or during a session the user can set a custom duration within the supported range (see V-1).
FR-54	Timer controls	The user can start, pause, resume, extend, and end a session. Every state change is reflected immediately in the UI.
FR-55	Single active session	At most one focus session may be active at a time; starting a new one while another is active requires an explicit decision (see BR-3).
FR-56	Pomodoro cycles	The user can run repeating work/break cycles: a work block, a short break, and after a configurable number of work blocks a long break.
FR-57	Break handling	During a break the task context is preserved; ending a break early is allowed and immediately starts the next work block.
FR-58	Focus time logging	All elapsed focus time (excluding paused intervals) is recorded against the linked task with a start timestamp, end timestamp, and duration.
FR-59	Session completion prompt	When the timer reaches zero the system notifies the user and prompts to mark the task complete, extend, or take a break.

| FR-60 | Interruption record | If the user pauses, leaves the app, or ends a session before completion, the session is recorded as an interruption with a user-selectable reason (optional). |

| FR-61 | Focus queue | The user can queue a sequence of tasks for consecutive focus sessions; when one session ends the next queued task is offered. |

| FR-62 | Distraction suppression | While a session runs, the app suppresses its own non-critical notifications and any in-app content not related to the focused task. |

| FR-63 | Do-not-disturb indicator | A persistent, dismissible indicator shows that focus is active, and its state is exposed to the operating system's DND mechanism where available. |

| FR-64 | Ambient sound | The user can optionally play a provided ambient/white-noise loop during a session, with an independent volume control. |

| FR-65 | Focus statistics | The user can view focus time per day/week and per task, plus session counts and completion rate. |

| FR-66 | Daily focus goal | The user can set a daily focus-time goal and see progress toward it. |

| FR-67 | Session persistence | An active session survives app backgrounding, navigation, and restart; on relaunch the timer resumes with correct remaining time. |

| FR-68 | Auto-start option | The user can opt in to automatically start breaks and the next work block without a manual confirm step. |

| FR-69 | Keyboard and shortcut control | Every timer control has a keyboard shortcut and, where available, a global shortcut or widget entry point. |

| FR-70 | Manual time correction | The user can add or subtract elapsed focus time on a recorded session to correct for time spent away. |

| FR-71 | Focus mode theming | Focus mode renders in a reduced, low-distraction layout showing only the timer, the task title, and controls. |

| FR-72 | Exit focus mode | Exiting focus mode returns the user to their previous location and, if a session was active, records/handles it per FR-58 and FR-60. |

| FR-73 | Multi-task attribution | Focus time can be split across two tasks when the user switches the linked task mid-session; each segment is recorded against its task. |

| FR-74 | Focus reminders | The user can schedule optional reminders to start a focus session at configured times. |

| FR-75 | Session notes | After a session the user can attach a short free-text note to the recorded session. |

Non-Functional Requirements (global ids — owned range NFR-31..NFR-45)

| ID | Category | Target |

|---|---|---|

| NFR-31 | Performance | Entering focus mode and rendering the first frame completes in < 150 ms on the reference device and < 400 ms on a low-end device. |

| NFR-32 | Performance | The countdown timer drifts by no more than ± 250 ms over a 60-minute continuous session. |

| NFR-33 | Scalability / availability | Focus sessions run fully offline; locally recorded time is retained and reconciled when connectivity returns. |

| NFR-34 | Security | Focus session data and notes are accessible only to the owning authenticated user; no cross-user leakage in logs or telemetry. |

| NFR-35 | Data / residency | Session records (timestamps, duration, task id, note) are retained for at least 24 months and are exportable/deletable by the user. |

| NFR-36 | Deployment / environment | Works on desktop and mobile web plus the packaged desktop/mobile shells; no feature requires hardware not present in the baseline environment. |

| NFR-37 | Accessibility | All timer controls are keyboard reachable, the remaining time is exposed via an ARIA live region (polite), and the countdown is not conveyed by color alone. |

| NFR-38 | Power | Ambient sound and the running timer must not prevent device sleep beyond what the platform requires for an actively running timer. |

| NFR-39 | Cross-device | Starting/ending a session on one device propagates state to the user's other signed-in devices within 10 s when online. |

| NFR-40 | Notification reliability | End-of-session and break notifications are delivered on time (± 2 s) when the app is backgrounded and notifications are permitted. |

| NFR-41 | Internationalization | Durations, dates, and times respect the user's locale and 12/24-hour preference; all copy is externalized for translation. |

| NFR-42 | Reliability | No recorded session is lost on crash/restart; at most 5 s of elapsed time may be lost in the worst case. |

| NFR-43 | Concurrency | Concurrent session starts from two devices resolve deterministically to a single active session (see BR-3). |

| NFR-44 | Observability | Session start/pause/end and timer-drift anomalies emit structured, PII-free telemetry. |

| NFR-45 | Usability | A first-time user can start a focus session in ≤ 2 interactions from the task view; controls remain operable with one hand on mobile. |

****User Stories (global ids — owned range US-31..US-45)****

- ****US-31:**** As a busy professional, I want to start a focus session on a task so that I work on it without distraction.
- ****US-32:**** As a busy professional, I want to choose how long a session lasts so that it fits my available time.
- ****US-33:**** As a busy professional, I want to pause and resume so that short interruptions do not break my session.
- ****US-34:**** As a busy professional, I want repeating work/break cycles so that I maintain sustainable attention.
- ****US-35:**** As a busy professional, I want the app to stop distracting me during focus so that I stay on task.
- ****US-36:**** As a busy professional, I want my focus time recorded against the task so that my history reflects real effort.
- ****US-37:**** As a busy professional, I want a prompt when the timer ends so that I know to stop or continue.
- ****US-38:**** As a busy professional, I want to queue the next task so that I flow from one session to the next.
- ****US-39:**** As a busy professional, I want optional ambient sound so that I can mask background noise.
- ****US-40:**** As a busy professional, I want to see my focus statistics so that I can understand my habits.
- ****US-41:**** As a busy professional, I want a daily focus goal so that I stay accountable.
- ****US-42:**** As a busy professional, I want my session to survive a restart so that I do not lose my progress.
- ****US-43:**** As a busy professional, I want to correct recorded time so that the history is accurate.
- ****US-44:**** As a busy professional, I want to switch the task mid-session so that unplanned work is still tracked.
- ****US-45:**** As a keyboard-first user, I want shortcuts for all timer controls so that I never leave the keyboard.

Behaviour

1. ****Entry:**** Focus mode can be entered from the task detail view, from a "Today" queue item, or from the result of a quick add. Entry requires a linked task; the UI offers task selection if none is supplied (see EH-2).
2. ****Session lifecycle:**** A session moves through `idle` → running → (paused ■ running) → completed | ended-early | abandoned`. Transitions are user-triggered except `running` → completed` (timer reaches zero) and `running` → abandoned` (see EC-6).

3. **Countdown.** While ``running`` the remaining time decreases in real time; while ``paused`` it is frozen. The displayed value derives from an absolute end timestamp so that backgrounding cannot desynchronize it.
4. **Cycles.** In cycle mode a work block is followed by a short break; after N completed work blocks (default 4, configurable) a long break replaces the short one. Cycle count and phase are shown in the UI.
5. **Completion.** At zero the app fires an end-of-session notification (NFR-40) and presents the completion prompt (FR-59). If auto-start (FR-68) is enabled, the next phase begins after a short grace period unless the user intervenes.
6. **Logging.** On every terminal transition the elapsed `*running*` time (pauses excluded) is written as one or more session records, attributed to the linked task(s) per FR-73, and optionally annotated with a note (FR-75).
7. **Suppression.** For the duration of ``running``, non-critical in-app notifications are queued rather than shown and are released when the session ends.
8. **Restoration.** On relaunch the app reads persisted session state; a session whose end time has already passed is completed retroactively with the elapsed time capped at the configured duration.

Business rules

- **BR-1:** A focus session must always reference exactly one linked task at any instant; task reassignment is recorded as a segment boundary.
- **BR-2:** Only time in the ``running`` state counts toward focus totals and statistics; ``paused`` and break intervals are excluded from focus totals but breaks are recorded separately.
- **BR-3:** If a session is already active, starting another is blocked until the user chooses one of: `*end current` and `*start new*`, `*queue new task*`, or `*cancel*`.
- **BR-4:** Aggregate focus time per session may not exceed the configured duration by more than the user-approved extension amount.
- **BR-5:** Break minutes are recorded but never counted toward the daily focus goal (FR-66).
- **BR-6:** A session record is immutable once written, except for the note and the manual correction permitted by FR-70, which is stored as an adjustment rather than an overwrite.
- **BR-7:** Focus statistics are computed from session records, never from derived counters, so that corrections and splits are always reflected.
- **BR-8:** Long-break eligibility resets when the cycle count resets (new day or explicit reset by the user).
- **BR-9:** Distraction suppression never blocks safety-critical, security, or system-level notifications.
- **BR-10:** Default durations are user-scoped settings; changing them does not alter sessions already recorded.

Validation

- **V-1:** Session duration must be a whole number of minutes within the inclusive range 1–180. Values outside the range are rejected inline with the allowed range shown.
- **V-2:** Break duration must be a whole number of minutes within the inclusive range 1–60.
- **V-3:** Work-block count before a long break must be an integer in the inclusive range 1–12.
- **V-4:** Manual time correction (FR-70) must keep total net focus time for the session ≥ 0 and ≤ 24 hours; empty or non-numeric input is rejected.
- **V-5:** A session note is at most 500 characters; leading/trailing whitespace is trimmed; an empty note is treated as "no note".
- **V-6:** Focus reminders (FR-74) require a valid time-of-day in the user's locale and at least one selected weekday.
- **V-7:** Daily focus goal must be a whole number of minutes in the inclusive range 1–1440.

- **V-8:** A segment produced by mid-session task switching (FR-73) must have a positive duration; zero-length segments are discarded silently.

Edge cases

- **EC-1:** Timer reaches zero while the app is backgrounded or closed → on resume, the completion prompt is shown and the session is finalized (see Behaviour 8).
- **EC-2:** Device clock changes or is manually altered while a session runs → remaining time is recomputed from the persisted absolute end timestamp using monotonic elapsed time; a negative remaining time clamps to zero and finalizes the session.
- **EC-3:** A session spans midnight → the record keeps its true start/end timestamps; statistics attribute focus time to the calendar day in which the time was actually spent.
- **EC-4:** Daylight-saving transition during a session → elapsed time is computed from monotonic duration, not wall-clock subtraction.
- **EC-5:** User pauses and resumes repeatedly → each interval is tracked; total paused time is reported separately and excluded from focus totals (BR-2).
- **EC-6:** User ends a session with less than 1 minute elapsed → the session is discarded and not stored, to avoid noise records.
- **EC-7:** Linked task is deleted mid-session → the session continues; the recorded segment is attributed to a "deleted task" placeholder that preserves the historical duration.
- **EC-8:** Linked task is completed mid-session → a non-blocking prompt offers to end the session or continue; continuing keeps attribution.
- **EC-9:** Task switched mid-session and switched back → two segments for the same task are created and summed in statistics (BR-7).
- **EC-10:** Ambient sound interrupted by an incoming call or another app's audio focus → sound pauses and resumes automatically when focus returns; the timer keeps running.
- **EC-11:** Two devices start sessions concurrently (NFR-43) → the earlier start wins; the later device shows the active session and offers *join* or *end-and-restart*.
- **EC-12:** User is in the middle of a break when the daily goal is reached → the goal indicator updates but the break is not truncated.
- **EC-13:** Free/busy calendar integration is unavailable (no data source configured) → no focus-time suggestion is shown; the manual picker remains fully functional. No mock data is substituted.

Error handling

- **EH-1:** Timer state fails to persist locally → the session continues in memory, a non-blocking warning is shown, and a retry is scheduled; if persistence still fails at session end, the record is held in memory for the remainder of the app run and the user is warned that it may be lost.
- **EH-2:** Session entered without a linked task → the app shows a task picker; dismissing it returns to the previous screen without starting a session.
- **EH-3:** End-of-session notification permission denied → the app falls back to an in-app alert shown on next app focus, and offers a link to notification settings.
- **EH-4:** Ambient sound asset fails to load or decode → sound is disabled, an inline message is shown, and the timer is unaffected (no silent failure).
- **EH-5:** Manual time correction rejected by validation (V-4) → inline error under the field, correction not applied, session unchanged.
- **EH-6:** Session sync to the server fails → the local record is retained and queued for retry (NFR-33); the user sees a "will sync" indicator rather than an error.
- **EH-7:** Statistics cannot be computed because records are still syncing → partial totals are shown with a "some sessions pending sync" label; no fabricated numbers are displayed.

- ****EH-8:**** Unhandled failure during focus mode → the app exits focus mode, restores the previous screen, preserves any locally written session record, and reports the failure without exposing internal detail.

Acceptance criteria

- ****AC-1:**** Given a task, when the user starts focus mode, then a session begins with the configured default duration and the task title is visible in the reduced layout. (FR-51, FR-52, FR-71)
- ****AC-2:**** Given an active session, when the user pauses and resumes, then the countdown stops and restarts and paused intervals are excluded from recorded focus time. (FR-54, BR-2)
- ****AC-3:**** Given a session running for 60 minutes, when measured against a reference clock, then drift is $\leq \pm 250$ ms. (NFR-32)
- ****AC-4:**** Given an active session, when the timer reaches zero, then a notification is delivered and a prompt offering complete / extend / break is shown. (FR-59, NFR-40)
- ****AC-5:**** Given cycle mode with 4 work blocks, when the 4th work block completes, then a long break starts instead of a short break. (FR-56, BR-8)
- ****AC-6:**** Given an active session, when a non-critical in-app notification is generated, then it is queued and released only after the session ends. (FR-62, BR-9)
- ****AC-7:**** Given a session ends after 20 minutes, when the user views the task, then 20 minutes of focus time is attributed to that task. (FR-58)
- ****AC-8:**** Given a running session, when the app is killed and relaunched, then the session resumes with the correct remaining time. (FR-67, NFR-42)
- ****AC-9:**** Given a session in progress, when the user switches the linked task, then two segments are recorded and each is attributed to its task. (FR-73, EC-9)
- ****AC-10:**** Given an active session, when the user starts a second session, then a blocking choice of end-and-start / queue / cancel is presented and only one session remains active. (FR-55, BR-3)
- ****AC-11:**** Given an ended session, when the user applies a -5 minute correction, then focus totals reflect the adjustment and the original record is unchanged. (FR-70, BR-6)
- ****AC-12:**** Given a session started from the task view, when the user operates entirely by keyboard, then all controls are reachable and the remaining time is announced via a polite live region. (NFR-37, FR-69)
- ****AC-13:**** Given a session under 1 minute, when the user ends it, then no session record is stored. (EC-6)
- ****AC-14:**** Given a daily goal of 120 minutes and 90 recorded focus minutes, when the user views the goal, then progress shows 90/120 and break minutes are excluded. (FR-66, BR-5)
- ****AC-15:**** Given no connectivity, when a session is run and completed, then the record is stored locally and syncs when the connection returns. (NFR-33, EH-6)

API behaviour

- ****API-1 — Start session.**** `POST /focus-sessions`` with `{ taskId, plannedDurationMinutes, cycleConfig? }` → `201`` returning `{ id, taskId, state: "running", startedAt, endsAt, plannedDurationMinutes }`. `409`` if an active session already exists (BR-3); `422`` if `taskId`` is missing/unknown or duration violates V-1.
- ****API-2 — Read current session.**** `GET /focus-sessions/active`` → `200`` with the active session and its segments, or `204`` when none is active. Must be safe and idempotent.
- ****API-3 — Transition state.**** `PATCH /focus-sessions/{id}`` with `{ action: "pause" | "resume" | "end" | "extend", extendMinutes?, reason? }` → `200`` with the updated session. Invalid transitions (e.g. resume while running) return `409`` with the current state.
- ****API-4 — Switch linked task.**** `POST /focus-sessions/{id}/segments`` with `{ taskId }` → `201`` closing the current segment and opening a new one; `422`` for a zero-length segment (V-8).

- **API-5** — Correct recorded time. **POST** /focus-sessions/{id}/adjustments` with `{ deltaMinutes, note? }` → `201`; `422` if the resulting net time violates V-4.
- **API-6** — Attach note. **PATCH** /focus-sessions/{id}` with `{ note }` → `200`; `422` if the note exceeds 500 characters (V-5).
- **API-7** — List history. **GET** /focus-sessions?from=&to=&taskId=&cursor=` → `200` with a cursor-paginated list of session records, excluding sub-minute discarded sessions (EC-6).
- **API-8** — Statistics. **GET** /focus-stats?granularity=day|week&from=&to=` → `200` with `{ focusMinutes, breakMinutes, sessionCount, completionRate }`; must be recomputed from records (BR-7). While records are pending sync the response includes `pendingSync: true` (EH-7).
- **API-9** — Settings. **GET/PUT** /focus-settings` for default duration, break durations, cycle count, auto-start, goal, reminders, ambient-sound preference; `422` on any value failing V-1, V-2, V-3, V-6, or V-7.
- **API-10** — Offline queue. All mutating calls above accept a client-generated idempotency key; retries with the same key must not create duplicate sessions or segments. Conflict resolution follows BR-3 and NFR-43 (earliest start wins).
- **API-11** — Errors. All endpoints return a stable machine-readable `code`, a human-readable `message`, and no PII in `code`. Auth failures return `401`; authorization failures `403`; unknown session `404`.

Priority

- **Must-have:** FR-51, FR-52, FR-54, FR-55, FR-58, FR-59, FR-62, FR-67, FR-71, FR-72, NFR-31, NFR-32, NFR-33, NFR-34, NFR-40, NFR-42.
- **Should-have:** FR-53, FR-56, FR-57, FR-60, FR-63, FR-65, FR-68, FR-70, FR-73, FR-75, NFR-35, NFR-37, NFR-39, NFR-41, NFR-43, NFR-45.
- **Nice-to-have:** FR-61, FR-64, FR-66, FR-69, FR-74, NFR-36, NFR-38, NFR-44.

F-4: Progress Analytics

> **Feature id:** F-4 · **Owned global id ranges:** FR-76..FR-100, NFR-46..NFR-60, US-46..US-60 · **Local ids** (BR-, V-, EC-, EH-, AC-, API-, DM-, T-, OQ-) are scoped to this feature and numbered sequentially from 1. References to F-1 / F-2 / F-3 ids are plain text only; this section never re-defines them.

Requirements

Functional Requirements (global ids — owned range FR-76..FR-100)

ID	Name	Description
FR-76	Analytics dashboard access	A dedicated "Progress" view is reachable from primary navigation and presents the user's personal productivity metrics on one scrollable surface.
FR-77	Completion metrics	The dashboard shows total tasks completed, total tasks created, and net change (created – completed) for the selected period.
FR-78	Completion rate over time	Completion rate (completed ÷ created, expressed as a percentage) is shown for the selected period and plotted as a time series at a granularity implied by the period length.
FR-79	Task throughput	Tasks completed per day / week / month are computed and displayed as bar or line series for the selected period.
FR-80	Completion streak	A current streak and a longest streak of consecutive active days (days with ≥1 completed task) are tracked and displayed, with the streak definition visible to the user.

| FR-81 | Focus-time analytics | Total focused minutes, session count, and completed-vs-interrupted session ratio are derived from focus sessions (F-3) and displayed for the selected period. |

| FR-82 | Focus distribution | Focused minutes are broken down by associated task, tag/project, and time of day, so the user can see where their focus time went. |

| FR-83 | Prioritization effectiveness | The dashboard reports whether tasks that F-1 ranked high were completed sooner than tasks it ranked low (e.g. average rank-to-completion lag, top-of-queue completion share). |

| FR-84 | Deferral & reschedule analysis | Count and proportion of tasks whose due date was postponed (rescheduled), and average postponement count per task, are reported for the period. |

| FR-85 | Overdue analysis | Overdue tasks (past due, not completed) are counted and shown as a trend and as a current snapshot, grouped by age bucket. |

| FR-86 | Productivity heatmap | A day-of-week × hour-of-day heatmap shows completion density, derived from completion timestamps, so the user can identify their most productive windows. |

| FR-87 | Weekly review summary | A one-screen weekly digest summarizes the week's completions, focus time, streak status, top completed tag/project, and outstanding overdue count. |

| FR-88 | Date-range selection | The user can select the analytics time window via presets (today, this week, this month, last 7/30/90 days, this year) and a custom start/end date range. |

| FR-89 | Analytics filtering | Analytics can be scoped by tag, project/context, priority band (F-1 importance), completion status, and task origin (manual vs. quick-add per F-2). |

| FR-90 | Grouping / segmentation | The user can group analytics by tag, project/context, priority band, or origin, and see per-group breakdowns of the core metrics. |

| FR-91 | Goal definition | The user can define personal goals such as "complete N tasks per day/week" or "focus M minutes per day/week". |

| FR-92 | Goal progress tracking | Each defined goal shows current progress toward its target for the current interval, with an on-track / at-risk / achieved state. |

| FR-93 | Period-over-period comparison | The user can compare the selected period against the immediately preceding equal-length period, with deltas shown per metric (absolute and %). |

| FR-94 | Chart visualization | Each metric is rendered as a chart appropriate to its type (time series for trends, bar for throughput, heatmap for the productivity grid); charts show axis labels, units, and hover/tap values. |

| FR-95 | Data export | The user can export the currently selected metrics and period to a machine-readable file (CSV) and a printable view, respecting the active filters. |

| FR-96 | Data freshness indicator | The dashboard displays the timestamp of the last successful data refresh and auto-refreshes on view load, or offers a manual refresh action. |

| FR-97 | No-data / first-run guidance | When there is insufficient data for a metric, the surface shows an explanatory empty state and the minimum input needed to populate it, rather than a misleading zero. |

| FR-98 | Insight callouts | The dashboard surfaces plain-language, deterministic callouts derived from the data (e.g. "You completed 30% more tasks than last week") with the underlying numbers shown. |

| FR-99 | Customizable dashboard | The user can choose which metric widgets appear and in what order, and the layout persists per user across sessions and devices. |

| FR-100 | Historical window & retention | Analytics are computed over a bounded historical window; the dashboard states the window covered and does not present metrics as if complete when data is partial or truncated. |

Non-Functional Requirements (global ids — owned range NFR-46..NFR-60)

ID	Area	Target / description
NFR-46	Performance (page)	The Progress dashboard reaches first meaningful render in <2.5s on a mid-range mobile device over 3G with a 12-month dataset.
NFR-47	Performance (aggregation)	Any single metric aggregation responds in <800ms p95 and <1.5s p99 for the maximum supported window; the UI shows a loading state beyond 300ms.
NFR-48	Scalability (data volume)	Analytics remain responsive for accounts with up to 50,000 tasks and 20,000 focus sessions; longer histories are aggregated server-side rather than shipped raw to the

client. |

| NFR-49 | Availability | The Progress view degrades gracefully and returns a usable shell when the aggregation backend is unavailable; the rest of the app remains fully functional. |

| NFR-50 | Security / access control | Analytics data is private to the authenticated owner; every request is authorized against the requesting user and never returns another user's data. |

| NFR-51 | Privacy | Analytics are computed from the user's own data only; no cross-user or third-party profiling is performed and no task content is sent to external analytics providers. |

| NFR-52 | Data residency | Analytics storage and computation respect the project's configured data-residency region; derived aggregates inherit the residency of their source data. |

| NFR-53 | Data accuracy / freshness | Derived metrics reconcile with source records within the documented freshness window (default ≤ 5 minutes after a mutation), and the freshness timestamp is user-visible. |

| NFR-54 | Degraded / offline | When the network is unavailable the last cached dashboard remains viewable with a stale-data indicator and export is disabled until connectivity returns. |

| NFR-55 | Internationalization | Dates, week start day, numbers, and percentages are formatted in the user's locale; durations use the user's 12/24-hour preference; RTL layout is supported. |

| NFR-56 | Accessibility | Charts provide an accessible text/table alternative; metrics are conveyed without relying on color alone; the view is fully keyboard navigable and satisfies WCAG 2.1 AA contrast. |

| NFR-57 | Cross-device consistency | The same metric definitions and dashboard layout yield consistent values across devices; layout customization (FR-99) syncs per user. |

| NFR-58 | Export integrity | Exported files match the on-screen values for the same filters/period within rounding tolerances, use a documented schema, and are valid/unopenable-free. |

| NFR-59 | Deployment / environment | Metric definitions, default history window, and aggregation cadence are environment-configurable and consistent across dev/staging/prod. |

| NFR-60 | Observability | Aggregation failures, latency, and cache hit rates are logged/metriced; anomalies (e.g. sudden zero-metric regressions) are alertable. |

User Stories (global ids — owned range US-46..US-60)

| ID | Story |

|----|-----|

| US-46 | As a busy professional, I want to see how many tasks I completed this week, so that I know whether I am keeping up with my workload. |

| US-47 | As a user, I want to see my completion rate over time, so that I can tell whether I am becoming more or less consistent. |

| US-48 | As a user, I want to see my current and longest streak, so that I stay motivated to keep a daily habit. |

| US-49 | As a user, I want to see how much focused time I accumulated, so that I understand the quality of my work, not just the quantity. |

| US-50 | As a user, I want to see where my focus time went, so that I can rebalance my effort between projects. |

| US-51 | As a user, I want to know whether prioritizing my tasks actually helped me finish the important ones first, so that I can trust or adjust my prioritization. |

| US-52 | As a user, I want to see how often I postpone tasks, so that I can spot chronic procrastination patterns. |

| US-53 | As a user, I want to see my overdue tasks trend, so that I can intervene before a backlog builds up. |

| US-54 | As a user, I want a heatmap of when I complete the most work, so that I can schedule demanding tasks at my peak hours. |

| US-55 | As a user, I want a concise weekly review, so that I can reflect in under a minute. |

| US-56 | As a user, I want to filter analytics by tag or project, so that I can evaluate one area of my life at a time. |

| US-57 | As a user, I want to set and track goals, so that I have a concrete target to aim for. |

US-58	As a user, I want to compare this period to the last, so that I can see whether things are improving.
US-59	As a user, I want to export my metrics, so that I can keep a personal record or share it.
US-60	As a user, I want to choose which metrics I see, so that my dashboard reflects what matters to me.

Behaviour

- On open, the Progress view loads the metric set for the default period (configurable; default "this week"), honoring persisted filters and layout from FR-99.
- Every metric is computed from the same underlying event records: task creation, completion, due-date mutation (reschedule), and focus-session start/end (F-3). Metrics are read-only views; this feature never mutates task or session data.
- Changing the date range (FR-88) or filters (FR-89) recomputes all visible metrics atomically against the new scope; widgets enter a loading state and do not display mixed-scope values.
- Period comparison (FR-93) requires an equal-length preceding window; when the preceding window predates the retention window, the delta is shown as "insufficient history" rather than a fabricated value.
- Goal progress (FR-92) is evaluated against the current interval boundary (day/week) using the same completion/focus events as the base metrics; state derives from projected pace vs. elapsed fraction of the interval.
- Insight callouts (FR-98) are deterministic functions of displayed metrics — no generative or speculative content — and always cite the numbers they are based on.

Business rules

- **BR-1** — A task counts as "completed in period P" if and only if its completion timestamp falls within P, regardless of its creation date.
- **BR-2** — A task counts as "created in period P" if and only if its creation timestamp falls within P. Completion rate (FR-78) = $\text{completed-in-P} \div \text{created-in-P}$; when created-in-P is 0 the rate is undefined and shown as "—", never as 0% or 100%.
- **BR-3** — Streak (FR-80): consecutive calendar days (in the user's locale and week-start convention) with ≥ 1 completed task. A gap of one or more zero-completion days resets the current streak; the longest streak is the maximum run ever recorded within the retention window.
- **BR-4** — Overdue (FR-85): a task is overdue when its due timestamp is in the past, it has no completion timestamp, and it is not explicitly marked cancelled. Overdue age buckets are defined by whole days past due.
- **BR-5** — Focus time (FR-81) counts only completed or interrupted session time actually elapsed, not the planned duration; a session paused and resumed is counted once with net elapsed time. Cancelled sessions with zero elapsed time are excluded.
- **BR-6** — Prioritization effectiveness (FR-83) compares the F-1 rank a task held at the start of the selected period against its completion lag; tasks with no F-1 rank (unscored/inactive) are excluded from this metric and the excluded count is disclosed.
- **BR-7** — Deferral (FR-84) counts each change to a task's due date that moves it later as one postponement; moving a due date earlier is not counted. Multiple postponements of the same task count separately.
- **BR-8** — Deleted tasks are excluded from all current-period totals going forward but are retained in already-materialized historical aggregates per the retention policy (FR-100) so past reports do not silently change.
- **BR-9** — All percentages are rounded for display (one decimal) but computed from unrounded counts; totals in a grouped breakdown (FR-90) must sum to the ungrouped total within rounding tolerance.

- **BR-10** — The "top completed tag/project" in the weekly review (FR-87) is chosen by completed-task count; ties are broken alphabetically for determinism.

Validation

- **V-1** — Custom date range: both start and end are required, $\text{start} \leq \text{end}$, and neither may be later than "now"; the maximum span is the configured retention window (default 365 days).
- **V-2** — Goal definition (FR-91): target value is a positive integer; focus-minute goals must be a multiple of 1 minute and within the retention-bounded weekly maximum; a task-count goal of 0 is rejected.
- **V-3** — Filters: an empty filter selection is treated as "all"; an explicitly selected tutorial that yields no matching tasks produces the no-data state (FR-97), not an error.
- **V-4** — Export (FR-95) requires a non-empty result set for the current scope; exporting an empty scope is disabled with an explanatory message.
- **V-5** — Date inputs are validated in the user's locale format and normalized to UTC internally; invalid or unparseable dates are rejected inline.
- **V-6** — Dashboard layout (FR-99) must contain at least one widget; a layout with all widgets removed is rejected with guidance to add at least one.

Edge cases

- **EC-1** — Zero tasks created in the period but tasks completed (backlog burn-down): completion rate is undefined (BR-2) and shown as "—" with an explanation, while throughput (FR-79) still displays.
- **EC-2** — Time-zone boundary: a task completed at 23:30 local that falls on the next UTC day is attributed to the user's local day for streak and heatmap purposes (FR-80, FR-86).
- **EC-3** — First-ever use / brand-new account: all widgets render FR-97 empty states with the minimum number of completed tasks needed to populate each metric.
- **EC-4** — A focus session spanning midnight splits elapsed time across the two local days proportionally for the heatmap and daily focus metrics.
- **EC-5** — Tasks with no due date are included in completion/throughput/streak metrics but excluded from overdue (FR-85) and never counted as deferrals (FR-84).
- **EC-6** — User changes locale or week-start convention mid-window: the view recomputes with the new convention and the change is reflected in the streak definition display (BR-3).
- **EC-7** — Very long history: when the requested window exceeds the retention boundary, the dashboard clamps to the boundary and clearly labels the truncated coverage (FR-100).
- **EC-8** — A goal target exceeds any achievable value within the interval (e.g. 100 completions/day): the goal is stored but permanently shown "at-risk"; no error.
- **EC-9** — Comparison window partially outside retention: deltas are suppressed and labeled "insufficient history" rather than computed from partial data (FR-93).
- **EC-10** — Concurrent edit during load: if a task is completed while the dashboard is open, the freshness indicator updates and the user is offered a refresh (FR-96) rather than the view silently mutating mid-read.

Error handling

- **EH-1** — Aggregation backend unavailable: the view renders the last successfully cached dashboard with a persistent stale-data banner; a retry action is offered; export is disabled (NFR-54).
- **EH-2** — Partial aggregation failure (one widget's data fails while others succeed): the failing widget shows an inline error with retry, and the rest of the dashboard remains usable.
- **EH-3** — Request timeout: the widget shows a timeout state with retry; no partial/inconsistent numbers are displayed for that widget.

- **EH-4** — Unauthorized request: the user is returned to authentication; no analytics data is rendered or cached for an unauthenticated session.
- **EH-5** — Export generation failure: an error toast is shown, the on-screen view is unaffected, and the user can retry; partial files are never offered for download.
- **EH-6** — Invalid persisted layout (e.g. from an older version): the layout is validated, invalid widgets are dropped with a non-blocking notice, and the user is returned to a valid dashboard rather than an error page.

Acceptance criteria

- **AC-1** — Given a period with 10 tasks created and 7 completed locally, when the user opens the Progress view for that period, then completed shows 7, created shows 10, and completion rate shows 70.0%.
- **AC-2** — Given a period in which zero tasks were created, when the dashboard renders, then completion rate displays "—" with an explanation and never shows 0% or 100%.
- **AC-3** — Given completions on 5 consecutive days followed by a zero-completion day, when streaks render, then the current streak shows 0 and the longest streak shows 5.
- **AC-4** — Given focus sessions totaling 130 elapsed minutes in the period, when the focus metrics render, then total focused minutes shows 130 and the session count matches the number of counted sessions per BR-5.
- **AC-5** — Given a custom range of 30 days, when the user changes a filter to a single tag, then all widgets recompute to that tag's scope and every widget reflects the same scope (no mixed values).
- **AC-6** — Given the user compares this week to last week, when both windows have data, then each metric shows an absolute and percentage delta consistent with the underlying values.
- **AC-7** — Given a goal of 5 completions/day and 3 completed at midday, when the goal widget renders, then it shows progress 3/5 and an "at-risk" or "on-track" state derived from elapsed interval fraction.
- **AC-8** — Given the currently selected metrics and filters, when the user exports, then the CSV contains exactly those metrics/rows and its values match the screen within rounding tolerance (BR-9).
- **AC-9** — Given insufficient data for a metric, when the widget renders, then it shows an FR-97 empty state naming the inputs needed, not a zero value.
- **AC-10** — Given a screen-reader user, when the dashboard renders, then every chart exposes an accessible text/table alternative and all controls are keyboard operable (NFR-56).
- **AC-11** — Given the aggregation backend is unreachable, when the user opens the view, then the last cached dashboard renders with a stale indicator and export is disabled (NFR-54, EH-1).
- **AC-12** — Given the user removes all widgets, when the layout is saved, then the change is rejected with guidance to keep at least one widget (V-6).

API behaviour

> Endpoints describe observable behaviour only; transport, storage, and framework choice belong to the Architect agent. All endpoints are owner-scoped and reject requests for other users' data.

- **API-1** — `GET /analytics/summary` — params: `from`, `to`, `filters`, `groupBy`. Returns the scalar metrics (completed, created, net, completionRate, streaks, focus totals, overdue count, deferral count) for the scope. `completionRate` is `null` when undefined (BR-2).
- **API-2** — `GET /analytics/timeseries` — params: `metric`, `from`, `to`, `granularity` (day/week/month), `filters`. Returns an ordered series of `{bucket, value}`; buckets align to the user's week-start convention and local day boundaries (EC-2).
- **API-3** — `GET /analytics/heatmap` — params: `from`, `to`, `filters`. Returns a 7×24 matrix of completion density with local-day/hour attribution (FR-86, EC-2, EC-4).

- **API-4** — `GET /analytics/breakdown` — params: `groupBy` (tag|project|priorityBand|origin), `from`, `to`, `filters`. Returns per-group values plus an `excluded` count for records lacking the grouping attribute; group values reconcile to the ungrouped total (BR-9).
- **API-5** — `GET /analytics/insights` — params: `from`, `to`, `filters`. Returns deterministic callouts (FR-98), each with `text` and the `metricValues` it cites; never returns generative content.
- **API-6** — `GET /analytics/goals` / `PUT /analytics/goals/{id}` — read and define personal goals; write validates per V-2 and returns the persisted goal.
- **API-7** — `GET /analytics/export` — params: `from`, `to`, `filters`, `metrics`. Returns a CSV with a documented schema (FR-95); responds `409`/disabled state when the scope is empty (V-4) or the view is in degraded mode (EH-1).
- **API-8** — `GET /analytics/preferences` / `PUT /analytics/preferences` — read/write the persisted dashboard layout and default period (FR-99); payload validated against V-6 and the supported widget set.

Response conventions: successful responses include `asOf` (freshness timestamp, FR-96) and `coverage` (`{from, to, truncated}` per FR-100). Scope-invalid requests (V-1, V-3) return a validation error identifying the offending parameter; authorization failures return a non-disclosing error and never leak metric values (NFR-50).

Priority

Must-have: FR-76, FR-77, FR-78, FR-79, FR-80, FR-81, FR-88, FR-94, FR-96, FR-97, FR-100 — the core dashboard, completion/streak/focus metrics, range selection, charting, freshness, and honest no-data/coverage handling.

Should-have: FR-82, FR-83, FR-84, FR-85, FR-87, FR-89, FR-90, FR-92, FR-93, FR-98 — segmentation, prioritization/deferral/overdue insight, weekly review, filtering, goal tracking, comparison, and callouts.

Nice-to-have: FR-86, FR-91 (goal definition UI beyond defaults), FR-95, FR-99 — heatmap, custom goal authoring, export, and dashboard customization.

Open questions

- **OQ-1** — Default analytics history window and retention boundary are unspecified in the product plan; a default of 365 days is assumed (FR-100, NFR-59). Confirm the intended retention window.
- **OQ-2** — Whether the streak should follow calendar days or "active days with logged activity" is ambiguous; BR-3 assumes calendar days. Confirm.
- **OQ-3** — Should analytics include tasks completed via any route only, or also count quick-add-created tasks separately (F-2 origin grouping, FR-90)? Confirm whether origin segmentation is required at launch.
- **OQ-4** — Export format scope: is CSV + printable view sufficient, or is a second machine-readable format expected? Assumption recorded in FR-95.

F-5: Integration Hub

> **Feature id:** F-5 · **Owned global id ranges:** FR-101..FR-125, NFR-61..NFR-75, US-61..US-75 · **Local ids** (BR-, EC-, AC-, API-, DM-, T-, OQ-) are scoped to this feature and numbered sequentially from 1. References to F-1 / F-2 / F-3 / F-4 ids are plain text only; this section never re-defines them.

Requirements

Functional Requirements (global ids — owned range FR-101..FR-125)

| ID | Name | The feature must... |

---|---|---

| FR-101 | Connection catalogue | Present a browsable catalogue of connectable source types (calendar, email, issue/backlog tracker, code host, messaging, file store, generic webhook bridge) with a plain-language description of what data each type can read and write. |

| FR-102 | Connect account | Let the user start and complete an authorization handshake with a provider without typing provider credentials into the product's own fields; the connection appears in the user's connection list only after a successful callback. |

| FR-103 | Connection lifecycle management | List all connections with label, type, account identity, state (`connected`, `degraded`, `expired`, `error`, `paused`, `disconnected`) and last-sync time; support rename/label, pause, resume, reconnect, disconnect, and delete. |

| FR-104 | Credential storage, refresh, revocation | Store credentials/tokens securely outside user-visible surfaces, refresh them automatically before expiry, and revoke them on disconnect; never expose secret values in the UI, logs, or exports. |

| FR-105 | Connection health & diagnostics | Provide a per-connection health view: state, last successful sync, next scheduled sync, consecutive-failure count, last error in plain language, and a "test connection" action. |

| FR-106 | Multiple accounts per provider | Allow more than one account of the same provider type to be connected simultaneously, each with a distinct user-assigned label, independent scope, direction, and mappings. |

| FR-107 | Sync direction per connection | Let the user choose inbound-only, outbound-only, or bidirectional sync independently per connection and per scope. |

| FR-108 | Field mapping | Let the user map external fields to task fields (title, due date/time, explicit importance, tags/project, notes/description, status/completion, assignee) and define a default for unmapped fields. |

| FR-109 | Sync scope filters | Let the user include/exclude by external container (calendar, folder, list, project, label, saved query) and by rule (e.g., only items with a due date, only items assigned to me). |

| FR-110 | Inbound item ingestion | Create or update tasks in the product from external items that match the connection's scope, preserving a stable link to the source item; imported due dates are treated as first-class deadlines by the prioritization feature (F-1), and imported completion events count toward the analytics feature (F-4). |

| FR-111 | Outbound task propagation | Push task create, update, complete, reopen, and delete events to the external system according to the connection's direction, scope, and mapping. |

| FR-112 | Field-level conflict policy | Resolve concurrent edits per field using a user-selected rule: product wins, external wins, most-recent-change wins, or always-ask; the chosen rule is visible and changeable per connection. |

| FR-113 | Conflict detection & resolution inbox | Detect conflicts, hold the losing value (never silently discard), and present a conflict inbox where the user can keep the product value, keep the external value, merge per field, or apply the resolution to all similar conflicts. |

| FR-114 | Idempotent sync | Guarantee that retries, duplicate events, and restarts never create duplicate tasks or duplicate outbound writes; every external item maps to at most one task. |

| FR-115 | Deletion, unlink & tombstone semantics | Define and enforce what happens when an external item is deleted, moved out of scope, or the connection is removed: the user pre-selects "keep as standalone task", "archive", or "delete", and no task is destroyed without that rule or an explicit confirmation. |

| FR-116 | Sync cadence & manual trigger | Run scheduled syncs at a per-connection configurable interval and expose a "Sync now" action that works regardless of the schedule. |

| FR-117 | Event-driven near-real-time updates | Where a provider supports change events, apply changes from those events with a scheduled-sync fallback so that lag targets in NFR-61 hold even when events are missed or delayed. |

| FR-118 | Sync progress & completion report | Show live progress for a running sync (items seen, created, updated, skipped, failed) and a completion summary with a link into the activity log. |

| FR-119 | Sync activity log / audit trail | Keep a per-connection, per-item chronological record of what changed, in which direction, by which rule, and with which outcome, filterable by date range, direction, and outcome. |

| FR-120 | Failure handling, retry & notification | Classify failures (auth expired, permission denied, rate-limited, provider unavailable, invalid mapping, payload rejected), retry transient classes with backoff, stop retrying permanent classes, and notify the user through the product's notification surface when action is required. |

| FR-121 | Rate-limit & quota handling | Detect provider throttling, defer and re-queue affected work without dropping it, report a `throttled` state on the connection, and never exhaust the provider quota with retry storms. |

| FR-122 | Selective pause | Pause and resume an entire connection or a single scope without deleting mappings, filters, or history; while paused, no inbound changes are applied and no outbound writes are sent. |

| FR-123 | Configuration export/import | Export connection configuration (label, direction, scope, mapping, conflict policy) as a portable file that contains no credentials or tokens, and import it to recreate a connection that then requires fresh authorization. |

| FR-124 | Permission scoping & transfer disclosure | Request only the access scopes the configured mapping and direction require, show the user exactly which fields will be read and written before the connection is activated, and require re-consent before any scope expansion takes effect. |

| FR-125 | Generic outbound webhooks & inbound endpoint | Allow user-defined outbound webhook subscriptions on task events with configurable payload fields and a shared secret, and accept inbound external payloads on a per-user endpoint that are mapped to task fields; both directions respect dedupe, auth, and the activity log. |

Non-Functional Requirements (global ids — owned range NFR-61..NFR-75)

| ID | Category | Target |

|---|---|---|

| NFR-61 | Performance — sync latency | Event-driven inbound change becomes visible in the product within 30 s at p95; scheduled-only connections within 60 s of the schedule tick at p95. |

| NFR-62 | Performance — sync throughput | A manual sync of 5,000 scoped items completes within 5 minutes; a manual sync request is acknowledged within 1 s. |

| NFR-63 | Scalability | Support at least 10 connections per user, 100,000 tracked external items per connection, and horizontal scaling of sync workers without per-user manual sharding. |

| NFR-64 | Availability & isolation | 99.9% monthly availability for the integration control plane; a provider outage or integration failure must not block task creation, prioritization (F-1), focus sessions (F-3), or analytics (F-4) for unaffected users. |

| NFR-65 | Security — secrets at rest & in transit | Tokens and shared secrets encrypted at rest with a managed key, TLS 1.2+ in transit, secrets redacted from logs, error messages, telemetry, and configuration exports. |

| NFR-66 | Security — authorization | Least-privilege scopes, per-connection revocation, and re-consent on scope expansion; a disconnect makes the stored credential unusable within 60 s. |

| NFR-67 | Data residency & retention | Integration configuration and sync logs stored in the deployment region declared for the product; sync logs retained per a configurable window (default 90 days) and connection data purged within 30 days of delete. |

| NFR-68 | Privacy & consent | No data is transmitted to a provider unless the user has activated a connection whose disclosed mapping covers that field; the disclosure shown at activation matches the fields actually transmitted. |

| NFR-69 | Observability | Every sync operation carries a correlation id; success rate, lag, queue depth, and error class are measured per connection and per provider, with alert thresholds on sustained failure or lag regression. |

NFR-70	Reliability & idempotency	At-least-once delivery with dedupe keys yielding exactly-once effect; a process restart resumes without duplicate tasks and without losing queued changes.
NFR-71	Resilience to provider failure	Circuit breaker per provider, exponential backoff with jitter, bounded retry window, and graceful degradation to scheduled sync when events fail.
NFR-72	Rate-limit fairness	Global and per-connection concurrency caps are configurable; queued work is scheduled fairly so no single connection starves others.
NFR-73	UI responsiveness	Integration list, health, conflict inbox, and activity log views are interactive within 2 s at p95 on a 3G-equivalent connection and remain usable with 1,000 log rows via paging or virtualization.
NFR-74	Deployment & environment	Connection definitions and sandbox/test credentials are environment-configurable; non-production environments never hold production secrets; schema migrations are reversible and integration contracts remain backward compatible for at least two released versions.
NFR-75	Locale & time-zone correctness	External timestamps are normalized to UTC internally and rendered in the user's locale and time zone; date-only external values keep the intended calendar day without time-zone shifting, including across DST boundaries.

****User Stories (global ids — owned range US-61..US-75)****

- ****US-61**** — As a busy professional, I want to connect my calendar so that deadline-bearing events appear as tasks automatically.
- ****US-62**** — As a busy professional, I want to connect my issue tracker and email inbox so that assignments become tasks without retyping.
- ****US-63**** — As a user, I want to choose which calendars, lists, and projects sync so that I am not flooded with irrelevant tasks.
- ****US-64**** — As a user, I want to control which fields sync and in which direction so that my data stays consistent with the tool I actually work in.
- ****US-65**** — As a user, I want conflicting edits resolved by my chosen rule so that I do not have to referee every change.
- ****US-66**** — As a user, I want a conflict inbox where I can resolve ambiguous cases myself so that nothing is silently overwritten.
- ****US-67**** — As a user, I want to trigger a sync on demand and watch its progress so that I know exactly when my data is current.
- ****US-68**** — As a user, I want an activity log of every synced change so that I can audit what happened and why.
- ****US-69**** — As a user, I want to be told when a connection breaks or expires so that I can restore it before my task list goes stale.
- ****US-70**** — As a user, I want to pause a connection temporarily without losing its configuration so that I can silence a noisy source and re-enable it later.
- ****US-71**** — As a user, I want to disconnect an account and have its access revoked so that I stay in control of my data.
- ****US-72**** — As a user, I want to connect two accounts of the same provider (work and personal) with separate labels so that contexts stay apart.
- ****US-73**** — As a user, I want imported deadlines to influence my priority queue exactly like native deadlines so that my "Today" list is trustworthy.
- ****US-74**** — As a user, I want to open the original external item from a task so that I can act on it in its own context.
- ****US-75**** — As a user, I want tasks created by quick add (F-2) and tasks used in a focus session (F-3) to carry their external link so that context is never lost.

Behaviour

1. **Connection creation.** The user picks a source type (FR-101), starts authorization (FR-102), is shown the exact fields and direction the connection will use (FR-124), completes the handshake, and lands on a configuration screen where scope (FR-109), direction (FR-107), mapping (FR-108), conflict policy (FR-112), cadence (FR-116), and deletion semantics (FR-115) are set before activation.
2. **Steady-state sync loop.** For each active scope the hub evaluates the source set, classifies each external item as new, changed, unchanged-out-of-scope, or deleted (FR-110, FR-115), applies the field mapping, resolves conflicts per policy (FR-112/FR-113), writes idempotently (FR-114), and records an activity entry per decision (FR-119).
3. **Outbound loop.** Task lifecycle events raised anywhere in the product (including F-2 quick add and F-3 focus completion) are matched against connections with outbound direction and a covering scope, mapped, and written to the provider with dedupe keys (FR-111, FR-114).
4. **Event acceleration.** Where the provider publishes change events, those events enqueue the same work units as scheduled sync; scheduled sync always remains the reconciliation backstop (FR-117).
5. **Failure loop.** Failures are classified, transient classes retried with backoff, permanent classes surfaced to the user, degraded connections flagged in the connection list, and dependent scopes continue independently (FR-120, NFR-71).
6. **Cross-feature integration.** A task linked to an external item renders its source badge and deep link in the task detail view and in focus mode (F-3); its due date and mapped importance participate in prioritization scoring (F-1) under the same rules as native values; its completion state flows into completion metrics (F-4) exactly once.

Business Rules

- **BR-1** — A connection becomes active only after authorization succeeds, at least one scope is selected, and a field mapping is valid; partial configurations persist as drafts and never sync.
- **BR-2** — Only one task may be linked to a given external item; additional matches update the existing task rather than creating a second one.
- **BR-3** — When the same field is edited in the product and externally between two sync passes, the connection's conflict policy decides the winner; a losing value is always preserved and reversible for the retention window.
- **BR-4** — Deletion semantics are declared before activation; if the user has not declared them, the safe default is "keep as standalone task" with the external link marked broken.
- **BR-5** — A paused connection (FR-122) sends no outbound writes. Changes made while paused are reconciled on resume using the same conflict policy; the resume summary states how many items were reconciled.
- **BR-6** — Disconnecting revokes and destroys the stored credential; already-imported tasks remain subject to the declared deletion semantics, and the user is told the resulting effect before confirming.
- **BR-7** — Sync never overwrites user-authored content with an empty external value; an omitted external field is treated as "no change", not as a deletion, unless the mapping explicitly declares "external authoritative for this field".
- **BR-8** — Imported tasks keep their source attribution permanently visible; removing the connection does not remove provenance from tasks that were kept.
- **BR-9** — Webhook subscriptions (FR-125) are scoped to the owning user's data only; inbound payloads may never address another user's tasks.
- **BR-10** — Configuration export (FR-123) contains no secrets, and import always requires fresh authorization before the imported connection can sync.
- **BR-11 (validation)** — A sync interval must be a whole number of minutes within the supported minimum and maximum; a value below the provider's own minimum is rejected with the provider minimum stated.
- **BR-12 (validation)** — A scope must select at least one container or rule; the empty scope is rejected as "would import nothing".

- ****BR-13 (validation)**** — A field mapping must map the title-equivalent field; a mapping that leaves the title unmapped cannot be saved as active.
- ****BR-14 (validation)**** — A conflict policy must name exactly one rule per field; two rules for the same field are rejected as contradictory.
- ****BR-15 (validation)**** — A webhook secret must meet the minimum length and entropy policy defined for the deployment, and a subscription without a secret cannot be enabled.
- ****BR-16 (validation)**** — A connection label must be unique within the user's connection list and non-empty.

Edge Cases

- ****EC-1**** — The external item's title is empty or whitespace only; the task title falls back to the declared title-default rule rather than creating an untitled task.
- ****EC-2**** — The external item has a date-only due value; it must not shift to the previous or next day when rendered (NFR-75).
- ****EC-3**** — An external item moves out of scope (relabelled, reassigned) — it is unlinked per the declared out-of-scope rule, not deleted silently.
- ****EC-4**** — The same external item is reachable through two scopes on the same connection; it is ingested once, with the union of matched scopes recorded.
- ****EC-5**** — The same provider account is connected twice under different labels; items are ingested once and attributed to the earliest active connection, with a duplicate-connection warning shown at setup.
- ****EC-6**** — A very large first sync (over the NFR-62 throughput envelope) proceeds in resumable batches with visible progress, and a refresh mid-sync does not restart it.
- ****EC-7**** — The user deletes a task that is linked and outbound-synced; the external side follows the declared delete-propagation rule, and if that rule is "ask", the user is prompted once.
- ****EC-8**** — The user edits a task while an inbound update for the same task is in flight; the in-flight change lands in the conflict inbox rather than overwriting.
- ****EC-9**** — The provider returns a payload with unexpected or additional fields; unknown fields are ignored, the known fields apply, and the event is logged as partially mapped.
- ****EC-10**** — A webhook is delivered out of order; ordering is not assumed, and the most-recent-change rule (or the configured rule) decides the final state.
- ****EC-11**** — Clock skew between the provider and the product pushes an external timestamp into the future or the past; timestamps are stored as received with the skew recorded in the activity log.
- ****EC-12 (error handling)**** — Authorization expires or is revoked externally: state becomes `expired`, scheduled work stops for that connection only, and the user is notified with a one-action reconnect path.
- ****EC-13 (error handling)**** — The user revokes access at the provider while a sync is running: the run terminates for that connection, no partial state is left inconsistent, and the connection moves to `disconnected`.
- ****EC-14 (error handling)**** — The provider is unavailable or returns 5xx: the circuit opens, work is re-queued, and the connection shows `degraded` rather than failing the whole hub.
- ****EC-15 (error handling)**** — The provider rate-limits or refuses the request: work is deferred with backoff, the connection shows `throttled`, and no item is dropped (NFR-72).
- ****EC-16 (error handling)**** — Outbound write is rejected as invalid (forbidden field, closed project): the item is quarantined, the task stays intact in the product, and the user is told which task and which reason.
- ****EC-17 (error handling)**** — An inbound webhook fails authentication or dedupe: it is rejected with no side effect and counted in a rejection metric.
- ****EC-18 (error handling)**** — Malformed or partially valid import configuration is loaded: valid sections apply, invalid sections are listed with reasons, and nothing activates until the user resolves them.

Error Handling

- **User-visible errors** name the connection, the affected item count, the reason in plain language, and a single primary recovery action (Reconnect, Adjust scope, Fix mapping, Retry, Open conflict).
- **Transient errors** are never surfaced as user errors while automatic retry is still within its window; only exhaustion or a permanent class triggers notification (FR-120).
- **Input errors** (BR-11..BR-16, EC-18) are reported inline against the offending field and block activation of the affected scope, not activation of unrelated scopes.
- **Data-safety errors** (EC-12..EC-16) must always leave the product-side task in a valid, readable state with provenance retained, even when the external write or read failed.

Acceptance Criteria

- **AC-1** — Given a valid provider account, when the user completes authorization and sets scope, direction, mapping, and conflict policy, then the connection appears as `connected` and the first sync starts within 60 s. (FR-101, FR-102, FR-103)
- **AC-2** — Given an active inbound connection, when a new matching external item is created and the provider supports events, then a linked task exists within 30 s at p95; when only scheduled sync is available, within one interval. (FR-110, FR-117, NFR-61)
- **AC-3** — Given a task linked to an outbound-synced connection, when the user edits its due date, then the external item's due date matches within one sync cycle. (FR-111)
- **AC-4** — Given a connection in bidirectional mode, when the same field is changed on both sides between sync passes, then the declared policy determines the stored value and the losing value is retrievable from the conflict record. (FR-112, BR-3)
- **AC-5** — Given a conflict record exists, when the user keeps the external value, then the task shows that value and no further conflict is raised for the same change. (FR-113)
- **AC-6** — Given a sync run is interrupted by a restart, when it resumes, then no duplicate task exists and every queued change is applied exactly once. (FR-114, NFR-70)
- **AC-7** — Given an external item is deleted, when the declared deletion rule applies, then the resulting task state matches the rule and the activity log records the decision. (FR-115, BR-4)
- **AC-8** — Given a connection is paused, when task changes occur, then no outbound writes are sent, and on resume a reconciliation summary reports the applied changes. (FR-122, BR-5)
- **AC-9** — Given the provider credential is revoked, when the next sync attempts to run, then the connection enters `expired`, other connections continue unaffected, and the user receives a reconnect notification. (FR-120, EC-12, NFR-64)
- **AC-10** — Given the provider throttles requests, when retries occur, then no item is dropped, the connection reports `throttled`, and retry traffic stays within the configured concurrency caps. (FR-121, EC-15, NFR-72)
- **AC-11** — Given the user opens a task imported from a provider, when they activate the source link, then the external item opens in the provider's own surface. (US-74, FR-110)
- **AC-12** — Given an imported task with an external due date, when the prioritization queue is computed, then that deadline affects the ranking exactly as a native due date would. (FR-110, US-73)
- **AC-13** — Given the user disconnects an account, when the disconnect completes, then the stored credential is unusable within 60 s and kept tasks retain their source attribution. (FR-104, EC-13, NFR-66, BR-8)
- **AC-14** — Given an activity log with more than 1,000 entries, when the user filters by date range and outcome, then the filtered result renders within the NFR-73 budget. (FR-119, NFR-73)
- **AC-15** — Given a configuration export is produced, when it is inspected, then it contains no credentials or tokens, and importing it requires fresh authorization before sync. (FR-123, BR-10)
- **AC-16** — Given a webhook subscription is enabled, when a matching task event occurs, then exactly one signed payload is delivered; an inbound payload replayed with the same dedupe key has no effect. (FR-125, BR-9, EC-17)

API Behaviour

Transport and concrete protocol are the Architect's decision; the semantics below are binding on whatever interface is chosen. All operations are user-scoped, authorize against the owning user's session, and return a correlation id usable in the activity log.

| ID | Operation | Semantics |

|---|---|---|

| API-1 | List available source types | Returns catalogue entries with the read/write fields each type supports and the scopes it will request. |

| API-2 | Begin authorization | Returns a handshake reference and the destination the user must visit; creates a pending, non-syncing connection draft. |

| API-3 | Complete authorization | Callback/redeem step; on success the draft becomes a configured connection, on failure it returns a classified reason and remains a draft. |

| API-4 | List / read connections | Returns every connection with label, type, state, direction, scope summary, last sync, next sync, and consecutive-failure count. |

| API-5 | Update connection | Accepts label, direction, scope, mapping, conflict policy, cadence, and deletion rule; validates atomically per BR-11..BR-16 and rejects the whole change set on any invalid member. |

| API-6 | Pause / resume connection or scope | Idempotent; resume returns the reconciliation summary. |

| API-7 | Test connection / read health | Performs a live probe; returns state, latency, and the last error in plain language without leaking secrets. |

| API-8 | Disconnect / delete connection | Revokes the credential, applies the declared deletion semantics to retained tasks, and purges connection data per NFR-67. |

| API-9 | Trigger sync | Enqueues a run (or a scoped re-sync); returns a run id immediately and never blocks for completion. |

| API-10 | Read run status | Returns phase, counts (seen, created, updated, skipped, failed, conflicted), and a terminal outcome. |

| API-11 | Read activity log | Cursor-paged, filterable by date range, direction, outcome, and item; entries are read-only and never mutated. |

| API-12 | Resolve conflict | Applies a keep-product / keep-external / per-field-merge decision; the decision is recorded and the resulting value is synced per the connection's direction. |

| API-13 | Inbound webhook receive | Authenticates the caller, dedupes by key, validates the payload, maps it to task fields, and returns a non-retryable status for permanently invalid payloads so senders stop retrying. |

| API-14 | Manage webhook subscriptions | Create/list/update/disable subscriptions; secrets are write-only and returned masked. |

| API-15 | Export / import configuration | Export omits all secrets; import validates and stages configuration, requiring authorization before activation. |

Data Model (concepts; storage is the Architect's decision)

- ****DM-1 Connection**** — owner, source type, label, account identity, state, direction, cadence, deletion rule, timestamps.

- ****DM-2 Credential record**** — connection reference, secret material, expiry, refresh metadata; never readable through any user-facing interface.

- ****DM-3 Scope**** — connection reference, container/rule selector, direction override, pause flag.

- ****DM-4 Field mapping**** — connection reference, external field, task field, authority flag, defaults.

- ****DM-5 External item link**** — connection, external item id, task reference, last seen version, provenance state (linked, broken, out-of-scope).

- **DM-6 Sync run** — connection, trigger type (schedule, manual, event), phase, counts, outcome, correlation id.
- **DM-7 Activity entry** — run reference, external item, task reference, direction, rule applied, outcome, timestamp.
- **DM-8 Conflict record** — task, field, product value, external value, detected time, state, resolution.
- **DM-9 Webhook subscription** — owner, target, event filter, secret reference, enabled flag, delivery stats.
- **DM-10 Provider definition** — source type capabilities, field catalogue, required scopes, minimum interval, event support.

Priority

| Priority | Requirements |

|---|---|

| **Must-have** | FR-101, FR-102, FR-103, FR-104, FR-105, FR-107, FR-108, FR-109, FR-110, FR-111, FR-112, FR-113, FR-114, FR-115, FR-116, FR-118, FR-119, FR-120, FR-121, FR-122, FR-124 |

| **Should-have** | FR-106, FR-117, FR-123, FR-125 |

| **Nice-to-have** | None in this feature; every requirement above is in scope and none may be dropped without explicit user approval. |

Feature-level priority: **must-have** (the hub is the mechanism by which external commitments enter the user's "Today" queue).

Traceability

| Requirement | Acceptance | Interface | Data | Notes |

|---|---|---|---|

| FR-101, FR-102 | AC-1 | API-1, API-2, API-3 | DM-1, DM-2, DM-10 | T-1: connect flow end-to-end against a sandbox provider account. |

| FR-103, FR-105 | AC-1, AC-9 | API-4, API-6, API-7 | DM-1 | T-2: lifecycle and health state-transition tests. |

| FR-104, FR-124 | AC-13 | API-8, API-15 | DM-2 | T-3: secret-leak scan over logs, errors, and exports. |

| FR-107, FR-108, FR-109 | AC-3 | API-5 | DM-3, DM-4 | T-4: direction/scope/mapping matrix tests. |

| FR-110, FR-117 | AC-2, AC-11, AC-12 | API-9, API-10 | DM-5, DM-6 | T-5: ingest latency and cross-feature ranking tests (F-1). |

| FR-111 | AC-3 | API-9 | DM-5 | T-6: outbound propagation for create/update/complete/reopen/delete. |

| FR-112, FR-113 | AC-4, AC-5 | API-12 | DM-8 | T-7: conflict-policy table tests for all four rules. |

| FR-114 | AC-6 | API-9 | DM-5, DM-6 | T-8: duplicate-event and restart-injection tests. |

| FR-115 | AC-7 | API-8 | DM-5 | T-9: deletion-rule matrix tests for all three semantics. |

| FR-116, FR-118, FR-122 | AC-2, AC-8 | API-6, API-9, API-10 | DM-6 | T-10: cadence, progress, and pause/resume reconciliation tests. |

| FR-119 | AC-14 | API-11 | DM-7 | T-11: activity-log completeness and filter performance tests. |

| FR-120, FR-121 | AC-9, AC-10 | API-7, API-10 | DM-6, DM-7 | T-12: fault-injection tests for each classified error class. |

| FR-123 | AC-15 | API-15 | DM-1, DM-3, DM-4 | T-13: export/import round-trip without secrets. |

| FR-125 | AC-16 | API-13, API-14 | DM-9 | T-14: signed-delivery, dedupe, and rejection tests. |

| NFR-61..NFR-75 | AC-2, AC-6, AC-10, AC-13, AC-14 | API-* | DM-* | T-15: non-functional verification plan owned by the Architect/Implement stages. |

Open Questions (for the USER — not decided by this agent)

- **OQ-1** — Which source types must be in the launch catalogue (FR-101), and in what order? The catalogue content is a scope decision for the user; this section assumes calendar, email/issue tracker, messaging, and generic webhook bridge are all in scope.
- **OQ-2** — Is true bidirectional write-back required for every connected source type, or is inbound-only acceptable for some types (for example email)? The spec above delivers bidirectional capability per connection and treats it as in scope.
- **OQ-3** — Which deployment region(s) must hold integration configuration and sync logs (NFR-67), and does any provider or user segment impose a residency restriction the spec must reflect?
- **OQ-4** — What is the required default retention for sync logs (assumed 90 days) and for kept tasks after a connection is deleted (assumed retained as standalone tasks)?
- **OQ-5** — Are multiple accounts of the same provider required at launch (FR-106 is marked should-have), or is one account per provider acceptable initially?
- **OQ-6** — Are outbound webhooks and inbound generic endpoints (FR-125) required at launch, or may they follow the first release? They remain in scope in this specification.

F-6: Dark Mode

> **Feature id:** F-6 · **Owned global id ranges:** FR-126..FR-150, NFR-76..NFR-90, US-76..US-90 · **Local ids** (BR-, EC-, AC-, API-, DM-, OQ-) are scoped to this feature and numbered sequentially from 1. References to F-1 / F-2 / F-3 / F-4 / F-5 ids are plain text only; this section never re-defines them.

Requirements

Functional Requirements (global ids — owned range FR-126..FR-150)

ID	Name	Requirement
FR-126	Theme preference selection	The user can choose exactly one of `light`, `dark`, or `system` as the active theme preference. The preference governs the whole product — no per-screen or per-widget themes.
FR-127	System preference following	While the preference is `system`, the product resolves the effective theme from the host OS/browser colour-scheme signal and re-resolves live when that signal changes, with no reload and no user action.
FR-128	Preference persistence	The chosen preference is persisted and re-applied on the next session on the same device, including after sign-out/sign-in, hard reload, and full app restart.
FR-129	Immediate application	Changing the preference or the system signal re-themes every already-rendered surface by the next paint. A manual reload is never required and never prompted for.
FR-130	Full-surface coverage	Every screen and every state (empty, loading, error, success, partial) of F-1 through F-5 — including modals, sheets, toasts, tooltips, menus, skeletons, and overlays — renders from theme tokens. No shipped surface retains a hard-coded light-only colour.
FR-131	No flash of incorrect theme	The effective theme is resolved and applied before first paint. A user whose stored preference is `dark` never sees a light frame on cold load or hard reload.
FR-132	Per-device override	A user may hold a different theme preference on different devices; `system` is evaluated independently per device.
FR-133	Scheduled switching	The user can define a dark window — explicit start/end local times, or "follow sunset/sunrise" for the device's location — that overrides the base preference while the window is active. The schedule can be enabled, disabled, and edited at any time.
FR-134	Theme-aware assets	Illustrations, empty-state art, iconography, avatar fallbacks, and any raster art ship a dark variant or are specified as themeable vectors. No light-baked art is placed on a dark surface.

| FR-135 | Data-visualisation adaptation | Charts in F-4 Progress Analytics use a dark-safe categorical palette with theme-appropriate gridline, axis, and label colours; series remain distinguishable in both themes and never rely on colour alone. |

| FR-136 | Rich and third-party content | User-authored and embedded content (notes, links, code blocks, webhook payload previews, F-5 integration previews, third-party frames) is legible in both themes, with a documented fallback for content that supplies its own light colours. |

| FR-137 | Accessible toggle affordance | A theme control is reachable from settings and from the global quick-access/command surface on every primary screen. The control always reflects the current *effective* theme, not merely the stored preference. |

| FR-138 | Keyboard and shortcut parity | The theme can be changed without a pointer; the shortcut is discoverable in the product's shortcut/help listing alongside the actions of F-2 and F-3. |

| FR-139 | High-contrast and reduced-transparency variants | Dark mode offers a high-contrast dark variant that raises contrast beyond the base dark palette, and honours OS "reduce transparency" and "reduce motion" signals for any theme-transition animation. |

| FR-140 | Print and export | Printing and non-interactive export of any view produce a legible light output regardless of the on-screen theme, unless the user explicitly requests a themed export. |

| FR-141 | Theme-aware notifications | In-app notifications, banners, and badge colours are theme-aware; outbound email and push notification rendering uses a fixed light-safe palette that stays legible in dark mail clients and OS notification centres. |

| FR-142 | Native chrome synchronisation | Where the host platform allows it, surrounding chrome (browser `theme-color`, status bar, splash background) is set to the current effective theme and updated on every change. |

| FR-143 | Reset to default | The user can reset the theme preference to the shipped default (`system`) in one action; the reset takes effect immediately and clears any session-scoped override. |

Non-Functional Requirements (global ids — owned range NFR-76..NFR-90)

| ID | Area | Target / Measurement |

|---|---|---|

| NFR-76 | Performance — switch latency | Theme change is visible within 100 ms of the triggering input on the reference device class (mid-range mobile, 4× CPU throttle). Measured with a recorded input-to-paint trace. |

| NFR-77 | Performance — first paint | 100% of cold loads and hard reloads paint the correct effective theme. Wrong-theme frames = 0, verified by frame-by-frame screenshot capture of the load sequence. |

| NFR-78 | Accessibility — contrast | WCAG 2.1 AA in dark mode: text $\geq 4.5:1$ (normal) / $\geq 3:1$ (large), non-text UI $\geq 3:1$ against adjacent colours. High-contrast dark variant targets $\geq 7:1$ for body text. Verified by automated contrast audit plus manual spot checks. |

| NFR-79 | Performance — rendering cost | Entering or leaving dark mode produces no long task > 50 ms and contributes ≤ 0.01 to Cumulative Layout Shift. Steady-state dark mode shows no continuous repaint and no idle CPU above the light-mode baseline. |

| NFR-80 | Performance — startup and payload | Theme resolution adds no blocking network round-trip before first paint; the resolved theme and its token set are available from local assets/state. |

| NFR-81 | Availability — offline and degraded | Selection, persistence, scheduling, and switching all work with zero network connectivity. No theme state is lost or reverted when connectivity returns. |

| NFR-82 | Scalability — coverage | Theming propagates exclusively through tokens: adding a new screen or component requires no per-theme branch. Unthemed-surface count is measurable and driven to zero. |

| NFR-83 | Security | Theme preference is treated as an untrusted enum. No preference value, schedule expression, location input, or custom colour reaches the DOM/CSS without allow-list validation. Theming introduces no new injection sink and no new script-execution path. |

| NFR-84 | Data and residency | Theme preference and schedule are device-local user settings. If synced, only the enum and schedule are stored — never content — in the region required by the product's data-residency policy. |

| NFR-85 | Consistency — cross-device | When preference sync is enabled, convergence across a user's devices is eventual within one foreground sync cycle, with last-write-wins per device and no silent loss of any device's local override. |

| NFR-86 | Deployment and environment | Light and dark token sets are identical across local, staging, and production. No environment may ship a partial or stale token set; token-set version is reported in diagnostics. |

| NFR-87 | Auditability of coverage | A build-time check reports themed-token coverage for every shipped component. The release gate fails when any component references a raw colour literal. |

| NFR-88 | Assistive technology | Effective theme and theme changes are exposed with correct semantics; a change is announced to screen readers without moving focus. |

| NFR-89 | Internationalisation | Dark-mode assets and palettes do not break RTL layouts, and no asset contains baked-in text. Locale-specific date/time formatting in the schedule follows the user's locale. |

| NFR-90 | Observability | The distribution of resolved themes and the count of detected wrong-theme frames are measurable in aggregate. Telemetry captures no user content, only theme state, resolution source, and app version. |

****User Stories (global ids — owned range US-76..US-90)****

| ID | Story |

|---|---|

| US-76 | As a late-night worker, I want a dark theme so that the screen does not glare and my eyes stay comfortable in a dim room. |

| US-77 | As a user whose OS already follows my routine, I want the app to follow my system theme so that I do not manage the same setting twice. |

| US-78 | As a returning user, I want my theme choice remembered so that I do not re-set it every session. |

| US-79 | As a keyboard-first user, I want to toggle the theme without leaving the keyboard so that I stay in flow. |

| US-80 | As a low-vision user, I want a high-contrast dark option so that I can read comfortably without switching to light. |

| US-81 | As a user opening the app at night, I want the theme applied instantly with no white flash so that launch is not painful. |

| US-82 | As a user with a fixed evening routine, I want the app to switch to dark automatically at a time I choose so that I never have to think about it. |

| US-83 | As a user who shares screenshots and exports, I want charts and text to stay readable in whichever theme I use and in the files I hand to others. |

Behaviour

1. ****Resolution.**** The effective theme is computed from a strict precedence chain, highest first: (a) an in-session temporary override, (b) an active scheduled dark window, (c) the stored user preference, (d) the OS colour-scheme signal when the preference is `system`, (e) the shipped default (effectively light). The resolution source is always knowable and is exposed to the UI so the toggle can show the effective theme.
2. ****Applied before paint.**** On load, the stored preference and schedule are read from local state synchronously, the effective theme is resolved, and the correct token set is applied before the first paint (FR-131, NFR-77).
3. ****Live re-resolution.**** A change to any input in the chain — user toggle, OS signal, schedule boundary crossing, device timezone change — re-resolves the theme and re-applies it within one paint (FR-127, FR-129, NFR-76).
4. ****Token-mediated theming.**** Every surface consumes semantic tokens (surface, surface-raised, border, text-primary, text-muted, accent, status-*, chart-*). Switching themes swaps the token set; it never rewrites component styles. This is what makes FR-130 and NFR-82 hold for surfaces belonging

to F-1 through F-5 as well as F-6 itself.

5. **Non-destructive switching.** A theme change never interrupts an active focus session or its timer (F-3), never clears a partially typed quick-add input (F-2), never loses a pinned/override state (F-1), never resets chart zoom or filter state (F-4), and never cancels an in-flight integration sync or connection handshake (F-5).

6. **Assets.** Icons and illustrations are either single-colour vectors tinted by token, or they ship an explicit dark variant selected by the active token set. No component recolours a raster asset with a CSS filter as its only strategy.

7. **Scheduling.** When the schedule is enabled, the dark window is evaluated against the device's local time. "Sunset/sunrise" mode resolves to local sun times for the device's location/timezone; if no location is available the user is offered explicit times (see Edge cases EC-5). Crossing midnight is a supported, normal window.

8. **Native chrome.** On every effective-theme change the browser `theme-color`, status bar, and splash background are updated (FR-142); on platforms without support these calls are silent no-ops.

9. **Outbound surfaces.** In-app notifications follow the theme; email and push rendering use a fixed light-safe palette with sufficient contrast in dark clients (FR-141).

10. **Diagnostics.** Token-set version, resolution source, and any theme failure are recorded locally and available to the user-facing diagnostics view alongside the state of F-5 connections.

Business rules

- **BR-1 — One preference.** Exactly one preference value is active per device: `light`, `dark`, or `system`. Custom colour pickers, per-screen themes, and per-project themes are out of scope for this feature.

- **BR-2 — Precedence is fixed.** The chain in Behaviour §1 is the only permitted resolution order. No feature may inject a theme value outside it.

- **BR-3 — Toggle during a scheduled window.** An explicit toggle while a schedule window is active is treated as a session-scoped temporary override. It does not delete or edit the schedule, and it is cleared at the next window boundary or at the start of the next session.

- **BR-4 — `system` always resolves.** If the OS signal is unavailable, unreadable, or reports an unrecognised value, `system` resolves to light. The product is never left unstyled or in an undefined theme.

- **BR-5 — Sync never clobbers silently.** With preference sync enabled, a device keeps its local override until the user explicitly chooses "apply to all devices". A remote value never overwrites a device's local override without that explicit action.

- **BR-6 — Theme changes never block work.** A theme change is non-blocking and non-destructive with respect to every operation in F-1..F-5 (see Behaviour §5).

- **BR-7 — Content is invariant.** Dark mode changes colour, elevation, and asset variant only. It does not change wording, data, ordering, or information density. A user switching themes must not lose or gain information.

- **BR-8 — Status colour semantics are preserved.** The colours that signal overdue/done/blocked (F-1), streak and at-risk state (F-4), and connection state (F-5) retain their meaning and remain distinguishable in dark mode. Colour is never the sole carrier of a status; an icon or label accompanies it.

- **BR-9 — Tokens are the only colour source.** Feature components reference semantic tokens. A raw colour literal in a feature component is a defect, not a stylistic choice (enforced by NFR-87).

- **BR-10 — Exports default to light.** Print and non-interactive export render from a deterministic light palette unless the user explicitly requests themed output. The on-screen theme is never silently baked into a file the user shares.

- **BR-11 — Third-party content degrades, not disappears.** Embedded content that only supports light mode is placed in a labelled light "island" container rather than forced to dark or hidden.

Validation

- **Preference value.** Must be one of the allow-listed enums (``light``, ``dark``, ``system``). Any other value is rejected in whole, logged, and the effective theme falls back to the last known-good value, then to ``system``.
- **Schedule times.** Start and end must be valid local times. A zero-length window (start equal to end) is rejected as invalid. Windows crossing midnight are accepted and normalised. An inverted window that is not a midnight crossing is rejected with a field-level error.
- **Location for sun-based scheduling.** "Sunset/sunrise" requires a resolvable location or timezone. If location is unavailable or denied, the setting does not silently fail — the user is prompted to supply explicit times, and the schedule is left disabled until resolved.
- **Sync payload.** A sync payload carrying an unknown preference value or an out-of-range schedule is rejected in whole. The local device retains its current setting and reports a passive sync-status item.
- **Token set integrity.** If a token set is missing, malformed, or incomplete, it is not applied at all. The last known-good set is used and the failure is surfaced in diagnostics (NFR-86).
- **Asset variant resolution.** If a dark asset variant referenced by a component does not exist, the build fails by name rather than shipping a light asset on a dark surface.

Edge cases

- **EC-1 — OS theme changes while backgrounded.** On resume, the effective theme matches the OS with no visible flash or intermediate light frame.
- **EC-2 — OS theme changes mid-focus-session.** The active F-3 session, its timer, and its paused/running state are unaffected; only the palette changes.
- **EC-3 — Stored preference conflicts with OS.** The stored preference wins, deterministically. There is no oscillation and no "flip-flop" as the OS signal is re-read.
- **EC-4 — Timezone or DST change during an active window.** The window is recomputed from local time; no duplicate or missed boundary persists beyond one minute.
- **EC-5 — Sun times unavailable.** Offline, permission denied, or polar day/night. The schedule falls back to explicit times if configured, otherwise to the base preference, and the user is informed exactly once.
- **EC-6 — Two devices with different preferences and sync on.** Each device keeps its own preference until the user explicitly pushes one to all devices (BR-5).
- **EC-7 — User content with inline light colours.** Pasted HTML or hex colours are constrained by a legibility rule so text never becomes invisible; the original value is preserved in the underlying data, not rewritten.
- **EC-8 — Embedded frame with no dark support.** Rendered inside a labelled light island container (BR-11); surrounding surfaces remain dark.
- **EC-9 — Dark-transparent logos on dark surfaces.** A dark asset variant is used, or a subtle backing plate is applied so the mark stays visible.
- **EC-10 — Print/export captured mid-transition.** Exports always render from a deterministic palette; transition timing cannot leak into output.
- **EC-11 — First-ever load, no stored preference, no OS signal.** Resolves to light with no flash and no prompt.
- **EC-12 — Local storage unavailable.** Private browsing or quota exceeded. The theme applies for the session, the user receives one non-blocking notice that persistence is off, and no error loop occurs.
- **EC-13 — Theme toggled while an optimistic update is in flight.** A quick-add commit (F-2) or focus-timer state change (F-3) is neither lost nor duplicated.
- **EC-14 — Large F-4 dataset re-themed mid-render.** Charts re-render with the new palette without losing zoom, scroll position, or applied filters.
- **EC-15 — High-contrast dark combined with reduce-transparency.** Both preferences are honoured together; neither silently cancels the other.

- **EC-16 — Rapid repeated toggling.** Fast successive toggles settle on the last requested value with no queuing, flicker, or stuck intermediate state.

Error handling

- **Token set failure.** Fall back to the last known-good set, record the failure in diagnostics, and show a passive notice. Never render unstyled content and never block the app.
- **Preference write failure.** Keep the theme for the current session, show exactly one non-blocking notice, retry on next launch, and do not repeat the notice once acknowledged.
- **Sync failure or conflict.** The device-local preference is authoritative for that device. Sync errors surface as a passive status item, never as a blocking modal, and never revert the on-screen theme.
- **Schedule resolution failure.** The schedule is disabled for that evaluation, the base preference applies, and the user is informed once with the reason (no location, invalid window, or unreadable clock).
- **Unsupported native chrome call.** Silent no-op. Content theming is unaffected and no error is surfaced to the user.
- **Asset variant missing at runtime.** Fall back to the light asset on a neutral backing plate so nothing becomes invisible, and raise a build/telemetry defect.
- **General rule.** No theming failure may ever block task capture (F-2), focus sessions (F-3), analytics (F-4), or integration sync (F-5). Theme degrades; functionality does not.
- **Reporting.** Every theme error is recorded with the resolution source, token-set version, platform, and app version — never with user content.

Acceptance criteria

- **AC-1.** Given any screen or state within F-1..F-5, when the preference is set to dark, then every visible element including empty, loading, error, and modal states renders from the dark token set, with no light-coloured surface or unstyled region remaining.
- **AC-2.** Given preference = dark, when the app is cold-loaded or hard-reloaded, then zero wrong-theme frames are observable in a recorded frame trace.
- **AC-3.** Given preference = system, when the OS theme is toggled while the app is open, then the app matches within 100 ms with no reload.
- **AC-4.** Given a stored preference, when the user signs out, signs back in, and restarts the app, then the effective theme is unchanged.
- **AC-5.** Given a schedule enabled for 20:00–07:00 local time, when local time crosses 20:00 then the effective theme becomes dark, and when it crosses 07:00 the base preference is restored.
- **AC-6.** Given any dark-mode screen, when contrast is measured, then body text is $\geq 4.5:1$, large text $\geq 3:1$, and non-text UI $\geq 3:1$; the high-contrast dark variant achieves $\geq 7:1$ for body text.
- **AC-7.** Given the F-4 analytics view in dark mode, then all series, gridlines, axes, and labels are legible and every series is distinguishable without relying on colour alone.
- **AC-8.** Given an active focus session (F-3) or a partially completed quick add (F-2), when the theme is toggled, then the session, timer, and typed input are all unchanged.
- **AC-9.** Given keyboard-only use, when the theme shortcut is invoked, then the theme changes, focus does not move, and the change is announced to assistive technology.
- **AC-10.** Given dark mode is active, when any view is printed or exported, then the output is legible and light-themed unless the user explicitly requested a themed export.
- **AC-11.** Given a component added later that references a raw colour literal, when the build-time coverage check runs, then the build fails and names the offending component.
- **AC-12.** Given local storage is unavailable, when the theme is changed, then it applies for the session and exactly one non-blocking notice is shown.

- **AC-13.** Given a session-scoped override set during an active schedule window, when the window ends or a new session starts, then the override is cleared and the scheduled behaviour resumes.
- **AC-14.** Given an explicit schedule with a zero-length window, when it is saved, then it is rejected with a field-level error and no schedule is stored.

API behaviour

- **API-1 — Read theme state.** Returns the effective theme, the stored preference, the resolution source (`user`, `system`, `schedule`, `override`, `default`), the active token-set version, and the schedule definition. Readable entirely from local state with no network dependency.
- **API-2 — Write theme state.** Accepts only the allow-listed preference enum plus an optional validated schedule. Idempotent: repeating the same write returns the same effective state. Unknown fields or out-of-range values are rejected in whole with field-level errors — never partially applied.
- **API-3 — Schedule evaluation.** Performed at read time from the device's local time and, for sun-based mode, a location/timezone input. The server never dictates the effective theme for a device.
- **API-4 — Optional preference sync.** Stores and returns only the preference enum and schedule, versioned by last-write timestamp per device. Conflicts resolve last-write-wins for the pushed value; a device's local override is preserved unless the user explicitly pushes it to all devices (BR-5).
- **API-5 — Non-interference.** No other endpoint's behaviour depends on the theme. No request from F-1..F-5 may be rejected, delayed, or altered because of a theme value, and theme state is never a prerequisite for another feature's API call.
- **API-6 — Token-set distribution.** Token sets are delivered as build-time assets, not fetched per request. The client never blocks first paint or a theme switch on a network call (NFR-80).
- **API-7 — Diagnostics.** Exposes token-set version, last resolution source, and the most recent theme error to the local diagnostics surface, without user content.

Data model (DM-)

- **DM-1 — Local theme record.** `preference` (enum), `schedule` (optional: mode `explicit` | `sunrise-sunset`, start, end, timezone), `device\_id`, `last\_updated`, `token\_set\_version`, `session\_override` (optional, transient).
- **DM-2 — Sync record (only when sync is enabled).** `user\_id`, `preference`, `schedule`, `device\_id`, `updated\_at`. Contains no task content, no analytics data, and no integration payloads.

Priority

Must-have. Confidence: high.

Dark mode is a first-class usability and accessibility requirement rather than a cosmetic extra. The product's primary persona uses it in evening and low-light contexts, the vision explicitly calls for structure without visual friction, and every other feature in scope (F-1 through F-5) renders on the same themed surfaces — leaving theming out would leave coverage gaps on screens that already exist. The must-have core is FR-126 through FR-132 plus FR-137, FR-140, and FR-143, measured by NFR-76 through NFR-78 and NFR-81. FR-133 (scheduled switching), FR-139 (high-contrast dark), and FR-142 (native chrome) are strong should-haves that ship in the same feature but may follow the core by one increment without breaking any acceptance criterion above.

Open Questions

- **OQ-1.** Is the theme preference intended to sync across a user's devices by default, or remain device-local unless the user opts in? This affects FR-128, FR-132, NFR-84, NFR-85, and API-4. This section assumes device-local by default with opt-in sync, and does not reduce scope either way — the

answer only changes the default.

- **OQ-2.** Does "Dark Mode" cover the product's public marketing/help surfaces, or only the authenticated application? This section assumes the authenticated application plus outbound notifications (FR-141), and treats marketing pages as outside this feature unless the user confirms otherwise.

- **OQ-3.** Is a fully custom accent colour (beyond light/dark/system and the high-contrast dark variant) in scope? BR-1 excludes it as written; the user should confirm, since including it would add requirements rather than remove any.

Functional Requirements

This section is the consolidated, global index of every functional requirement. Detailed per-feature behaviour, business rules, validation, edge cases, error handling, acceptance criteria and API behaviour live in the corresponding `## F-<n>` sections above; the entries below are the canonical FR ids and their one-line definitions. Every FR id is defined exactly once and belongs to the feature whose owned range contains it.

F-1 — Intelligent Task Prioritization (FR-1 .. FR-25)

ID	Requirement	Description	Priority
----	-------------	-------------	----------

---	---	---	---
-----	-----	-----	-----

FR-1	Automatic priority scoring	The system assigns every active task a numeric priority score derived from the user's prioritization profile.	must-have
------	----------------------------	---	-----------

FR-2	Ranked priority queue	The system exposes an ordered "Today" queue of active tasks sorted by descending priority score.	must-have
------	-----------------------	--	-----------

FR-3	Ranking factors	Scoring uses at minimum: due date/time, explicit user importance, estimated effort, task age, and dependency state.	must-have
------	-----------------	---	-----------

FR-4	Factor weighting controls	The user can adjust the relative weight of each ranking factor.	should-have
------	---------------------------	---	-------------

FR-5	Explainability	The user can view, per task, a plain-language explanation of why it has its current rank.	should-have
------	----------------	---	-------------

FR-6	Manual override	The user can pin a task to the top of the queue or force it into a specific rank position.	must-have
------	-----------------	--	-----------

FR-7	Override scope and expiry	Overrides can be indefinite or time-boxed (until a date/time or until task completion).	should-have
------	---------------------------	---	-------------

FR-8	Override visibility and removal	Pinned/overridden tasks are visually marked and the override can be removed by the user at any time.	must-have
------	---------------------------------	--	-----------

FR-9	Deadline urgency escalation	Tasks approaching their due date/time escalate in rank automatically as the deadline nears.	must-have
------	-----------------------------	---	-----------

FR-10	Dependency awareness	Tasks blocked by unfinished dependencies rank below their blockers and are flagged as blocked.	must-have
-------	----------------------	--	-----------

FR-11	Importance flag	The user can mark a task as important; importance raises the score by the configured weight.	must-have
-------	-----------------	--	-----------

FR-12	Effort estimate	The user can assign an estimated effort to a task; effort feeds the scoring function.	should-have
-------	-----------------	---	-------------

FR-13	Task aging	Older incomplete tasks accumulate a bounded age bonus so none are starved indefinitely.	should-have
-------	------------	---	-------------

FR-14	Scoring profile	Prioritization uses a named profile (weight set) the user can create, rename, select and delete.	should-have
-------	-----------------	--	-------------

FR-15	Automatic recomputation	Scores recompute on relevant change events (task edits, completion, deadline passage, time tick).	must-have
-------	-------------------------	---	-----------

| FR-16 | Queue refresh cadence | The queue re-sorts at a bounded cadence so time-based escalations appear without manual refresh. | should-have |

| FR-17 | Overdue handling | Overdue tasks are surfaced in a dedicated "Overdue" group above the main queue. | must-have |

| FR-18 | Queue filtering | The user can filter the queue by tag, project, due window and importance without changing underlying scores. | should-have |

| FR-19 | Queue search | The user can search the queue by title/tag and see results ranked by the same score. | should-have |

| FR-20 | Bulk actions | The user can select multiple queue tasks and complete, snooze, reschedule or re-tag them in one action. | should-have |

| FR-21 | Snooze/defer | The user can snooze a task until a chosen date/time, removing it from the queue until then. | should-have |

| FR-22 | Completion effect | Completing a task removes it from the queue and recomputes dependents' scores. | must-have |

| FR-23 | Recurrence awareness | Recurring tasks reappear with a fresh score when their next occurrence becomes active. | should-have |

| FR-24 | Score visibility | Each queue row can reveal the task's current score or a normalized rank indicator on demand. | nice-to-have |

| FR-25 | Profile default | A sensible built-in default profile applies before the user customizes weights. | must-have |

F-2 — Quick Add (FR-26 .. FR-50)

ID	Requirement	Description	Priority
FR-26	Single-input capture	A single free-text line is the only mandatory input; one confirm action creates a task, reachable from every primary screen.	must-have
FR-27	Natural-language date/time parsing	Natural-language date/time expressions are recognized and bound to the task's due date/time.	must-have
FR-28	Priority & importance parsing	Inline priority markers map to the task's explicit-importance field consumed by F-1 prioritization.	must-have
FR-29	Tag / project / context parsing	`#tag` and `@context` tokens attach to the task; unrecognized tokens remain part of the title.	should-have
FR-30	Parsed-field preview	Interpreted fields (title, due date/time, importance, tags) are previewed before commit.	should-have
FR-31	Ambiguity resolution	Ambiguous parsed fragments are highlighted with candidate interpretations offered before commit.	should-have
FR-32	Undo create	After creating a task the user can undo the creation within a short window.	should-have
FR-33	Confirmation feedback	A created task shows a brief confirmation with its parsed fields and a link to open it.	must-have
FR-34	Keyboard-first flow	Quick Add opens via a single global shortcut and commits with Enter; Escape cancels without creating.	must-have
FR-35	Global reachability	Quick Add is reachable from every primary screen via a persistent affordance and the shortcut.	must-have
FR-36	Empty input handling	Submitting empty or whitespace-only input creates nothing and gives inline guidance.	must-have
FR-37	Multi-task capture	Each non-empty line in a pasted block becomes a separate task in one confirm.	should-have
FR-38	Duplicate detection	If an identical open task title exists the system warns and offers merge or create-anyway.	nice-to-have

| FR-39 | Recent defaults | New tasks inherit sensible defaults (project/tag context) from the active screen. | should-have |

| FR-40 | Voice capture | The user can dictate a task line, parsed by the same rules as typed input. | nice-to-have |

| FR-41 | Parser feedback | Unrecognized tokens remain in the title and are visually distinguished so nothing is lost. | must-have |

| FR-42 | Date shortcut vocabulary | Common shorthands resolve to locale-aware dates. | should-have |

| FR-43 | Time zone handling | Parsed times resolve in the user's current time zone, with an override for explicit zones. | must-have |

| FR-44 | Priority precedence | Explicit priority markers override inherited or default importance. | must-have |

| FR-45 | Context token creation | A previously unseen `#tag` or `@context` is created on confirm rather than rejected. | should-have |

| FR-46 | Edit before commit | The preview fields are editable prior to confirming the create. | should-have |

| FR-47 | Batch feedback | When multiple tasks are created a summary count with per-item correction is shown. | nice-to-have |

| FR-48 | Offline queueing | Captures made while offline are stored locally and created once connectivity returns. | should-have |

| FR-49 | Latency budget | Parse and create complete within the Quick Add latency budget defined by NFR-16. | must-have |

| FR-50 | Accessibility | Quick Add is fully operable by keyboard and announces parse results to assistive tech. | must-have |

F-3 — Focus Mode (FR-51 .. FR-75)

ID	Requirement	Description	Priority
FR-51	Enter focus on a task	The user can start a focus session bound to a single existing task from task detail, the Today queue or a quick-add result.	must-have
FR-52	Default session length	Sessions run for a configurable default duration (built-in default 25 minutes) in whole minutes.	must-have
FR-53	Custom session length	Before or during a session the user can set a custom duration within the supported range.	should-have
FR-54	Timer controls	The user can start, pause, resume, extend and end a session, with every state change reflected immediately.	must-have
FR-55	Single active session	At most one focus session may be active at a time; starting another requires an explicit decision.	must-have
FR-56	Post-session summary	On completion a summary is shown with the task, elapsed time and offered next actions.	should-have
FR-57	Focus-mode UI	During a session the UI presents a distraction-reduced view centred on the active task.	must-have
FR-58	Session completion	When the timer elapses the session ends, the user is notified, and can complete the task or extend.	must-have
FR-59	Early end	The user can end a session early; elapsed time is still recorded.	must-have
FR-60	Session logging	Each completed, extended or abandoned session is recorded with task, start, end and duration.	must-have
FR-61	Extension handling	Extending adds the chosen minutes to the remaining time without restarting the elapsed count.	should-have
FR-62	Background accuracy	The timer stays accurate when the tab is backgrounded or the device sleeps, reconciled on resume.	must-have

| FR-63 | Notifications | An audible/visual cue fires at session end, subject to user settings. | should-have |

| FR-64 | Do-not-disturb | While in focus mode the product suppresses its own non-critical notifications. | should-have |

| FR-65 | Task switching | Switching the focused task mid-session requires ending the current session first, respecting FR-55. | should-have |

| FR-66 | Break sessions | After a focus session the user may start an optional break session of configurable length. | should-have |

| FR-67 | Manual time logging | The user can log time spent on a task without running a timer. | should-have |

| FR-68 | Time aggregation | Recorded focus time aggregates per task and is available to analytics (F-4). | should-have |

| FR-69 | Session notes | The user can attach a short note to a session on completion. | nice-to-have |

| FR-70 | Timer accessibility | Timer state and controls are exposed to assistive tech and fully keyboard operable. | must-have |

| FR-71 | Persistence across reload | An active session survives a page reload without losing elapsed time. | must-have |

| FR-72 | Concurrent devices | A session started on one device is the single active session across the user's devices. | should-have |

| FR-73 | Focus history view | The user can browse a chronological list of past focus sessions. | should-have |

| FR-74 | Focus settings | Default length, break length, notification and DND behaviour are configurable and persisted. | should-have |

| FR-75 | Completion linkage | Marking the focused task complete at session end is offered as the default action. | should-have |

F-4 — Progress Analytics (FR-76 .. FR-100)

ID	Requirement	Description	Priority
FR-76	Analytics dashboard access	A dedicated "Progress" view is reachable from primary navigation and presents personal metrics on one surface.	must-have
FR-77	Completion metrics	Total completed, total created and net change for the selected period are shown.	must-have
FR-78	Completion rate over time	Completion rate (completed ÷ created) is shown for the period and plotted as a time series.	must-have
FR-79	Task throughput	Tasks completed per day/week/month are computed and displayed as bar or line series.	must-have
FR-80	Completion streak	Current and longest streak of consecutive days with at least one completion are shown.	should-have
FR-81	Period selector	The user can select the analyzed period (day, week, month, custom range).	must-have
FR-82	Priority distribution	Distribution of completed tasks by priority bucket is shown.	should-have
FR-83	Focus time analytics	Total focus time recorded in Focus Mode (F-3) is shown per period.	should-have
FR-84	Focus-to-completion ratio	Ratio of focused minutes to tasks completed is shown.	nice-to-have
FR-85	Queue health	Counts of overdue, due-soon and blocked tasks are summarized.	should-have
FR-86	Aging tasks	The number of tasks older than a threshold is reported.	nice-to-have

| FR-87 | Created vs completed trend | A dual-series chart of tasks created and completed over the period is shown. | should-have |

| FR-88 | Tag/project breakdown | Completion counts are broken down by tag and project. | should-have |

| FR-89 | Daily goal | The user can set a daily completion goal and see progress against it. | should-have |

| FR-90 | Weekly goal | The user can set a weekly completion goal and see progress against it. | should-have |

| FR-91 | Comparative period | The dashboard can compare the selected period to the preceding period. | nice-to-have |

| FR-92 | Data export | The user can export the analytics data for the selected period as a file. | should-have |

| FR-93 | Empty-period state | When a period has no data the dashboard shows an explanatory empty state. | must-have |

| FR-94 | Time zone correctness | Period boundaries are computed in the user's time zone. | must-have |

| FR-95 | Drill-down | Selecting a chart point opens the underlying task list for that bucket. | should-have |

| FR-96 | Streak rules | The streak definition is documented and applied consistently. | should-have |

| FR-97 | Metric definitions | Each metric has an accessible definition explaining how it is computed. | should-have |

| FR-98 | Refresh/currency | Metrics recompute as tasks change without requiring a reload beyond the fetch cadence. | should-have |

| FR-99 | Accessibility of charts | Charts provide non-visual equivalents (tables/summaries) for screen readers. | must-have |

| FR-100 | Personal scope only | Analytics reflect only the signed-in user's own data. | must-have |

F-5 — Integration Hub (FR-101 .. FR-125)

ID	Requirement	Description	Priority
FR-101	Connection catalogue	A browsable catalogue of connectable source types with a plain-language description of data read and written.	must-have
FR-102	Connect account	The user can complete an authorization handshake without typing provider credentials into product fields.	must-have
FR-103	Connection lifecycle management	Connections are listed with label, type, identity, state and last-sync time, supporting rename, pause, resume, reconnect and disconnect.	must-have
FR-104	Credential handling	Provider credentials/tokens are never displayed in full and are stored securely per NFR-64.	must-have
FR-105	Data scope disclosure	Before authorizing, the user sees exactly what data the connection reads and writes.	must-have
FR-106	Sync direction	Each connection declares whether it is read-only, write-only or bidirectional.	should-have
FR-107	Manual sync	The user can trigger an immediate sync for any connection.	must-have
FR-108	Sync schedule	Automatic sync runs on a configurable cadence per connection.	should-have
FR-109	Sync error surfacing	Failed syncs set the connection to `error`/`degraded` and surface an actionable message.	must-have
FR-110	Re-authentication	An `expired` connection prompts reconnect without losing mapping configuration.	must-have
FR-111	Item mapping	Imported items map to product tasks with source provenance retained.	must-have

| FR-112 | Conflict resolution | When the same item changes in both systems the user is shown a conflict-resolution choice. | should-have |

| FR-113 | Deduplication | Re-importing the same source item does not create duplicate tasks. | must-have |

| FR-114 | Selective import | The user can scope which source lists/calendars/projects are imported. | should-have |

| FR-115 | Write-back | For bidirectional connections, product changes propagate to the source per the declared rules. | should-have |

| FR-116 | Disconnect behaviour | Disconnecting stops sync and offers a choice to keep or remove imported tasks. | must-have |

| FR-117 | Sync history | The user can view a log of recent sync runs with timestamps and outcomes. | should-have |

| FR-118 | Rate-limit handling | Sync respects provider rate limits and backs off without data loss. | should-have |

| FR-119 | Webhook bridge | The generic webhook bridge accepts inbound payloads mapped to task creation. | nice-to-have |

| FR-120 | Retry policy | Transient sync failures retry with backoff before being marked failed. | must-have |

| FR-121 | Connection health summary | A single view summarizes the health of all connections. | should-have |

| FR-122 | Permissions revoke | The user can revoke a connection's granted permissions from within the product. | should-have |

| FR-123 | Audit trail | Connection changes (connect, pause, disconnect, reconnect) are recorded with timestamps. | should-have |

| FR-124 | Provider-agnostic design | Adding a new source type does not change the user-facing connection lifecycle. | should-have |

| FR-125 | Offline resilience | Loss of connectivity marks connections `degraded` and recovers automatically when restored. | should-have |

F-6 — Dark Mode (FR-126 .. FR-150)

ID	Requirement	Description	Priority
FR-126	Theme preference selection	The user can choose exactly one of `light`, `dark` or `system` as the active theme preference. must-have	
FR-127	System preference following	While `system` is selected the product resolves the effective theme from the host colour-scheme signal and re-resolves live. must-have	
FR-128	Preference persistence	The chosen preference is persisted and re-applied on the next session, after sign-out/in, reload and app restart. must-have	
FR-129	Immediate application	Changing the preference or the system signal re-themes every already-rendered surface without reload. must-have	
FR-130	No flash of wrong theme	On load the correct theme is applied before first paint, avoiding a flash of the wrong theme. must-have	
FR-131	Cross-screen consistency	Every screen, modal, overlay and empty/loading/error state honours the active theme. must-have	
FR-132	Scheduled auto switching	The user can optionally schedule theme switching by time of day. nice-to-have	
FR-133	Contrast compliance	Text and UI meet WCAG 2.1 AA contrast in both themes, per NFR-79. must-have	
FR-134	Semantic colour tokens	Components consume semantic tokens rather than hard-coded colours so theming is uniform. should-have	

FR-135	Charts and data viz	Analytics (F-4) charts and any data visualisation adapt to the active theme.	must-have
FR-136	Third-party embeds	External content that cannot be themed is shown on a compliant surface.	should-have
FR-137	Images and media	Illustrations/media have theme-appropriate variants or an acceptable neutral treatment.	should-have
FR-138	System signal detection	The product reads `prefers-color-scheme` where available and falls back to `light` when unknown.	must-have
FR-139	User override of system	Selecting `light`/`dark` stops following the system until `system` is re-selected.	must-have
FR-140	Settings entry point	Theme preference is settable from a clearly labelled Settings location.	must-have
FR-141	Account-level default	Theme preference may follow the user across devices without overriding a device-local explicit choice.	should-have
FR-142	Print/export	Printed/exported views remain legible regardless of the active theme.	should-have
FR-143	Reduced motion	Theme transitions respect `prefers-reduced-motion`.	should-have
FR-144	Focus indicators	Focus indicators remain visible in both themes.	must-have
FR-145	Loading state theming	Skeletons, spinners and loaders use theme tokens.	should-have
FR-146	Error state theming	Error, warning and success states remain distinguishable in both themes.	must-have
FR-147	Persistence edge cases	Persistence survives sign-out/sign-in, reload and app restart, reinforcing FR-128.	must-have
FR-148	No layout shift	Switching theme does not cause layout shift (CLS).	should-have
FR-149	Preference reset	The user can reset to the default (`system`) preference.	should-have
FR-150	Accessibility announcement	Changing theme is announced to assistive tech without stealing focus.	must-have

****Coverage check:**** FR-1..FR-150 map one-to-one onto features F-1..F-6 with no gaps, no duplicates and no ids outside the assigned ranges.

Non-Functional Requirements

Product-wide NFRs below apply to every feature (F-1 .. F-6). Ids are global and unique across the spec; feature-scoped NFRs are declared inside their own feature sections and referenced here only by plain text where relevant.

Performance

ID	Requirement	Target (measurable)	Measurement
NFR-1	Initial load of the app shell on a typical desktop connection	First Contentful Paint ≤ 1.0 s; Time to Interactive ≤ 3.0 s; on a simulated 3G (Fast 3G) profile the interactive shell is usable ≤ 2.0 s	Lab run on throttled profile (e.g. Lighthouse CI) on the production build, median of 5 runs
NFR-2	Largest contentful paint on the primary screens (Today queue, task detail, Progress)	LCP ≤ 2.5 s at p75	Real-user monitoring (RUM) p75 over trailing 28 days
NFR-3	Visual stability during load and re-theming	Cumulative Layout Shift ≤ 0.1 at p75	RUM p75 / Lighthouse on production build
NFR-4	Interaction responsiveness	Every user-initiated action (tap, click, key) produces a visible acknowledgement ≤ 100 ms; any operation exceeding 2 s shows a progress/loading indicator	RUM event timing; manual measurement for long operations

| NFR-5 | Priority scoring and Today-queue recomputation (F-1) | Full recompute for a user with 5,000 active tasks completes ≤ 300 ms; incremental recompute after a single task edit ≤ 50 ms | Server-side timing histogram, p95 |

| NFR-6 | Quick-add capture-to-preview (F-2) | Parse + preview render ≤ 150 ms after the last keystroke, for input up to 280 characters | Client-side timing instrumentation, p95 |

| NFR-7 | Focus-session timer accuracy (F-3) | Displayed session time deviates from wall-clock by ≤ 1 s over a 24-hour continuous session; timer display refreshes at least once per second | Automated timing test comparing monotonic clock to rendered value |

| NFR-8 | Progress analytics rendering (F-4) | Dashboard for a selected period first contentful render ≤ 2 s and fully rendered ≤ 3 s for up to 5 years of a user's data | Server aggregation timing + client render timing, p95 |

| NFR-9 | Integration sync throughput and latency (F-5) | An incremental sync completes ≤ 60 s after trigger for a source with $\leq 10,000$ changed items; initial/backfill sync reports progress and does not block the UI | Sync worker timing histogram, p95; UI must remain interactive during backfill |

| NFR-10 | Theme application latency (F-6) | A preference change or OS colour-scheme change re-themes all rendered surfaces within one animation frame (≤ 16 ms) with no flash of the previous theme on next load | Client timing test; visual regression of first paint |

| NFR-11 | Bundle discipline | Initial JS payload for the shell ≤ 200 KB gzipped; feature areas (Focus, Progress, Integrations) are code-split | Build-size budget check in CI (fails build on regression) |

Scalability & Availability

| ID | Requirement | Target (measurable) | Measurement |

|---|---|---|---|

| NFR-12 | Per-user data volume | Support $\geq 50,000$ active tasks and $\geq 500,000$ historical events per user without degradation beyond the NFR-5/NFR-8 budgets | Load test with synthetic user at that volume |

| NFR-13 | Concurrency | Support $\geq 10,000$ concurrent active users with p95 API latency ≤ 500 ms for read endpoints and ≤ 800 ms for write endpoints | Load test against staging at target concurrency |

| NFR-14 | Availability | Monthly availability ≥ 99.9 % for core read/write paths (task list, quick add, focus timer persistence) | Uptime monitor over calendar month |

| NFR-15 | Horizontal scalability | Application tier scales horizontally without sticky sessions; adding instances increases throughput linearly (within 10 % of ideal) up to 4 $\times$ baseline | Scale-out benchmark in staging |

| NFR-16 | Elastic capacity | Autoscaling reacts to sustained load within 5 minutes; no 5xx increase during a 3 $\times$ traffic spike sustained for 15 minutes | Chaos/load test |

| NFR-17 | Graceful degradation | If a non-core subsystem (F-4 analytics, F-5 integrations, F-6 preference service) is unavailable, the core paths (F-1, F-2, F-3) remain fully functional with an in-context degraded-mode notice; no data loss on recovery | Fault-injection test per subsystem |

| NFR-18 | Offline tolerance for in-progress capture | Quick-add entries and focus-session state created while offline are queued and reconciled on reconnect without duplication or loss | Automated offline/reconnect test |

Security

| ID | Requirement | Target (measurable) | Measurement |

|---|---|---|---|

| NFR-19 | Transport security | All client $\leftrightarrow$ server and server $\leftrightarrow$ provider traffic uses TLS 1.2 or higher; HSTS enabled; no plaintext HTTP access to authenticated surfaces | Config audit + external TLS scan |

| NFR-20 | Encryption at rest | All persisted user data (tasks, analytics aggregates, integration metadata, focus history) is encrypted at rest with a managed key; rotating keys supported without

downtime | Storage configuration audit |
NFR-21	Authentication & session safety	Passwords (if used) stored only as salted, adaptive hashes; sessions expire after a configurable idle period and use secure, HttpOnly, SameSite cookies (or equivalent token handling); no credentials ever written to client storage in plaintext	Code/security review + automated session tests
NFR-22	Third-party authorization (F-5)	Provider credentials are never entered into product-owned credential fields; OAuth access/refresh tokens are encrypted at rest, never logged, never returned to the client, and are revoked on disconnect	Security review + log-scanning test
NFR-23	Tenant/data isolation	A user can access only their own data; every data access is authorized against the requesting identity; cross-tenant access attempts are denied and audited	Automated isolation test suite (must pass on every release)
NFR-24	Input handling hardening	All user-supplied content (titles, tags, integration payloads, webhook bodies) is validated and safely encoded on output; protections against XSS, CSRF, injection, and SSRF (for webhook bridges) are enforced	SAST/DAST in CI + targeted abuse tests
NFR-25	Rate limiting & abuse protection	Authenticated endpoints enforce per-user rate limits; webhook/inbound endpoints enforce per-source limits and reject unauthenticated or oversized payloads	Load test + configuration audit
NFR-26	Auditability of security-relevant events	Authentication, connection grant/revoke, and data-export/delete events are recorded in an append-only audit log with actor, timestamp, and outcome	Log review + automated audit-event tests
NFR-27	Dependency & vulnerability posture	No known high/critical vulnerabilities in production dependencies; scanning runs on every build and blocks release on new high/critical findings	SCA scan gate in CI

Data & Residency

ID	Requirement	Target (measurable)	Measurement
NFR-28	Data residency	Users can be pinned to a declared storage region; all primary data and backups for that user remain in the selected region	Configuration + automated residency assertion test
NFR-29	Data ownership & export	The user can export all their data (tasks, focus history, analytics source data, connection metadata) in a machine-readable format; export completes ≤ 5 minutes for a maximum-volume user	End-to-end export test
NFR-30	Right to erasure	A delete request removes or irreversibly anonymizes all personal data within 30 days, including derived analytics aggregates and cached integration data; the completion is recorded in the audit log	Deletion verification test on staging
NFR-31	Retention defaults	Operational logs retained ≤ 90 days by default; analytics source events retained per user-configurable retention with a documented floor and ceiling; no indefinite retention without an explicit policy	Retention configuration audit
NFR-32	Backup & recovery	Automated backups at least daily with an RPO ≤ 24 h and RTO ≤ 4 h for core data; restore is exercised at least quarterly	Backup/restore drill record
NFR-33	Legal basis & consent	Data collection is limited to what each feature requires; integration scopes are minimized to the declared read/write capability; consent for each connected source is recorded and revocable	Scope review + consent-state test
NFR-34	Derived data transparency	Aggregated/derived metrics (F-4) are recomputable from source events and are removed or recomputed when their source data is deleted or corrected	Recomputation test after source mutation

Deployment & Environment

ID	Requirement	Target (measurable)	Measurement
NFR-35	Environment separation	Independent development, staging, and production environments with no shared production data in non-production tiers	Environment inventory audit
NFR-36	Reproducible builds & deployment	Every deploy is produced from a versioned, immutable build artifact; deployment is automated and idempotent	CI pipeline audit
NFR-37	Zero-downtime releases	Core paths remain available during deploys and schema migrations; migrations are backward-compatible and reversible	Deploy rehearsal + availability monitor during release
NFR-38	Configuration & secrets management	Configuration is externalized per environment; secrets are injected at runtime from a managed store and never committed to the repository	Secret-scanning gate in CI
NFR-39	Observability	Structured logs, metrics, and distributed traces cover every request path; dashboards and alerts exist for the NFR-1..NFR-18 budgets, error rate, and availability	Observability coverage review; alert-routing test
NFR-40	Rollback	Any release can be rolled back to the previous known-good version within 15 minutes without data loss	Rollback drill
NFR-41	Client platform support	Supported browsers: current and previous major versions of Chrome, Firefox, Safari, Edge; responsive layouts for viewports 320 px–2560 px; no horizontal scroll at 320 px	Cross-browser/cross-viewport test matrix
NFR-42	Accessibility conformance	WCAG 2.1 Level AA across all features: text contrast $\geq 4.5:1$, UI contrast $\geq 3:1$, full keyboard operability, semantic landmarks, screen-reader labels, visible focus (≥ 2 px), no flashing > 3 Hz, skip-to-content, and both light and dark themes independently conformant	Automated a11y scan (zero critical) + manual audit for dark-mode parity
NFR-43	Internationalization readiness	UI strings are externalized; dates, times, numbers, and currency render per user locale; layout supports RTL without truncation	Locale/RTL smoke test
NFR-44	Perceived-quality budget monitoring	Performance and availability budgets (NFR-1..NFR-18) are enforced as release gates; a regression beyond budget blocks promotion to production	CI gate configuration audit

User Stories

Stories are grouped by feature. Global ids `US-1 .. US-15` belong to F-1, `US-16 .. US-30` to F-2, `US-31 .. US-45` to F-3, `US-46 .. US-60` to F-4, `US-61 .. US-75` to F-5, and `US-76 .. US-90` to F-6. References to other features' ids are plain text only.

F-1 — Intelligent Task Prioritization

US-1: See my work in priority order

- As a busy professional, I want my active tasks ranked for me automatically, so that I always know what to do next without manually sorting.
- Acceptance Criteria: Every active task has a numeric priority score (FR-1); the "Today" queue is sorted by descending score (FR-2); the order is stable across reloads when inputs are unchanged.

US-2: Have multiple factors feed the ranking

- As a user with mixed deadlines, I want ranking to consider due dates, importance, effort, age, and dependencies, so that the order reflects real-world urgency.
- Acceptance Criteria: Each listed factor influences the score (FR-3); changing any single factor on a task can change its rank.

US-3: Understand why a task is ranked where it is

- As a skeptical user, I want a plain-language explanation per task, so that I can trust the automatic ordering.
- Acceptance Criteria: Each task exposes an explanation naming the dominant factors (FR-5); the explanation is readable without jargon.

US-4: Tune the ranking to my style

- As a user who values deadlines over effort, I want to adjust factor weights, so that ranking matches how I actually work.
- Acceptance Criteria: Weight controls exist per factor (FR-4); saving weights recomputes the queue; changes persist across sessions.

US-5: Override the ranking when I disagree

- As a user, I want to pin a task to the top, so that the system does not hide something I must do now.
- Acceptance Criteria: A pin places the task at rank 1 (FR-6); the pinned task is visually marked (FR-8); removing the pin restores automatic ordering.

US-6: Time-box my override

- As a user, I want an override to expire, so that I do not leave stale pins that distort my queue forever.
- Acceptance Criteria: An override may be indefinite, until a date/time, or until completion (FR-7); on expiry the task returns to its computed rank automatically.

US-7: Escalate genuine deadlines

- As a user with a hard deadline, I want urgent tasks to rise as the deadline approaches, so that I don't miss commitments.
- Acceptance Criteria: Deadline proximity increases urgency in the score (FR-9); a task past due is never buried below a non-urgent task solely by age or effort.

US-8: Respect dependency order

- As a user, I want a task blocked by an unfinished dependency to rank below its blocker, so that I don't start work I can't finish.
- Acceptance Criteria: A blocked task is ranked below its unfinished blockers; completing the blocker updates both tasks' ranks.

US-9: See an empty queue gracefully

- As a new user, I want a helpful prompt when nothing is active, so that I am not confused by a blank screen.
- Acceptance Criteria: An empty "Today" state shows an explanation and a call-to-action to add a task.

US-10: Stay responsive with a large backlog

- As a heavy user, I want the queue to load and reorder quickly, so that prioritization never slows me down.

- Acceptance Criteria: The queue renders within the product-wide performance targets for realistic list sizes; reordering after an edit is visible within 100 ms of acknowledgement.

US-11: Operate the queue by keyboard

- As a keyboard user, I want to navigate and pin/unpin without a mouse, so that I can work efficiently.
- Acceptance Criteria: All queue actions (open, pin, override, remove override) are reachable and operable by keyboard with visible focus.

F-2 — Quick Add

US-12: Capture a task in one line

- As a user with a fleeting thought, I want to add a task by typing one line and pressing Enter, so that capture never interrupts my flow.
- Acceptance Criteria: A single mandatory free-text field exists (FR-26); one confirm action creates the task; the input is reachable from every primary screen.

US-13: Use natural language for due dates

- As a user, I want to type "tomorrow 5pm" and have it understood, so that I don't open a date picker.
- Acceptance Criteria: Recognized expressions bind to the task's due date/time (FR-27); the resolved value is shown before confirmation.

US-14: Mark importance inline

- As a user, I want to type "!!" or "urgent" to flag importance, so that the task feeds prioritization correctly.
- Acceptance Criteria: Markers map to the explicit-importance field consumed by F-1 (FR-28); the marker is removed from the title.

US-15: Tag and contextualize inline

- As a user, I want to type "#work" and "@home", so that tasks are organized as I capture them.
- Acceptance Criteria: `#tag` and `@context` tokens attach to the task (FR-29); unrecognized tokens remain in the title.

US-16: Preview what will be parsed

- As a cautious user, I want to see how my line will be interpreted before I commit, so that I can catch mistakes.
- Acceptance Criteria: Interpreted fields (title, due date/time, priority, tags) are previewed before confirmation (FR-30).

US-17: Correct a misparse

- As a user, I want to edit a parsed field in place, so that a wrong interpretation does not become a wrong task.
- Acceptance Criteria: Each parsed field is editable before confirmation; edited values override the parser output.

US-18: Fall back safely on unparseable input

- As a user typing something unusual, I want the whole line kept as the title if nothing parses, so that I never lose input.
- Acceptance Criteria: When no tokens are recognized, the full line becomes the title and no fields are invented.

US-19: Resolve ambiguity predictably

- As a user, I want ambiguous phrases handled consistently, so that behavior is learnable.
- Acceptance Criteria: Ambiguous expressions follow a documented rule (e.g. nearest future occurrence); the chosen interpretation is shown in the preview.

US-20: Undo an accidental add

- As a user, I want to undo a just-created task, so that a slip does not clutter my list.
- Acceptance Criteria: A confirmation affords undo for a short window; undo removes the created task entirely.

US-21: Respect my timezone

- As a traveler, I want parsed times interpreted in my current timezone, so that "5pm" means 5pm where I am.
- Acceptance Criteria: Parsed times use the user's effective timezone; a timezone change re-renders existing due times consistently.

US-22: Capture while offline

- As a user on a flaky connection, I want quick add to work offline, so that capture is never blocked.
- Acceptance Criteria: The task is accepted locally and clearly marked as pending sync; it syncs without duplication when connectivity returns.

US-23: Avoid accidental duplicates

- As a user, I want a warning when I add a near-identical open task, so that I don't create clutter.
- Acceptance Criteria: A near-duplicate warning appears before commit; the user can confirm or cancel.

US-24: Complete the flow by keyboard only

- As a keyboard user, I want to open, preview, and confirm quick add without a mouse, so that capture stays fast.
- Acceptance Criteria: The entire quick-add flow is keyboard-operable with visible focus and sensible tab order.

F-3 — Focus Mode

US-25: Start focusing on a task

- As a user, I want to begin a focus session from a task, so that I commit a block of time to it.
- Acceptance Criteria: A session can start from the task detail view, the "Today" queue (from F-1 ids are plain-text references), or a quick-add result (from F-2 ids are plain-text references) (FR-51).

US-26: Use my preferred session length

- As a user, I want a default session length I can rely on, so that starting focus requires no setup.
- Acceptance Criteria: A configurable default (built-in 25 minutes) applies (FR-52); the default persists across sessions.

US-27: Set a custom duration

- As a user with a longer task, I want to set a custom length, so that the timer matches my work.
- Acceptance Criteria: Custom durations within the supported range (per feature validation V-1 in this feature file) are accepted before or during a session (FR-53).

US-28: Control the timer

- As a user, I want to pause, resume, extend, and end, so that the session fits reality.
- Acceptance Criteria: All controls exist and update the UI immediately (FR-54); paused time is not counted as focused time.

US-29: Never run two sessions at once

- As a user, I want clear handling when a session is already active, so that my time records stay accurate.
- Acceptance Criteria: Starting a new session while one is active forces an explicit decision (per business rule BR-3 in this feature file) (FR-55).

US-30: See my session at a glance

- As a user, I want an unambiguous on-screen timer, so that I know how long remains.
- Acceptance Criteria: Remaining time is legible, updates at least once per second, and is visible without scrolling.

US-31: Work without distraction

- As a user, I want a distraction-free surface during focus, so that I stay on one task.
- Acceptance Criteria: Non-essential chrome is suppressed during a session; the focused task's title remains visible.

US-32: Be reminded when time is up

- As a user, I want a clear end-of-session signal, so that I can transition deliberately.
- Acceptance Criteria: Session completion produces a noticeable, dismissible signal; the user is offered stop or extend.

US-33: Know focused time was recorded

- As a user, I want focus time attributed to the task, so that my history reflects the effort.
- Acceptance Criteria: Completed (and per policy, paused/ended) sessions record focused duration against the task (FR-56 onward in this feature file).

US-34: End a session early

- As a user, I want to stop before time is up, so that an interrupted block is not lost.
- Acceptance Criteria: Ending early records elapsed time; the task is not auto-completed without confirmation.

US-35: Keep the timer accurate in the background

- As a user, I want the timer to stay correct if I switch tabs, so that the app doesn't drift.
- Acceptance Criteria: Elapsed time is derived from wall-clock timestamps, not tick counting, so it is correct after backgrounding.

US-36: Recover from an interrupted session

- As a user whose app closed mid-session, I want a sane recovery, so that nothing is silently lost.
- Acceptance Criteria: On return, the session is either restored or the user is prompted; no session remains "running" with no UI.

US-37: Review my focus history

- As a user, I want to see past sessions, so that I can understand my patterns.
- Acceptance Criteria: A history list shows task, date, planned and actual duration for each session.

US-38: Operate focus mode accessibly

- As a user relying on assistive tech, I want the timer and controls accessible, so that I can use focus mode fully.
- Acceptance Criteria: Timer changes are announced politely, controls have accessible names, and no flashing exceeds safe limits.

F-4 — Progress Analytics

US-39: Reach my progress view

- As a user, I want a dedicated Progress view, so that I can review how I am doing in one place.
- Acceptance Criteria: The view is reachable from primary navigation and presents personal metrics on one scrollable surface (FR-76).

US-40: See completion at a glance

- As a user, I want to see completed, created, and net change, so that I understand my throughput.
- Acceptance Criteria: Completed, created, and net (created – completed) are shown for the selected period (FR-77).

US-41: Track completion rate over time

- As a user, I want completion rate plotted over time, so that I can spot trends.
- Acceptance Criteria: Completion rate (completed ÷ created, as a percentage) is plotted at a granularity implied by the period (FR-78).

US-42: See my throughput

- As a user, I want to see tasks completed per day/week/month, so that I can gauge capacity.
- Acceptance Criteria: Throughput series are computed and displayed for the selected period (FR-79).

US-43: Keep my streak visible

- As a user motivated by streaks, I want current and longest streak shown, so that I stay consistent.
- Acceptance Criteria: Current and longest streak are shown for completing at least one task per day (FR-80).

US-44: Choose the period

- As a user, I want to change the time window, so that I can look at a week, month, or year.
- Acceptance Criteria: Period selection updates every metric and chart on the page consistently.

US-45: Inspect a data point

- As a user, I want exact values on demand, so that a chart is not the only source of truth.
- Acceptance Criteria: Hovering or tapping a series reveals the precise values and date for that point.

US-46: See a helpful empty state

- As a new user, I want guidance when there is no data yet, so that I don't think analytics are broken.
- Acceptance Criteria: No-data periods show an explanation and a call-to-action, not empty or broken charts.

US-47: Access analytics non-visually

- As a screen-reader user, I want the metrics available as text, so that charts are not a barrier.
- Acceptance Criteria: Every chart has a textual or table equivalent carrying the same values.

US-48: Keep day boundaries correct

- As a user, I want my analytics to respect my timezone, so that "today" means my today.
- Acceptance Criteria: Day-based metrics use the user's effective timezone; a timezone change does not retroactively shift historical records.

US-49: Keep analytics fresh

- As a user, I want metrics to reflect recent activity, so that the view is trustworthy.
- Acceptance Criteria: Metrics update after task completion within the product-wide interaction targets; the view indicates when it last refreshed.

US-50: Export my data

- As a user, I want to export my metrics, so that I can keep or share a record.
- Acceptance Criteria: A structured export (e.g. CSV) of the selected period is produced matching the on-screen values.

US-51: Keep analytics private

- As an individual user, I want my progress to be visible only to me, so that personal data stays personal.
- Acceptance Criteria: Analytics are scoped to the signed-in user; no other user can retrieve them.

F-5 — Integration Hub

US-52: Browse what I can connect

- As a user, I want a catalogue of source types, so that I know what is possible before committing.
- Acceptance Criteria: The catalogue lists connectable types with plain-language read/write descriptions (FR-101).

US-53: Connect an account safely

- As a user, I want to authorize a provider without typing my password into the app, so that my credentials stay safe.
- Acceptance Criteria: Connection uses a provider authorization handshake; the connection appears only after a successful callback (FR-102).

US-54: Manage existing connections

- As a user, I want to see and control my connections, so that I stay in charge of what is linked.
- Acceptance Criteria: A list shows label, type, account identity, state, and last-sync time, and supports lifecycle actions (FR-103).

US-55: Label my connections

- As a user with several accounts, I want to rename them, so that I can tell them apart.
- Acceptance Criteria: Connection labels can be set and edited and are shown everywhere the connection appears.

US-56: Pause and resume syncing

- As a user, I want to pause a connection, so that I can stop data flow without disconnecting.
- Acceptance Criteria: Pausing halts sync while preserving configuration; resuming restores sync without re-authorizing.

US-57: Recover an expired connection

- As a user whose token expired, I want a clear reconnect path, so that syncing resumes with minimal effort.
- Acceptance Criteria: Expired/error states are visible with a prominent reconnect action; successful reconnect clears the error state.

US-58: Disconnect completely

- As a user, I want to disconnect, so that no further data is read or written.
- Acceptance Criteria: Disconnecting revokes stored credentials and stops sync; the remaining imported data policy is stated to the user before confirming.

US-59: Know sync status

- As a user, I want last-sync time and state visible, so that I can tell whether data is current.
- Acceptance Criteria: Every connection shows its state (`connected`, `degraded`, `expired`, `error`, `paused`, `disconnected`) and last-sync time.

US-60: Understand a degraded integration

- As a user, I want plain-language errors, so that I know whether to retry or reconnect.
- Acceptance Criteria: Degraded/error states explain the cause in plain language and offer the appropriate recovery action.

US-61: Import work from a source

- As a user, I want items from a connected source to become tasks, so that my external work appears in one place.
- Acceptance Criteria: Items from a read-capable source are imported with source attribution; duplicates are not created on repeat syncs.

US-62: Write back to a source

- As a user, I want supported changes to sync outward, so that I don't maintain two systems.
- Acceptance Criteria: For write-capable sources, supported changes (e.g. completion) propagate outward; failures surface as connection errors.

US-63: Resolve sync conflicts predictably

- As a user with two-way sync, I want conflicts handled by a clear rule, so that data doesn't silently diverge.
- Acceptance Criteria: A documented conflict rule applies deterministically; when the rule cannot decide, the user is asked.

US-64: Protect my credentials

- As a security-conscious user, I want tokens stored and handled securely, so that a breach doesn't expose my accounts.
- Acceptance Criteria: Credentials are stored in an encrypted secret store, never in client-readable plaintext, and are revocable per connection.

US-65: Use an accessible connector UI

- As a user with assistive tech, I want the connection list and its actions accessible, so that I can manage integrations.
- Acceptance Criteria: Connection states are announced, actions have accessible names, and empty/error states are readable.

F-6 — Dark Mode

US-66: Pick my theme

- As a user, I want to choose light, dark, or system, so that the app suits my environment.

- Acceptance Criteria: Exactly one of `light`, `dark`, `system` is selectable and governs the whole product (FR-126).

US-67: Follow my system setting

- As a user, I want the app to follow my OS theme, so that it matches everything else on my device.
- Acceptance Criteria: On `system`, the effective theme resolves from the OS/browser signal and re-resolves live on change, without reload (FR-127).

US-68: Keep my choice next time

- As a user, I want my preference remembered, so that I don't re-select it every session.
- Acceptance Criteria: The preference persists across reload, sign-out/sign-in, and full restart on the same device (FR-128).

US-69: Switch instantly

- As a user, I want switching to be immediate, so that nothing looks half-updated.
- Acceptance Criteria: Changing the preference or the system signal re-themes every rendered surface without a reload (FR-129).

US-70: Avoid a flash of the wrong theme

- As a user, I want no bright flash on load in dark mode, so that the app doesn't dazzle me.
- Acceptance Criteria: The correct theme is applied before first paint; no light-to-dark flash occurs on reload.

US-71: Read comfortably in both themes

- As a user, I want sufficient contrast in either theme, so that content stays legible.
- Acceptance Criteria: Text and UI elements meet WCAG 2.1 AA contrast in both themes.

US-72: Keep imagery and charts consistent

- As a user, I want visuals tuned per theme, so that nothing becomes unreadable or jarring.
- Acceptance Criteria: Images, illustrations, and chart palettes have theme-appropriate variants; no chart series becomes indistinguishable.

US-73: Print and export sensibly

- As a user printing a page, I want legible output regardless of theme, so that documents are usable.
- Acceptance Criteria: Print styles use an appropriate ink-saving palette independent of the screen theme.

US-74: Keep my theme across devices

- As a signed-in user, I want my theme to follow me, so that each device feels consistent.
- Acceptance Criteria: When signed in, the preference is stored with the account and applied on other devices; the local choice wins only when the user is offline.

US-75: Theme every state

- As a user, I want error, empty, and loading states themed too, so that nothing breaks the look.
- Acceptance Criteria: Error, empty, loading, and modal surfaces are fully themed in both themes.

API Contracts

This section defines the functional, transport-level contracts for every feature F-1 .. F-6. Protocol binding (REST/GraphQL, sync vs. async delivery, storage engine, auth mechanism) is the Architect's decision; the contracts below state the observable behaviour the implementation must satisfy. No technology, framework, database, or vendor is selected here.

`API-` ids are local to this section, numbered sequentially from 1 and grouped by feature. `FR-`, `NFR-`, and `US-` ids are global and are referenced as plain text only — never re-defined.

0. Scope map

Feature	API ids	Contracts	Traces to
F-1 Intelligent Task Prioritization	API-1 .. API-9	9	FR-1 .. FR-25 range (FR-1, FR-2, FR-3, FR-4, FR-5, FR-6, FR-7, FR-8, FR-9)
F-2 Quick Add	API-10 .. API-14	5	FR-26 .. FR-30 range
F-3 Focus Mode	API-15 .. API-20	6	FR-51 .. FR-56 range
F-4 Progress Analytics	API-21 .. API-25	5	FR-76 .. FR-80 range
F-5 Integration Hub	API-26 .. API-35	10	FR-101 .. FR-103 range
F-6 Dark Mode	API-36 .. API-39	4	FR-126 .. FR-129 range

1. Cross-cutting API conventions

Convention	Contract
Base path & versioning	All contracts are served under a single versioned prefix (`/v1`). Changes are additive within a version; removals or semantic changes require a new version. Deprecations are announced with at least 12 months' notice and expose a `Deprecation` / `Sunset` response header.
Payload format	JSON request and response bodies, UTF-8. `Content-Type: application/json`. Responses that carry no body return no body (not an empty JSON object).
Identifiers	All entity ids are opaque strings. Clients must not parse meaning out of them, and ids are never reused.
Timestamps	All timestamps are UTC, ISO-8601 with offset. User-local display values are derived from the user's IANA timezone, never from a server-local clock.
Clock authority	For any timed feature (F-3 sessions, F-1 urgency, F-2 relative dates) the server clock is authoritative. Time-sensitive responses carry a `serverTime` field so a client with a skewed clock can self-correct.
Authorization scoping	Every endpoint operates strictly on data owned by the authenticated principal. There is no contract that reads or mutates another user's rows. Unauthenticated calls return `401`; authenticated calls that target out-of-scope resources return `404` (never `403` for existence-leaking resources) or `403` where existence is already known to the caller.
Error envelope	Every non-2xx response body uses one shape: <pre>{ "error": { "code": "<stable_machine_code>", "message": "<human readable>", "field": "<optional field path>", "details": { "requestId": "<traceable id>", "retryable": <bool> } } }</pre> . `code` values are stable strings (e.g. `invalid_weight_set`, `session_already_active`, `reconnect_required`). `requestId` is echoed in the response header for support correlation.

| Status codes in use | `200` OK, `201` Created, `202` Accepted (async work queued), `204` No Content, `207` Multi-Status (batch results), `400` malformed request, `401` unauthenticated, `403` forbidden, `404` unknown or out-of-scope, `409` state conflict, `410` gone/expired, `412` precondition failed, `413` payload too large, `422` semantically invalid, `429` rate limited, `500` unexpected server error, `502` upstream provider failure, `503` temporarily unavailable. |

| Idempotency | Every state-changing `POST`, `PATCH`, and `DELETE` accepts an `Idempotency-Key` header. A replay with the same key and body returns the original status and body for 24 hours; a replay with the same key and a different body returns `409 idempotency\_key\_reuse`. |

| Concurrency control | Mutable resources expose an `ETag` and accept `If-Match` on mutation. A stale `If-Match` returns `412 stale\_precondition` with the current representation in the error `details`. |

| Pagination | Cursor-based: request `?limit=&cursor=`, response `{ "items": [...], "nextCursor": "<opaque|null>" }`. Default `limit` 50, maximum 200; `limit` outside range returns `422 invalid\_limit`; a malformed or expired cursor returns `422 invalid\_cursor`. |

| Rate limiting | `429` responses carry `Retry-After` and the rate-limit headers. Interactive endpoints (F-2 quick add, F-3 session controls, F-6 preference writes) are rate limited per principal and must remain usable under burst while bulk/analytics endpoints are limited more tightly. |

| Freshness disclosure | Derived reads (ranking, analytics, sync state) return the instant they were computed plus a freshness value, so a client can label stale data instead of silently showing it as current. |

| Secrets | No response body, error detail, log line, or analytics payload ever contains a provider credential, refresh token, or authorization code. Integration credentials are write-only: accepted (if at all) only through the handshake in API-27/API-28 and never read back. |

| Audit | Every mutation records actor, entity, before/after values where applicable, and timestamp. Audit records are readable by the owning principal on request; they are never silently discarded. |

| Domain events | Successful mutations emit an internal event (`task.created`, `task.completed`, `focus\_session.completed`, `integration.connection\_state\_changed`, `integration.sync.completed`, `preference.theme\_changed`) that downstream features consume — F-4 analytics consumes task and focus-session events; F-1 rank recomputation consumes task mutations. Event delivery is at-least-once; consumers must be idempotent. |

2. F-1 — Intelligent Task Prioritization — API Contracts

ID	Method / Path	Purpose (traces)	Request (key fields)	Success	Failures
API-1	`GET /v1/tasks?state=active&sort=priority&limit=&cursor=`	Ranked "Today" queue of active tasks by descending score (FR-2)	`state` (default `active`), `sort`, `limit`, `cursor`, optional `projectId`, `tag`	`200 { "items": [ { "taskId", "title", "priorityScore", "rank", "factors": { "dueUrgency", "importance", "effort", "age", "dependencyState" }, "override": { ... } \} null, "dueAt" } ], "nextCursor", "computedAt", "profileVersion" }` — order is stable across identical reads	`400 invalid_sort`, `422 invalid_limit`, `422 invalid_cursor`, `401`
API-2	`GET /v1/tasks/{taskId}/priority`	Per-task score with factor breakdown (FR-1, FR-3, FR-9)	Path `taskId`; optional `at=<timestamp>` to evaluate the score at a past/future instant	`200 { "taskId", "score", "rank", "computedAt", "profileVersion", "factors": [ { "name", "rawValue", "normalisedValue", "weight", "contribution" } ], "effectiveUrgency", "escalationActive": <bool> }`	`404 unknown_task`, `409 task_not_active` (completed/deleted task has no live score), `422 invalid_at`
API-3	`GET /v1/tasks/{taskId}/priority/explanation`	Plain-language reason for the current rank (FR-5)	Path `taskId`; optional `locale`	`200 { "taskId", "summary": "<one sentence>", "dominantFactors": [ { "name", "text" } ], "overrideNote": "<string null>", "generatedAt", "profileVersion", "locale" }`	Text must name dominant factors in the user's language without internal jargon, weights, or raw scores. `404 unknown_task`, `409 task_not_active`, `406 unsupported_locale`

| API-4 | `GET /v1/me/prioritization-profile` | Read current factor weights (FR-4) | — | `200 { "profileVersion", "weights": { "dueUrgency", "importance", "effort", "age", "dependencyState" }, "updatedAt" }` | `401` |

| API-5 | `PUT /v1/me/prioritization-profile` | Set relative factor weights (FR-4) | `{ "weights": { ... }, "ifVersion": <int> }`; each weight numeric in a fixed supported band, at least one weight strictly positive | `200` new profile with incremented `profileVersion`; the queue is recomputed asynchronously (see API-8) | `422 invalid\_weight\_set` (unknown factor, negative, all-zero), `412 stale\_precondition` (ifVersion mismatch, `details.current` returned), `401` |

| API-6 | `POST /v1/tasks/{taskId}/overrides` | Create a pin or forced rank position (FR-6, FR-7) | `{ "mode": "pin" \| "position", "position": <int> \| null, "expiry": { "type": "never" \| "until" \| "onCompletion", "until": <timestamp> \| null } }` | `201 { "overrideId", "taskId", "mode", "requestedPosition", "effectiveRank", "expiresAt": <timestamp\|null>, "createdAt" }`; the task's position in API-1 reflects the override on the next read | `404 unknown\_task`, `409 override\_exists` (details carries the active override id), `422 invalid\_position` (below 1 or beyond queue length + 1), `422 expiry\_in\_past`, `422 invalid\_expiry\_type` |

| API-7 | `DELETE /v1/tasks/{taskId}/overrides/{overrideId}` | Remove an override so natural ranking resumes (FR-8) | Path ids | `204`; repeat calls on an already-removed override also return `204` (idempotent) | `404 unknown\_task`, `404 unknown\_override` |

| API-8 | `POST /v1/prioritization/recomputations` | Request a full rescore of the principal's active tasks (FR-1, FR-2) | `{ "reason": "manual" \| "profile\_change" \| "bulk\_edit", "scope": "all\_active" }` | `202 { "jobId", "scope", "requestedAt", "estimatedCompletion" }` | `409 recomputation\_in\_progress` (details carries the running `jobId`), `401` |

| API-9 | `GET /v1/prioritization/recomputations/{jobId}` | Poll recomputation status (FR-1) | Path `jobId` | `200 { "jobId", "status": "queued" \| "running" \| "succeeded" \| "failed", "tasksRescored", "startedAt", "finishedAt", "error": <error envelope \| null> }` | `404 unknown\_job`, `401` |

****Ordering guarantees (F-1).**** API-1 order is deterministic for identical inputs and identical profile version: ties break on due date ascending, then explicit importance descending, then creation time ascending. A task with an active `pin` override is always returned at rank 1; a task with a `position` override occupies the requested rank and all other tasks shift down without reordering among themselves.

****Consistency (F-1).**** API-1 and API-2 may serve a read that is up to a bounded staleness behind the most recent mutation; the bound and the `computedAt` value are the contract. After API-8 reports `succeeded`, subsequent reads must reflect the new profile.

3. F-2 — Quick Add — API Contracts

ID	Method / Path	Purpose (traces)	Request (key fields)	Success	Failures
API-10	POST /v1/quick-add/parse	Dry-run interpretation of one free-text line; nothing is persisted (FR-30)	{ "text": "<line>", "timezone": "<IANA>", "now": <timestamp\ null> } `200 { "title", "dueAt": <timestamp\ null>, "importance": <int\ null>, "tags": [ ... ], "projectId": <string\ null>, "contexts": [ ... ], "parseWarnings": [ { "code", "token", "message" } ], "unparsedTokens": [ ... ] }`. Tokens that are not recognised remain part of `title`. `400 empty_input`, `413 input_too_long` (limit 500 characters), `422 invalid_timezone`, `401`		
API-11	POST /v1/quick-add	Single-confirm capture: parse and create in one call (FR-26, FR-27, FR-28, FR-29)	{ "text": "<line>", "timezone": "<IANA>", "overrides": { "title"? , "dueAt"? , "importance"? , "tags"? , "projectId"? } } + `Idempotency-Key` `201 { "taskId", "title", "dueAt", "importance", "tags", "projectId", "state": "active", "parsedFields": <same shape as API-10>, "warnings": [ ... ], "createdAt" }` `400 empty_input`, `413 input_too_long`, `422 invalid_timezone`, `422 invalid_override` (explicit		

override contradicts the parser contract, e.g. malformed timestamp), `409 idempotency\_key\_reuse`, `401` |

| API-12 | `POST /v1/quick-add/{draftId}/confirm` | Two-step variant: commit a previewed draft the user may have edited (FR-30) | `{ "accepted": { "title", "dueAt", "importance", "tags", "projectId" } }` | `201` task payload identical in shape to API-11 | `404 unknown\_draft`, `410 draft\_expired` (draft TTL is 15 minutes and is returned as `expiresAt` when the draft is created), `409 draft\_already\_confirmed` (details carries the created `taskId`), `422 invalid\_field` |

| API-13 | `DELETE /v1/tasks/{taskId}?reason=quick\_add\_undo` | Undo an accidental capture inside the undo window (FR-26) | Path `taskId`, query `reason` | `204`; repeat calls return `204` (idempotent) | `409 undo\_window\_expired` (the task may still be deleted through normal deletion), `403 undo\_not\_applicable` (task was created by a path other than quick add), `404 unknown\_task` |

| API-14 | `POST /v1/quick-add/batch` | Replay a client-side capture queue captured while offline | `{ "items": [ { "clientId", "text", "capturedAt", "timezone" } ] }`, max 100 items, each with its own idempotency basis on `clientId` | `207 { "results": [ { "clientId", "status": "created" | "duplicate" | "rejected", "taskId": <string|null>, "warnings": [ ... ], "error": <error envelope|null> } ] }` — a partial failure never rejects the whole batch | `413 batch\_too\_large`, `422 too\_many\_items`, `401` |

****Capture guarantees (F-2).**** An unparseable or ambiguous date/time expression is ****never**** a creation failure: the field is left unset, the token stays in the title, and the response carries a `parseWarnings` entry (API-10, API-11). Only structurally invalid input (empty text, over-length text, invalid timezone, invalid explicit override) is rejected. The `dueAt` value returned for relative expressions such as "in 2 hours" is resolved against the server clock at creation time and returned absolute, so it is stable across later reads.

4. F-3 — Focus Mode — API Contracts

ID	Method / Path	Purpose (traces)	Request (key fields)	Success	Failures
API-15	POST /v1/focus-sessions	Start a session bound to one task (FR-51, FR-52, FR-53, FR-55)	{ "taskId", "plannedMinutes": <int null>, "startImmediately": <bool> } + `Idempotency-Key`	201 { "sessionId", "taskId", "state": "running" "paused", "plannedMinutes", "startedAt", "elapsedSeconds", "remainingSeconds", "serverTime", "expiresAt" }	404 unknown_task, 409 session_already_active (details carries `activeSessionId`, `taskId`, `remainingSeconds` so the client can offer the explicit decision in BR-3), 422 duration_out_of_range (details carries the supported min/max), 401
API-16	GET /v1/focus-sessions/active	Read the single active session, if any (FR-55)	—	200 session payload identical in shape to API-15, with `serverTime` and `remainingSeconds` recomputed for this read; 204 when no session is active	401
API-17	PATCH /v1/focus-sessions/{id}	Pause, resume, extend, or end a session (FR-54, FR-56)	{ "action": "pause" "resume" "extend" "end", "extendMinutes": <int null>, "reason": <string null> }	200 updated session payload including `state`, `elapsedSeconds`, `remainingSeconds`, `serverTime`, and — for `end` — `completedAt`, `actualMinutes`, `outcome`; `extend` returns the new `expiresAt`	404 unknown_session, 409 invalid_transition (e.g. `resume` on an ended session; details names the current state), 410 session_expired (server-side timer already elapsed before the call arrived), 422 extend_out_of_range, 401
API-18	GET /v1/me/focus-settings	Read the session defaults and supported band (FR-52, FR-53)	—	200 { "defaultMinutes", "minMinutes", "maxMinutes", "pauseAllowed": <bool>, "notificationPreference": "in_app" "os" "both" "none", "updatedAt" }	401
API-19	PUT /v1/me/focus-settings	Change defaults for future sessions (FR-52, FR-53)	{ "defaultMinutes", "notificationPreference"? }	200 updated settings; `defaultMinutes` must be a whole number inside the supported band	422 out_of_range (details carries `minMinutes`/`maxMinutes`), 422 invalid_notification_preference, 412 stale_precondition, 401

| API-20 | `GET /v1/focus-sessions?taskId=&from=&to=&limit=&cursor=` | Session history for the task and for F-4 rollups | Filters `taskId`, `from`, `to`, `state`; pagination params | `200` { "items": [{ "sessionId", "taskId", "state", "plannedMinutes", "actualMinutes", "startedAt", "endedAt", "outcome" }], "nextCursor" } | `422` range_invalid` (from` after `to`), `422` range_too_wide` (maximum 366 days), `404` unknown\_task`, `401` |

****Timer guarantees (F-3).**** Session duration is authoritative on the server. Every session response carries `serverTime` and `remainingSeconds`; a client that resumes after sleep reconciles to those values rather than to its local timer. A session that elapses while the client is absent transitions server-side to an ended state and is reported by API-16/API-17 with `state = "ended"` plus its `outcome`, rather than being silently dropped. Only one session per principal may be in a non-ended state (FR-55); the conflict response is the sole path that lets the client start a competing session, and it requires an explicit client decision before it retries.

5. F-4 — Progress Analytics — API Contracts

ID	Method / Path	Purpose (traces)	Request (key fields)	Success	Failures
API-21	GET /v1/analytics/summary	Single-surface headline metrics (FR-76, FR-77, FR-78)	period ∈ {7d, 30d, 90d, custom}; from/to required when custom; timezone required (IANA)	200 { "period": { "from", "to", "granularity" }, "timezone", "created": <int>, "completed": <int>, "net": <int>, "completionRate": <decimal 0..1>, "completionRateSeries": [{ "bucketStart", "value" }], "tasksCompletedPerBucket": [{ "bucketStart", "value" }], "computedAt", "dataFreshnessSeconds" }	422 invalid_period, 422 range_invalid, 422 range_too_wide (366-day maximum), 422 invalid_timezone, 422 range_in_future (from after now), 401
API-22	GET /v1/analytics/completion-rate	Completion-rate time series (FR-78)	granularity ∈ {day, week, month}; from, to, timezone	200 { "granularity", "timezone", "series": [{ "bucketStart", "bucketEnd", "completed", "created", "rate" }], "computedAt" }	422 granularity_not_supported_for_range (e.g. day over more than 92 days), 422 range_invalid, 422 invalid_timezone, 401
API-23	GET /v1/analytics/throughput	Tasks completed per day/week/month (FR-79)	Same shape as API-22	200 { "granularity", "timezone", "series": [{ "bucketStart", "bucketEnd", "completed" }], "computedAt" }	Same as API-22
API-24	GET /v1/analytics/streaks	Current and longest completion streaks (FR-80)	timezone (required)	200 { "currentStreakDays", "currentStreakStart": <date null>, "longestStreakDays", "longestStreakRange": { "from", "to" } null, "asOfDate", "computedAt" }	422 invalid_timezone, 401
API-25	GET /v1/analytics/export	Export the metrics or the underlying task events for the selected period	format ∈ declared set, Content-Type-negotiated	200 streaming document when the result fits the synchronous limit; otherwise 202 { "jobId", "estimatedCompletion" } plus a status read at /v1/analytics/export/{jobId}	422 unsupported_format, 422 range_too_wide, 422 invalid_timezone, 429 export_rate_limited, 401

****Read-only and completeness guarantees (F-4).**** Every analytics contract is read-only. A period with no activity returns `200` with zero-filled buckets — never `404` and never a missing series. A `to` value in the future is clamped to the request time and the clamped value is echoed in `period`. All bucket boundaries are computed in the caller's declared timezone, and the timezone used is always echoed so a client can verify the day boundary it is displaying. `dataFreshnessSeconds` is disclosed so a stale dashboard is labelled rather than presented as live. Task, completion, and focus-session inputs to these aggregates are consumed from the domain events declared in §1; the underlying task and

session records remain owned by F-1, F-2, and F-3.

6. F-5 — Integration Hub — API Contracts

ID	Method / Path	Purpose (traces)	Request (key fields)	Success	Failures
API-26	GET /v1/integrations/catalogue	Browsable catalogue of connectable source types (FR-101)	Optional `type` filter 200 { "items": [{ "type", "displayName", "description", "capabilities": { "read": [...], "write": [...] }, "handshake": "redirect", "scopes": [...], "requiredFields": [...], "availability": "available" \ "unavailable" }] } — description states in plain language what data is read and written	401	
API-27	POST /v1/integrations/connections	Begin the authorization handshake for a source type (FR-102)	{ "type", "label" } + `Idempotency-Key`	201 { "connectionId", "state": "pending_authorization", "authorizationUrl", "stateTokenExpiresAt" }	422 unknown_type, 422 provider_unavailable, 422 credentials_not_accepted (any provider credential field in the body is rejected by design), 409 connection_already_pending, 401
API-28	GET /v1/integrations/callback?state=&code=&error=	Complete the handshake and materialise the connection (FR-102)	`state` (single-use, expires), `code` or `error` from the provider Redirect to the application with a success marker and the `connectionId`; the connection then appears in API-29 with state `connected`, `accountIdentity`, and `lastSyncAt`	400 invalid_state (unknown, replayed, or expired), 400 missing_code, 403 principal_mismatch, 502 provider_exchange_failed (no connection is created; the caller may start again with API-27), 409 duplicate_connection (details carries the existing `connectionId`)	
API-29	GET /v1/integrations/connections	List connections with identity, state, and last sync (FR-103)	Optional `state` filter, pagination params 200 { "items": [{ "connectionId", "type", "label", "accountIdentity", "state": "connected" \ "degraded" \ "expired" \ "error" \ "paused" \ "disconnected", "lastSyncAt": <timestamp\ null>, "lastError": { "code", "message", "at" } \ null, "scopes", "pausedAt": <timestamp\ null> }], "nextCursor" } — never returns tokens or secrets	422 invalid_state, 401	
API-30	PATCH /v1/integrations/connections/{id}	Rename, pause, or resume a connection (FR-103)	{ "label"? : <string>, "state"? : "paused" \ "active" }	200 updated connection payload (shape of API-29 item)	404 unknown_connection, 422 invalid_label, 409 reconnect_required (resuming an `expired` or `error` connection), 409 invalid_transition (e.g. `active` on a `disconnected` connection), 412 stale_precondition, 401
API-31	POST /v1/integrations/connections/{id}/reconnect	Re-run the handshake for an existing connection (FR-103)	{ "label"? } + `Idempotency-Key`	201 { "connectionId", "state": "pending_authorization", "authorizationUrl", "stateTokenExpiresAt" }	404 unknown_connection, 409 already_connected, 422 provider_unavailable, 401
API-32	DELETE /v1/integrations/connections/{id}	Disconnect and revoke access (FR-103)	Path id; optional `?purgeImportedData=<bool>`	202 { "connectionId", "state": "disconnected", "revocation": { "status": "succeeded" \ "pending", "attemptedAt" } } — repeat calls return 202 (idempotent; `revocation.status` may advance from `pending` to `succeeded` on a later read)	404 unknown_connection, 409 purge_in_progress (details carries the running purge job id), 401
API-33	POST /v1/integrations/connections/{id}/sync	Trigger a manual sync of an existing connection	{ "scope": "incremental" \ "full", "reason": "manual" \ "post_reconnect" } + `Idempotency-Key`	202 { "syncRunId", "connectionId", "scope", "status": "queued", "requestedAt", "estimatedCompletion" }	404 unknown_connection, 409 sync_in_progress (details carries the running `syncRunId`), 409 connection_not_syncable (state is `paused`, `expired`, `error`, or `disconnected`; details names the state and the required recovery action, e.g. API-31), 429

sync_rate_limited` (with `Retry-After`), `502 provider_unavailable` (details carries `retryable: true`),
`401` |
| API-34 | `GET /v1/integrations/connections/{id}/sync-runs?status=&limit=&cursor=` | Sync run history
with per-run outcome counts and bounded failure detail | Optional `status` filter, pagination params |
`200 { "items": [{ "syncRunId", "connectionId", "scope", "status": "queued" \| "running" \| "succeeded" \|
"partial" \| "failed", "startedAt", "finishedAt": <timestamp\|null>, "counts": { "read", "created", "updated",
"skipped", "failed" }, "failureCount", "failuresTruncated": <bool>, "failures": [{ "itemRef", "code",
"message", "retryable", "at" }], "nextRetryAt": <timestamp\|null> }], "nextCursor", "computedAt" }` —
`failures` is capped and `failuresTruncated` plus `failureCount` disclose the cap | `404`
unknown_connection`, `422 invalid_status`, `422 invalid_limit`, `422 invalid_cursor`, `401` |
| API-35 | `POST /v1/integrations/webhooks/{sourceToken}` | Inbound ingestion endpoint for the
generic webhook bridge source type; machine-to-machine, single-purpose rotatable token, not a user
session | Raw JSON body; `X-Signature` (HMAC over the raw body, constant-time verified),
`X-Delivery-Id` (provider-assigned, used for dedupe), optional `X-Timestamp` for replay-window
enforcement | `202 { "accepted": true, "duplicate": false, "deliveryId", "connectionId", "receivedAt" }` —
ingestion is always asynchronous; a replayed `X-Delivery-Id` within the dedupe window returns `200 {
"accepted": true, "duplicate": true, "deliveryId" }` | `401 invalid_signature` (also returned when
`X-Timestamp` falls outside the accepted replay window, details carries `reason`), `404`
unknown_source_token` (also returned for a revoked token, so revocation is not distinguishable from
non-existence), `413 payload\_too\_large`, `429` (rate limited, with `Retry-After`), `503`
ingestion_unavailable` (explicitly `retryable: true` so the provider retries) |

****Integration guarantees (F-5).****

- ****No credentials in the product's own fields (FR-102).**** API-27 rejects any body containing a provider credential field (`422 credentials\_not\_accepted`). Credentials exist only inside the handshake completed at API-28 and are never returned by API-29, API-30, API-31, API-34, or any error envelope.
- ****Connectivity is asserted only after callback success.**** A connection created by API-27 is in `pending\_authorization` and is absent from the user-facing "connected" view until API-28 completes. A failed or abandoned handshake leaves no connection and no partial state; `state` tokens are single-use and expire, and a replayed or expired token yields `400 invalid\_state`.
- ****State is always recoverable, never a dead end.**** Every non-`connected` state returned by API-29 maps to a named recovery contract: `expired` and `error` → API-31; `paused` → API-30; `degraded` → API-33 then API-34; `disconnected` → API-27. `lastError` always carries a stable `code`, a human message, and a timestamp, so the UI never has to render an unexplained failure.
- ****Partial sync never presents as full success.**** A run with mixed results is reported as `partial` with `counts` and a bounded `failures` list (API-34), never as `succeeded`. Retryable failures carry `retryable: true` and `nextRetryAt`.
- ****Revocation is attempted and its outcome is disclosed.**** API-32 reports `revocation.status` rather than assuming provider-side revocation succeeded, and `purgeImportedData` is opt-in — imported user data is not silently deleted.
- ****Inbound ingestion never validates synchronously.**** API-35 acknowledges delivery and defers interpretation, so a malformed payload surfaces as a `failed`/`partial` run in API-34 instead of a provider-visible error that would trigger duplicate retries.
- ****Identity is never inferred.**** `accountIdentity` is supplied by the provider callback; the contract never derives it from a user-entered label.

7. F-6 — Dark Mode — API Contracts

ID	Method / Path	Purpose (traces)	Request (key fields)	Success	Failures
---	---	---	---	---	---

| API-36 | `GET /v1/me/preferences/theme?systemSignal=light|dark|no-preference` | Read the persisted preference and its effective resolution (FR-126, FR-127, FR-128) | Optional `systemSignal` reports the host OS/browser colour-scheme signal observed by the client | `200` { "preference": "light" \| "dark" \| "system", "resolvedTheme": "light" \| "dark" \| null, "resolutionSource": "user_preference" \| "system_signal" \| "account_default", "requiresClientResolution": <bool>, "tokensVersion", "updatedAt", "serverTime" } . When `preference` is `system` and `systemSignal` is supplied, `resolvedTheme` mirrors it; when `systemSignal` is omitted, `resolvedTheme` is `null` and `requiresClientResolution` is `true` (the server never guesses another device's OS signal). | `422 invalid\_system\_signal`, `401` |

| API-37 | `PUT /v1/me/preferences/theme` | Set the single whole-product theme preference (FR-126, FR-128, FR-129) | { "preference": "light" \| "dark" \| "system", "systemSignal"? : "light" \| "dark" \| "no-preference", "ifVersion"? : <int> } + `Idempotency-Key` | `200` payload identical in shape to API-36 with the new preference applied and `updatedAt` advanced; the change is durable before the response is sent | `422 invalid\_preference` (value outside the closed set), `422 invalid\_system\_signal`, `412 stale\_precondition` (ifVersion mismatch; `details.current` returned), `409 idempotency\_key\_reuse`, `401` |

| API-38 | `GET /v1/theme/tokens?theme=light|dark&version=` | Read the versioned design-token set for one theme, so every surface themes from one source (FR-129) | `theme` required; optional `version` to pin a known token set | `200` { "theme", "version", "tokens": { "<tokenName>": "<value>" }, "generatedAt" } with `ETag` and a long-lived `Cache-Control`; `304 Not Modified` on a matching `If-None-Match` | `422 invalid\_theme`, `422 unknown\_token\_version`, `401` (token set is readable only by an authenticated principal) |

| API-39 | `DELETE /v1/me/preferences/theme` | Reset the preference to the account default (`system`) (FR-126, FR-128) | — | `204`; repeat calls also return `204` (idempotent) | `401` |

****Theme guarantees (F-6).****

- ****Exactly one preference, product-wide (FR-126).**** `preference` is a closed enum of three values and is stored per principal. There is deliberately ****no**** contract that sets a theme for a single screen, route, or widget — per-surface theming is not expressible in this API.
- ****Live system following without a round trip (FR-127).**** A change to the host colour-scheme signal is resolved on the client and applied immediately; it does not require a server write and must not trigger a reload. The server participates only to echo `resolvedTheme` when the client chooses to report its signal via `?systemSignal=`.
- ****Persistence and flash-free cold start (FR-128).**** `preference` is durable server-side and re-applied on the next read after sign-out/sign-in, hard reload, or app restart. Clients may cache the last known { preference, resolvedTheme, tokensVersion } so the correct theme is applied before first paint; a cache miss must fall back to `account\_default`, never to a light-only render.
- ****Immediate, complete application (FR-129).**** API-37 returns the fully resolved state in the same response, and API-38 pins the token set by `version`, so a client can apply the change optimistically and reconcile on the next read without any partially themed state. Because tokens are versioned and `ETag`-cached, a theme change never requires re-deriving colour values per component.
- ****No preference is unreachable.**** `no-preference` from the system signal is a valid input that surfaces as `account\_default` resolution rather than an error.
- ****Invalid values are rejected, never coerced.**** A preference outside `light|dark|system` is `422 invalid\_preference`; the contract never silently substitutes a default, so a client bug cannot silently change a user's theme.

8. Cross-contract guarantees

Guarantee	Contract obligation

| **Offline tolerance** | Contracts that create or mutate user work are safe to queue client-side and replay: API-11 and API-14 (quick add), API-17 (session control, reconciled against `serverTime`), API-37 (theme). Every replayable call is idempotent by `Idempotency-Key` or by the client-supplied item id in API-14. Read contracts (API-1, API-21..API-25, API-29, API-34, API-36) are safe to serve from cache but must carry and display their `computedAt`/`dataFreshnessSeconds`/`updatedAt` value rather than presenting cached data as live. |

| **No cross-principal access** | Every endpoint is scoped to the authenticated principal; the webhook ingress (API-35) authenticates by scoped token, not by session, and yields no read access to any other principal's data. |

| **No secret egress** | No contract returns provider credentials, access/refresh tokens, signing keys, or source tokens. Tokens are write-only or single-use, and appear in no error envelope, `details` object, or log-correlated `requestId` payload. |

| **Stable machine codes** | Every `4xx`/`5xx` carries a `code` from §1 plus a `retryable` boolean. Clients branch on `code` and `retryable`; human-readable `message` is never parsed. |

| **Explicit freshness** | Derived reads (`priorityScore`, `rank`, analytics buckets, `lastSyncAt`, `resolvedTheme`) always disclose when the value was computed. The API never presents a derived value without its computation instant. |

| **Reversible user actions** | Actions with destructive or surprising effects are reversible through a contract: quick-add capture (API-13), override removal (API-7), disconnect without purge (API-32), theme reset (API-39). |

| **Bounded work** | List reads are paginated (§1), sync failure detail is capped with a truncation flag (API-34), and analytics/export ranges are bounded with an explicit `422` rather than a silent partial result (API-21..API-25). |

| **Feature coverage** | Every contract above belongs to exactly one feature and only reuses ids within its own range: F-1 → API-1..API-9, F-2 → API-10..API-14, F-3 → API-15..API-20, F-4 → API-21..API-25, F-5 → API-26..API-35, F-6 → API-36..API-39. |

9. API-level open questions

These are scoped to this section and are for the user to decide; they are not resolved by this contract set.

ID	Question	Why it matters

| OQ-1 | Should the theme preference be per-principal (shared across that user's devices) or per-device? | FR-128 requires persistence on the same device and is satisfied either way; a per-principal model makes a preference follow the user across devices, while a per-device model requires additional client-local storage semantics not expressed in API-36/API-37. |

| OQ-2 | Should a focus session that elapses server-side while the client is offline be recorded as `completed` or as `abandoned`? | API-17 and API-20 both surface an `outcome` field, but the value set is not fixed by FR-54/FR-56. |

| OQ-3 | Should quick-add undo (API-13) be exposed as a single most-recent action or as a short, addressable history? | FR-26 specifies one confirm action; the undo window duration and depth are not specified upstream. |

| OQ-4 | Should a `position` override (API-6) be allowed beyond the current queue length? | API-6 currently rejects positions below 1 or beyond queue length + 1; permitting far-future positions would require the contract to define what happens when the queue grows or shrinks. |

| OQ-5 | Is a completion streak (FR-80) measured in calendar days in the user's timezone only, or may a user choose a custom day boundary? | API-24 requires a `timezone` and computes it; a custom boundary would widen the contract. |

Design Direction

The product reads as a **calm, high-signal personal command center** — the visual opposite of a noisy kanban board. Its mood is **quiet confidence**: deliberately low-chrome and low-anxiety, so that a busy professional opening the app feels the day get smaller rather than larger. The visual language is built on three pillars — **clarity, momentum, and restraint**. Clarity means one dominant thing per screen: the ranked "Today" queue (F-1) is a single vertical spine of tasks, never a grid of competing panels; the capture surface (F-2) is a single line floating above everything, never a form. Momentum means the interface always shows forward motion — tasks visibly rise, complete, and fall away with soft, short transitions, and a Focus session (F-3) collapses the entire product down to one task, one timer, one exit. Restraint means color is spent, not sprayed: a near-neutral canvas with a single confident accent reserved for the thing that matters right now (top-ranked task, active timer, primary action), while everything else is expressed through weight, spacing, and hierarchy rather than hue.

Visually the system is **typographic first and chromatic second**. A generous type scale does the heavy lifting: task titles are the largest, most legible element in the app; metadata (due time, effort, tags, rank explanation) is consistently smaller, lower-contrast, and never competes with the title. Spacing is spacious on the queue and tight inside dense analytical surfaces (F-4), so that "calm" and "information-rich" coexist without contradiction. Surfaces are layered rather than bordered — a soft, shallow elevation model communicates what is background, what is card, and what is an overlay, keeping the app free of a heavy box-and-line grid. Iconography is a single line-weight family, used sparingly and always paired with a label the first time it appears; nothing important is conveyed by icon or color alone.

Color is defined as **semantic roles, not fixed values** — canvas, surface, elevated surface, primary text, secondary text, hairline, accent, and a small set of state colors (success/complete, warning/at-risk, danger/overdue, info) plus connection health states for the Integration Hub (F-5: `connected`, `degraded`, `expired`, `error`, `paused`, `disconnected`). Because theming is a first-class product feature (F-6), every role must resolve to a legible value in both light and dark, and the dark theme must be designed as its own considered palette — deeper, desaturated surfaces with slightly warmed accents — not an inversion of light. No surface, chart series, or state token may rely on a value that only works in one theme, and every pairing must meet the accessibility contrast targets in §3.2. Priority is expressed through **position, weight, and a restrained accent bar**, never through a rainbow of severity colors — the ranking (F-1) should be readable at a glance from the shape of the list alone.

Motion is **short, purposeful, and reversible**: 120–200 ms for state changes, ease-out on entry, and no animation that delays a user from acting. Rank reorderings animate so the user can follow what moved and why; the ranking explanation (F-1) expands inline rather than hijacking the screen. Focus Mode (F-3) is the one place the product is allowed to feel cinematic — a dimmed periphery and a single elevated timer — and, correspondingly, the one place where exiting must always be one obvious, unmissable action. Analytics (F-4) prefers one honest series over a dashboard of competing charts: a small number of well-labeled visualizations, with the underlying numbers always readable as text, and never a decorative chart that cannot be explained in words.

The overall posture toward the user is **non-judgmental mentorship**: the app nudges (streaks, at-risk flags, suggested next task) but never scolds, gamifies aggressively, or reddens a screen. Empty and error states are written in the same voice — plain, warm, actionable — and the design direction treats "nothing here yet" as a first-class, attractive screen rather than an absence to be patched with a spinner.

Components

The component system is layered in four tiers so that no feature (F-1 ... F-6) re-implements shared behaviour. Naming is by noun-role only — **no framework, library, vendor, or rendering technology is implied by any component name**; technology binding remains the Architect's decision.

| Tier | Contents | Product knowledge |

|---|---|---|

| 1. Primitives | Tokens + atoms | none |

| 2. Composed components | Reusable compounds | product-aware, feature-agnostic |

| 3. Feature components | Bound to exactly one feature (F-n) | feature-specific |

| 4. Surfaces | Screens / overlays assembled from tiers 1–3 | screen-specific |

****Universal contract.**** Every component, in every tier, is *theme-aware* (consumes F-6 tokens, FR-126 ... FR-129) and *state-complete*: each declares ****default / hover / focus-visible / active / disabled / loading / empty / error**** variants. No component ships without its empty, loading, and error state.

1. Primitives

1.1 Design tokens

| Token group | Purpose | Consumers |

|---|---|---|

| Typography scale (Display / Title / Body / Label / Caption / Mono) | Typographic-first hierarchy;

****Mono is reserved**** for F-3 timer digits and F-4 metric numbers | all |

| Spacing scale (4-pt base) | Rhythm + low-chrome density | all |

| Colour tokens (canvas, surface, surface-raised, text-primary/secondary/muted, border, accent, accent-contrast, semantic success/warning/danger/info) | Single-accent restraint — colour is *spent*, not *sprayed*; semantic colours reserved for state | all |

| Elevation tokens (flat / raised / overlay) | Quiet depth, used sparingly | all |

| Motion tokens (duration fast/base/slow; easing standard/enter/exit) | Soft, short transitions — visible forward momentum | all |

| Radius + border tokens | Consistent, calm geometry | all |

| Theme token set (light values / dark values / system-resolved) | F-6 theming | all |

1.2 Atoms (product-agnostic)

`Button` (primary / secondary / ghost / destructive / link) · `IconButton` · `InputText` · `TextArea` · `Checkbox` · `RadioGroup` · `Switch` · `Slider` (consumed by the F-1 weight editor) · `Select` / `Dropdown` · `Combobox` · `Chip` / `Tag` / `Pill` · `Badge` / `StatusDot` · `Tooltip` · `Popover` / `HoverCard` · `Dialog` / `Modal` · `Drawer` / `Sheet` · `Toast` / `Snackbar` · `Skeleton` · `ProgressBar` · `ProgressRing` · `Spinner` (indeterminate, only for ≥ 2 s waits) · `Avatar` · `Divider` · `Card` / `Surface` · `Link` · `Icon` (vector set) · `InlineError` / `HelperText` · `FocusRing` (2 px visible outline) · `VisuallyHidden` (a11y).

2. Composed components

| Component | Purpose | Traces to |

|---|---|---|

| `AppShell` | Chrome: header, primary-nav region, content region, and global slots (capture bar, active-session indicator) | all |

| `PrimaryNavigation` | Desktop rail / mobile bottom tabs (≤ 5 items) with current-section affordance | all |

| ``CaptureBarHost`` | Global single-line capture mount point; reachable from every primary screen | F-2 (FR-26) |

| ``VirtualizedList`` | Long-list rendering for queue, connections, sync logs | F-1, F-5 |

| ``ListRow`` (generic) | Row scaffold: leading slot · body · meta · trailing actions | F-1, F-4, F-5 |

| ``TimeRangePicker`` | Period selection feeding charts and metrics | F-4 |

| ``ChartFrame`` | Chart chrome — title, legend, axes — plus empty / loading / error / insufficient-data states | F-4 |

| ``MetricTile`` | Single-number stat with label, delta, and trend affordance | F-4 |

| ``SettingsSection`` / ``PreferenceRow`` | Grouped settings scaffolding | F-1, F-3, F-5, F-6 |

| ``ConfirmDialog`` | Destructive / irreversible confirmation | all |

| ``StatusBanner`` | Persistent global banner (offline, degraded, expired) | F-5, cross-cutting |

| ``UndoToast`` | Time-boxed reversal for reversible actions | F-1, F-2, F-5 |

| ``InlineExplainer`` | "Why does this look like this" popover/sheet pattern | F-1, F-4, F-5 |

| ``EmptyStateBlock`` | Illustration + message + CTA, reused by **every** list/surface | all |

3. Feature components

3.1 F-1 — *Intelligent Task Prioritization*

| Component | Purpose |

---|---

| ``TodayQueue`` | The ranked vertical spine of active tasks (FR-2) — the single dominant element per the design direction |

| ``TaskRow`` | One task: rank position, title, due, tags, effort, age, dependency marker, pin/override marker |

| ``RankBadge`` | Ordinal rank indicator rendered on ``TaskRow`` |

| ``PriorityScoreIndicator`` | Subtle numeric / strength cue for the FR-1 score |

| ``WhyThisRankExplainer`` | Plain-language explanation of the dominant factors (FR-5), built on ``InlineExplainer`` |

| ``FactorWeightEditor`` | Per-factor sliders (FR-4) writing the prioritization profile |

| ``PrioritizationProfileSettings`` | Persisted profile (weights, defaults) via ``SettingsSection`` |

| ``OverrideControl`` | Pin-to-top / force-rank affordance (FR-6) |

| ``OverrideExpiryPicker`` | Indefinite vs. time-boxed / until-completion choice (FR-7) |

| ``OverrideMarker`` | Visible pinned/overridden state + remove action (FR-8) |

| ``UrgencyEscalationBadge`` | Deadline-urgency signal (FR-9) |

| ``DependencyIndicator`` | Dependency state surfaced on ``TaskRow`` |

| ``RecomputeIndicator`` | Non-jarring live re-rank animation when scoring recomputes |

| ``TaskDetailPanel`` | Detail view hosting explainer, override controls, dependencies |

3.2 F-2 — *Quick Add*

| Component | Purpose |

---|---

| ``QuickAddInput`` | The single mandatory free-text line (FR-26) |

| ``NaturalLanguageHintLayer`` | Underline/highlight of recognised date-time tokens (FR-27) |

| ``PriorityMarkerHint`` | Inline recognition of ``!!`` / ``!`` / "high" / "urgent" / "p1" (FR-28) |

| ``TagContextHint`` | Recognition of ``#tag`` and ``@context`` tokens (FR-29) |

| ``ParsedFieldPreview`` | Editable chips previewing interpreted fields **before** confirm (FR-30) |

| ``ParsedFieldChip`` | A single editable interpreted field (title / due / priority / tags) |

| ``TagSuggestionPopover`` | Completion for tag and context tokens |

| `CaptureUndoToast` | Post-confirm reversal of the last capture (built on `UndoToast`) |

3.3 F-3 — Focus Mode

| Component | Purpose |

|---|---|

| `FocusShell` / `FocusOverlay` | Collapses the product to one task + one timer + one exit (the design-direction "momentum" surface) |

| `FocusTimer` | Large Mono countdown — the single dominant element (FR-52, FR-53) |

| `TimerControls` | Start / pause / resume / extend / end (FR-54) |

| `SessionLengthPicker` | Default (25 min) + custom length within the supported range (FR-53, V-1) |

| `SessionProgressRing` | Ambient progress, deliberately non-distracting |

| `SingleSessionConflictDialog` | Explicit decision when another session is already active (FR-55, BR-3) |

| `SessionEndCard` | Completion state + optional next action |

| `ActiveSessionIndicator` | Global affordance shown while a session runs, outside `FocusShell` |

| `BreakPrompt` | Post-session break / continue choice |

3.4 F-4 — Progress Analytics

| Component | Purpose |

|---|---|

| `ProgressDashboard` | The scrollable "Progress" surface (FR-76) |

| `PeriodSelector` | Consumes `TimeRangePicker`; drives every tile and chart |

| `CompletionMetricTile` | Completed / created / net change (FR-77) |

| `CompletionRateChart` | Time-series completion rate at the granularity implied by period length (FR-78) |

| `ThroughputChart` | Bar/line tasks-per-day/week/month (FR-79) |

| `StreakCard` | Current + longest completion streak (FR-80) |

| `ChartDataPopover` | Per-datapoint detail on hover/focus |

| `NoDataState` | Friendly empty analytics state with CTA |

| `InsufficientDataState` | Data present but not yet enough for a stat/series |

3.5 F-5 — Integration Hub

| Component | Purpose |

|---|---|

| `ConnectionCatalogue` | Browseable source types with plain-language read/write description (FR-101) |

| `ConnectorCard` | One connectable type inside the catalogue |

| `ConnectFlowLauncher` | Starts the authorization handshake (FR-102); **no provider credentials are ever typed into product fields** |

| `AuthCallbackState` | Success / failure landing surface after the callback |

| `ConnectionList` | All connections with label, type, account identity, state, last-sync (FR-103) |

| `ConnectionRow` | A single connection in the list |

| `ConnectionStateBadge` | `connected` / `degraded` / `expired` / `error` / `paused` / `disconnected` |

| `ConnectionActionsMenu` | Rename · pause · resume · reconnect · disconnect |

| `LastSyncCaption` | Relative last-sync time + resync affordance |

| `DegradedBanner` | `StatusBanner` variant for degraded / expired connections |

| `SyncLogDrawer` | Per-connection sync history |

| `WebhookEndpointConfigurator` | Generic webhook-bridge setup |

3.6 F-6 — Dark Mode

Component	Purpose
---	---
<code>`ThemePreferenceControl`</code>	Exactly one of <code>`light`</code> / <code>`dark`</code> / <code>`system`</code> (FR-126)
<code>`ThemeProvider`</code> (context)	Resolves and distributes the effective theme to every component
<code>`SystemThemeListener`</code>	Watches the OS/browser colour-scheme signal and re-resolves live, with no reload (FR-127)
<code>`ThemePersistenceAdapter`</code>	Persists + re-applies the preference across sessions, reloads, sign-in/out (FR-128)
<code>`ThemeableSurface`</code>	Every surface / token consumer; guarantees immediate re-theme of already-rendered UI (FR-129)
<code>`ThemePreviewSwatch`</code>	Visual sample of each preference option
<code>`InlineThemeToggle`</code>	Optional quick toggle in the header
---	---

4. Cross-feature composition rules

- **One dominant thing per surface.** ``TodayQueue`` (F-1) is the spine of the home surface; ``QuickAddInput`` (F-2) floats above it as the only overlay line; ``FocusShell`` (F-3) temporarily *replaces* the spine entirely.
- **Focus Mode suspends but never destroys context.** While ``FocusShell`` is active, F-1 / F-2 / F-4 surfaces are retained and restored on exit; ``ActiveSessionIndicator`` stays visible outside ``FocusShell``.
- **Theming wraps everything.** ``ThemeProvider`` (F-6) is the outermost provider; every component consumes theme tokens, so no feature ships a hard-coded colour.
- **Integration states are ambient, never modal.** F-5 ``degraded`` / ``expired`` states surface via ``StatusBanner`` and per-row ``ConnectionStateBadge`` inside F-1 / F-2 — never as blocking dialogs.
- **Capture is always reachable.** ``AppShell`` mounts ``CaptureBarHost`` on every primary screen (FR-26). Inside ``FocusShell`` the bar is hidden to honour "one dominant thing" and restored on exit.
- **State completeness is mandatory.** Each component declares default / hover / focus-visible / active / disabled / loading / empty / error plus its theme-resolved token set.

5. Component → feature traceability

Component group	Feature	Representative global ids
---	---	---
Primitives (theme tokens)	all / F-6	FR-126 ... FR-129
<code>`AppShell`</code> , <code>`CaptureBarHost`</code> , <code>`PrimaryNavigation`</code>	all / F-2	FR-26
F-1 group	F-1	FR-1 ... FR-9
F-2 group	F-2	FR-26 ... FR-30
F-3 group	F-3	FR-51 ... FR-56
F-4 group	F-4	FR-76 ... FR-80
F-5 group	F-5	FR-101 ... FR-103
F-6 group	F-6	FR-126 ... FR-129

Local component ids are scoped within their owning feature and numbered sequentially from 1 (BR-, EC-, AC-, API-, DM-, T-, OQ-, V-); they are not re-declared here.

Data Models

> Scope: logical, technology-neutral modelling. Storage engine, indexing strategy, physical schema, transport and secret custody are the Architect's decision. No framework, database, vendor or protocol is implied by any name below. Every model is owned by one or more features (`F-1` ... `F-6`) and exists to satisfy the already-defined FR/NFR/US/API ranges for those features. Model ids `DM-1` ... `DM-24` are defined here once and referenced elsewhere as plain text.

0. Modelling conventions

#	Convention	Rule
1	Identity	Every entity carries an immutable, opaque, globally unique `id`. Ids are never reused after deletion and are safe to mint on a client before server confirmation, so offline capture (F-2) never blocks or loses a task.
2	Ownership	Every user-owned entity carries `ownerId`. All reads and writes are scoped to the authenticated owner; no cross-owner reference is valid.
3	Time	All instants are stored in UTC at millisecond precision. User-facing calendar dates are always derived using the `timezoneId` recorded on `DM-22` at the moment of the event, and the timezone used is persisted alongside any rollout.
4	Soft delete	User-visible entities carry `deletedAt`. Deletion is reversible for a bounded grace period, then physically purged. Deletion never cascades silently across an integration boundary (see `DM-18`).
5	Concurrency	Every mutable entity carries a monotonic `version`. A write against a stale `version` is rejected rather than silently merged.
6	Derivation	Anything computable — priority score, queue rank, activity rollups, streaks, connection health — is a **derived entity** that can always be recomputed from source entities. Derived stores are caches, never sources of truth.
7	Enum discipline	Enumerations are closed sets. Unrecognized values arriving from an external system (F-5) map to an `unknown` member instead of being rejected, so one malformed payload cannot fail a whole sync.
8	Secret isolation	No credential, token, or signing secret is ever stored on a functional entity, emitted in a response, or written to an audit entry. Secrets are referenced only by an opaque handle (`DM-16`).
9	Provenance	Any value that can originate from a human, a parser, or an external system records its origin. This is what makes two-way sync conflicts and "why did this change?" answerable.

1. Entity map

DM id	Entity	Type	Serves	Primary related requirements
DM-1	Task	source of truth	F-1, F-2, F-3, F-4, F-5	FR-1, FR-2, FR-3, FR-26, FR-27, FR-28, FR-29, FR-51, FR-76, FR-77
DM-2	Tag	source of truth	F-1, F-2	FR-29, FR-2
DM-3	TaskTagLink	join	F-1, F-2	FR-29, FR-3
DM-4	Project	source of truth	F-1, F-2	FR-29, FR-2
DM-5	TaskDependency	source of truth	F-1	FR-3, FR-9
DM-6	PriorityProfile	source of truth	F-1	FR-1, FR-3, FR-4
DM-7	PriorityScore	derived	F-1	FR-1, FR-2, FR-5
DM-8	RankOverride	source of truth	F-1	FR-6, FR-7, FR-8
DM-9	CaptureAttempt	source of truth	F-2	FR-26, FR-27, FR-28, FR-29, FR-30
DM-10	FocusSession	source of truth	F-3 (feeds F-4)	FR-51, FR-52, FR-53, FR-54, FR-55, FR-56

DM-11	FocusSessionEvent	append-only	F-3	FR-54, FR-55, FR-56
DM-12	ProgressPreference	source of truth	F-4	FR-76, FR-78, FR-80
DM-13	DailyActivityRollup	derived	F-4	FR-77, FR-78, FR-79
DM-14	StreakRecord	derived	F-4	FR-80
DM-15	Connection	source of truth	F-5	FR-101, FR-102, FR-103
DM-16	ConnectionSecretRef	source of truth (handle only)	F-5	FR-102, FR-103
DM-17	SyncRun	append-only record	F-5	FR-101, FR-103
DM-18	ExternalItemLink	source of truth	F-5	FR-101, FR-103, and F-1/F-4 consumers of synced tasks
DM-19	IntegrationAction	source of truth (queue record)	F-5	FR-103 and every write-back the catalogue advertises as writable
DM-20	WebhookDelivery	append-only record	F-5	FR-101, FR-103 (generic webhook bridge)
DM-21	ThemePreference	source of truth	F-6	FR-126, FR-127, FR-128, FR-129
DM-22	UserProfile	source of truth	all	FR-27, FR-28, FR-52, FR-76, FR-126
DM-23	Device	source of truth	F-2, F-3, F-6	FR-128, FR-55
DM-24	AuditEvent	append-only	all	FR-5, FR-6, FR-8, FR-103, FR-128

2. Core entities

DM-1 — Task

The single source of truth for a unit of work. Every other feature reads or writes it; no feature keeps a private copy of a task.

Field	Type (logical)	Req	Rules / notes
`id`	opaque id	yes	Immutable, never reused.
`ownerId`	ref → `DM-22`	yes	Immutable after creation.
`title`	string, 1–500 after trim	yes	Non-empty after trimming. Long input is truncated for display, never silently lost.
`notes`	text	no	Free-form; not parsed.
`status`	enum `active` \| `completed` \| `canceled` \| `archived`	yes	Default `active`. See transition invariant below.
`createdAt`	timestamp (UTC)	yes	Server-assigned; also the anchor for the "task age" factor (FR-3).
`updatedAt`	timestamp (UTC)	yes	Any field change updates it.
`dueAt`	timestamp (UTC) \| null	no	Null means "no deadline". Consumed by the due-date factor and by urgency escalation (FR-3, FR-9).
`dueHasTime`	boolean	yes (default false)	Distinguishes "due 5 March" from "due 5 March 14:00" so a date-only deadline is never rendered as a late-evening time. Only meaningful when `dueAt` is set.
`importance`	integer 0–100 \| null	no	Explicit user importance. Null means "not stated" and must be treated as neutral by scoring, not as zero (FR-3, FR-28).
`importanceOrigin`	enum `user` \| parsed \| integration \| default	yes	Provenance of `importance` (convention 9).
`effortEstimateMinutes`	integer ≥ 1 \| null	no	Estimated effort. Null is neutral, not zero (FR-3).
`projectId`	ref → `DM-4` \| null	no	Null resolves to the owner's default project.
`dependencyState`	enum `none` \| ready \| blocked	derived	Computed from `DM-5`. Never written directly.
`origin`	enum `manual` \| quick_add \| integration \| recurring	yes	Provenance. `quick\_add` links to `DM-9`; `integration` implies at least one `DM-18` link.

`captureAttemptId`	ref → `DM-9` \ null	no	Present when the task was created through the capture surface.
`completionOrigin`	enum `manual` \ `focus\_session` \ `integration` \ null	no	Set when `status` becomes `completed`; distinguishes a user tick from an externally observed completion (relevant to F-4 metrics).
`completedAt`	timestamp (UTC) \ null	no	Set iff `status = completed`.
`canceledAt` / `archivedAt`	timestamp (UTC) \ null	no	Set iff the matching status is active.
`deletedAt`	timestamp (UTC) \ null	no	Soft delete.
`version`	integer, monotonic	yes	Optimistic concurrency (convention 5).

****Invariants****

1. `completedAt` is non-null ****iff**** `status = completed`; `canceledAt` is non-null ****iff**** `status = canceled`; `archivedAt` is non-null ****iff**** `status = archived`.
2. Allowed status transitions: `active` → `completed`, `active` → `canceled`, `active` → `archived`, `completed` → `active` (reopen), `canceled` → `active` (restore), `archived` → `active` (restore). `completed` → `canceled` is not allowed — reopen first.
3. `dueHasTime` may only be true when `dueAt` is set.
4. A task with `deletedAt` set is excluded from the Today queue, from all metrics (F-4), and from any outgoing sync write (F-5), but its links (`DM-18`) are retained in an orphaned state so a re-import does not resurrect a deleted task silently.
5. `title` is never empty: a capture that parses to an empty title (see `DM-9`) must fail visibly before a task is created.

DM-2 — Tag

Field	Type	Req	Rules
`id`	opaque id	yes	—
`ownerId`	ref → `DM-22`	yes	—
`label`	string 1–50	yes	Display form, preserves user casing.
`normalizedLabel`	string	yes	Case-folded and whitespace-collapsed; uniqueness is enforced on this field.
`colorToken`	enum token name	no	A ****theme token****, not a raw colour value, so tags re-theme correctly under F-6 (FR-129).
`createdAt` / `updatedAt`	timestamp	yes	—
`archivedAt`	timestamp \ null	no	Archived tags stay attached to history but are not offered for new tasks.
`usageCount`	integer	derived	Recomputed from `DM-3`.
 Uniqueness: one tag per `(ownerId, normalizedLabel)`.

DM-3 — TaskTagLink

Field	Type	Req	Rules
`id`	opaque id	yes	—
`ownerId`	ref → `DM-22`	yes	Must equal the owner of both ends.
`taskId`	ref → `DM-1`	yes	Deleting the task removes its links.
`tagId`	ref → `DM-2`	yes	Deleting the tag removes its links.
`createdAt`	timestamp	yes	—
`origin`	enum `user` \ `parsed` \ `integration`	yes	Provenance (convention 9).

Uniqueness: one row per `(taskId, tagId)`.

DM-4 — Project

Field	Type	Req	Rules
---	---	---	---
`id`	opaque id	yes	—
`ownerId`	ref → `DM-22`	yes	—
`name`	string 1–100	yes	—
`normalizedName`	string	yes	Uniqueness enforced here.
`isDefault`	boolean	yes	Exactly one default project per owner ("Inbox"); it cannot be deleted or archived.
`sortKey`	string	no	User-controlled ordering.
`createdAt` / `updatedAt`	timestamp	yes	—
`archivedAt`	timestamp \ null	no	Archiving re-homes its tasks to the default project; it never deletes them.

DM-5 — TaskDependency

Field	Type	Req	Rules
---	---	---	---
`id`	opaque id	yes	—
`ownerId`	ref → `DM-22`	yes	Both tasks must share this owner.
`blockedTaskId`	ref → `DM-1`	yes	The task that cannot proceed.
`blockingTaskId`	ref → `DM-1`	yes	The task that must finish first.
`createdAt`	timestamp	yes	—
`state`	enum `active` \ `satisfied` \ `invalidated`	derived	`satisfied` when the blocking task is `completed`; `invalidated` when either side was deleted or reopened after satisfaction.

****Invariants****

1. No self-reference: `blockedTaskId` ≠ `blockingTaskId`.
2. The dependency graph is ****acyclic****. Adding an edge that would create a cycle is rejected with an explanatory error, never silently dropped.
3. Duplicate edges are rejected: uniqueness on `(blockedTaskId, blockingTaskId)`.
4. Deleting the blocking task does not delete the edge; it flips the dependent task to `dependencyState` = `ready` and the edge to `invalidated`, so the ordering change is explainable (FR-5).
5. This entity feeds the dependency factor of `DM-7` directly; it is never duplicated into the task.

3. F-1 — Prioritization entities

DM-6 — PriorityProfile

Field	Type	Req	Rules
---	---	---	---
`id`	opaque id	yes	—
`ownerId`	ref → `DM-22`	yes	One <i>active</i> profile per owner in the initial design (see OQ-1).
`revision`	integer, monotonic	yes	Increments on every weight change.
`weights`	list of `{ factor, weight }`	yes	`factor` ∈ `due_date`, `importance`, `effort`, `age`, `dependency` (the FR-3 minimum set). `weight` is a decimal 0–100. Weights are **relative** : scoring

normalizes them, so any set of weights is valid. |
 | `neutralPolicy` | map `factor` → neutral value` | yes | Explicit declaration of what "no due date", "no effort estimate" and "no dependencies" contribute. Prevents a missing input from being scored as zero. |
 | `isActive` | boolean | yes | Exactly one active profile per owner. |
 | `createdAt` / `updatedAt` | timestamp | yes | — |
 | `source` | enum `default` \| `user\_tuned` | yes | Whether the user has ever edited weights (FR-4). |

****Invariants****

1. Every factor in the FR-3 minimum set is present in `weights`; a profile missing a factor is invalid.
2. `revision` never decreases; a rollback writes a new `revision` that reinstates old values, so history stays linear.
3. Profiles are versioned, not overwritten. `DM-7` references the `revision` it used, which is what makes an old ranking reproducible and an explanation truthful (FR-5).

DM-7 — PriorityScore (derived)

Field	Type	Req	Rules
`id`	opaque id	yes	—
`taskId`	ref → `DM-1`	yes	—
`ownerId`	ref → `DM-22`	yes	—
`profileId` / `profileRevision`	ref + integer	yes	Exact inputs used.
`score`	decimal, normalized 0–100	yes	Always defined: neutral policy (`DM-6`) guarantees every factor has a value.
`factorContributions`	list of `{ factor, rawInput, normalized 0–1, weight, contribution }`	yes	The substrate of the FR-5 explanation and of the weight editor's preview (FR-4).
`band`	enum `overdue` \| `due\_today` \| `upcoming` \| `later` \| `undated` \| `blocked`	derived	Human-legible grouping used by the queue UI.
`rank`	integer ≥ 1	derived	Position in the queue snapshot this score belongs to.
`queueScope`	enum `today` \| `project` \| `search`	yes	Rank is only meaningful inside its scope.
`explanationText`	text	derived	Plain-language rendering of the dominant factors (FR-5); regenerated from `factorContributions`, never hand-written.
`computedAt` / `expiresAt`	timestamp	yes	`expiresAt` drives the freshness expectation in NFR-5.

****Invariants****

1. Derived and disposable: deleting all scores must leave the queue fully reconstructable.
2. Scores are monotonic in their inputs — increasing importance or nearing a deadline can never *lower* a task's score under a fixed profile revision.
3. A score is never used to compute another score; deep chains are forbidden so recomputation stays bounded.
4. `rank` is unique within `(queueScope, snapshot)` — two tasks never share a rank.

DM-8 — RankOverride

Field	Type	Req	Rules
`id`	opaque id	yes	—
`ownerId` / `taskId`	refs	yes	—
`type`	enum `pin\_top` \| `fixed\_rank` \| `boost`	yes	`fixed\_rank` requires `fixedRank`; the others must not set it.

`fixedRank`	integer ≥ 1 \ null	no	Required iff `type = fixed\_rank`; may exceed the current queue length, in which case the task lands at the tail (recorded as an applied-with-adjustment event).
`createdAt`	timestamp	yes	—
`createdByDeviceId`	ref $\rightarrow$ `DM-23`	no	—
`expiresAt`	timestamp \ null	no	Time-boxed override (FR-7).
`expiresOnCompletion`	boolean	yes (default true)	Override auto-releases when the task completes (FR-7).
`reason`	string ≤ 200	no	Optional note shown in the explanation.
`isActive`	boolean	yes	—
`releasedAt`	timestamp \ null	no	Set when released manually, by expiry, or by completion.

****Invariants****

1. At most **one** active override per task; creating a second requires the first to be released explicitly (the UI must ask, not silently replace).
2. An override never modifies `DM-7.score` — it changes final ordering only. Removing an override must return the task to exactly the position its score implies, which is why scores are retained while an override is active.
3. Auto-release on expiry or completion is recorded in `DM-24` and surfaces as a transient, dismissible notice so the user sees the queue re-order (FR-8).
4. Overrides never expire while `isActive = true` and `expiresAt` is null (indefinite is a first-class choice).

4. F-2 — Quick Add entities

DM-9 — CaptureAttempt

Every capture creates a record, even one that is discarded, so parsing can be explained and improved and so an interrupted capture is recoverable.

Field	Type	Req	Rules
`id`	opaque id	yes	Client-mintable so an offline capture is never lost (convention 1).
`ownerId`	ref $\rightarrow$ `DM-22`	yes	—
`deviceId`	ref $\rightarrow$ `DM-23`	no	—
`rawText`	string, 1–1000	yes	The user's untouched input.
`capturedAt`	timestamp, client + server	yes	Both retained; the server timestamp is authoritative for ordering, the client timestamp preserves intent when the device was offline.
`timezoneAtCapture`	IANA string	yes	Relative expressions such as "tomorrow 5pm" are resolved with this zone, not the current one (see DM-22 invariant 2).
`parseResult`	structured	derived	`title`, `dueAt`, `dueHasTime`, `importance`, `importanceOrigin = parsed`, `tagTokens[]`, `projectToken`, `unrecognizedTokens[]`.
`parseConfidence`	map `field  $\rightarrow$  decimal 0–1`	derived	Drives the preview and the "check this" affordance (FR-30).
`parserVersion`	string	yes	Provenance of the interpretation; a re-parse can be offered when the parser improves.
`status`	enum `parsed \ applied \ discarded \ superseded`	yes	`applied` implies exactly one `taskId`.
`taskId`	ref $\rightarrow$ `DM-1` \ null	no	Set iff `status = applied`.
`discardReason`	enum `user\_cancelled \ empty\_title \ duplicate \ error` \ null	no	—

****Invariants****

1. Nothing that fails to parse is silently dropped: an unrecognized token stays in `title`, and its presence is visible in the preview (FR-29, FR-30).
2. A capture produces at most one task (FR-26 defines one line → one task); a second confirm while the first is in flight is deduplicated by `id`, not by creating two tasks.
3. `status = applied` requires a non-empty resolved `title`. A capture that resolves to an empty title is blocked pre-creation with an inline error, and the raw text remains editable.
4. Applying a capture is a single atomic step: task + tags + project link are created together or not at all.
5. `rawText` retention is bounded by the retention policy in §10 (see OQ-2).

5. F-3 — Focus Mode entities

DM-10 — FocusSession

Field	Type	Req	Rules
---	---	---	---
`id`	opaque id	yes	—
`ownerId`	ref → `DM-22`	yes	—
`taskId`	ref → `DM-1`	yes	A session is always bound to one existing task (FR-51); unbound "just a timer" sessions are not modelled (see OQ-3).
`state`	enum `running` \ `paused` \ `completed` \ `abandoned` \ `expired`	yes	Single active session per owner (FR-55).
`plannedDurationMinutes`	integer 1–600	yes	Initialized from
`DM-22.defaultFocusDurationMinutes` (built-in default 25), overridable per session (FR-52, FR-53).			
`extensionMinutesTotal`	integer ≥ 0	yes	Sum of all extensions in this session (FR-54).
`startedAt`	timestamp	yes	Server clock is authoritative.
`lastResumedAt`	timestamp \ null	yes	Non-null iff `state = running`.
`endedAt`	timestamp \ null	no	Non-null iff `state ∈ {completed, abandoned, expired}`.
`effectiveDurationSeconds`	integer ≥ 0	derived	Active time only; pauses excluded. Never exceeds wall-clock duration.
`accumulatedPauseSeconds`	integer ≥ 0	derived	—
`pauseCount` / `interruptionCount` integer ≥ 0 derived Interruption = pause followed by an out-of-app return, per FR-56's interruption reporting.			
`outcome`	enum `target_reached` \ `stopped_early` \ `abandoned` \ `auto_expired`	no	Non-null iff the session has ended.
`taskCompletedInSession`	boolean	yes (default false)	Set only by an explicit user action; ending a session never auto-completes the task.
`deviceId`	ref → `DM-23`	no	—
`version`	integer	yes	Optimistic concurrency (convention 5).

Invariants

1. ****At most one session per owner is in `running` or `paused` at any instant.**** Enforced as a uniqueness constraint over the active states, not by UI alone.
2. `plannedDurationMinutes` + `extensionMinutesTotal` must stay within the supported range; an extension that would exceed it is rejected with a clear message rather than silently clamped.
3. `endedAt` ≥ `startedAt`; `effectiveDurationSeconds` ≤ (endedAt – startedAt).
4. Deleting the bound task while a session is active does ****not**** silently kill the session — the session is preserved as an orphaned record and the user is asked whether to stop or re-bind (see §8, EC for F-3).
5. A session for a `completed` task may only be started as a fresh session; sessions are never retro-attached to a closed range.

6. Client/server clock disagreement is resolved in favour of the server, and the correction is recorded (see `DM-11`).

DM-11 — FocusSessionEvent

Append-only. The session's timeline must be reconstructable and reconcilable across devices.

Field	Type	Req	Rules
---	---	---	---
`id`	opaque id	yes	—
`sessionId`	ref → `DM-10`	yes	—
`type`	enum `started` \ `paused` \ `resumed` \ `extended` \ `ended` \ `abandoned` \ `auto_expired` \ `clock_corrected` \ `reconciled`	yes	—
`at`	timestamp, server	yes	—
`clientAt`	timestamp	no	Retained when it differed from `at` by more than the tolerance, so a discrepancy is explainable.
`deltaMinutes`	integer	no	Required for `extended`.
`actorDeviceId`	ref → `DM-23`	no	—
`note`	string	no	Free-form, non-secret.

*****Invariants*****

1. Records are immutable; a correction is a new record, never an edit.
2. The ordered event list must be sufficient to recompute `effectiveDurationSeconds`, `pauseCount` and `outcome` from scratch.
3. Events for a session must form a valid state machine; an event that would produce an illegal transition is rejected. Duplicate deliveries of the same event (retry, dual device) are idempotent by `id`.

6. F-4 — Progress Analytics entities

DM-12 — ProgressPreference

Field	Type	Req	Rules
---	---	---	---
`id`	opaque id	yes	—
`ownerId`	ref → `DM-22`	yes	Unique per owner.
`defaultPeriod`	enum `7d` \ `30d` \ `90d` \ `custom`	yes	Default `7d`.
`customRangeStart` / `customRangeEnd`	date (owner-local)	no	Required iff `defaultPeriod` = `custom`; start ≤ end.
`weekStartDay`	enum `monday` \ `sunday` \ `saturday`	yes	Default `locale`, resolved via `DM-22`.
`streakDefinition`	enum `days_with_completion` \ `days_with_focus_session`	yes	Default `days_with_completion` (FR-80).
`metricVisibility`	map `metric` → boolean	yes	Which dashboard blocks are shown.
`lastViewedAt`	timestamp	no	—
`updatedAt`	timestamp	yes	—

*****Invariants*****

1. Changing `streakDefinition` takes effect from the change date forward; historical `DM-14` rows keep their original definition and are not rewritten. Both facts are shown in the UI so the change is never mysterious.
2. Changing `weekStartDay` recomputes weekly series but never alters daily rollups.

DM-13 — DailyActivityRollup (derived)

One row per owner per ****owner-local calendar date****.

Field	Type	Req	Rules
id	opaque id	yes	—
ownerId	ref → DM-22	yes	—
dateLocal	date (owner-local)	yes	Uniqueness on (ownerId, dateLocal).
timezoneUsed	IANA string	yes	The zone in force when the date was bucketed (convention 3).
tasksCreated	integer ≥ 0	derived	From DM-1.createdAt.
tasksCompleted	integer ≥ 0	derived	From DM-1.completedAt and completionOrigin.
tasksCompletedViaFocus	integer ≥ 0	derived	Subset of the above; supports FR-77/FR-79 breakdowns.
netChange	integer	derived	tasksCreated – tasksCompleted (FR-77).
completionRate	decimal 0–100 \ null	derived	tasksCompleted ÷ tasksCreated; null when tasksCreated = 0 — never rendered as 0 %, which would be a lie.
focusMinutes	integer ≥ 0	derived	Sum of DM-10.effectiveDurationSeconds ÷ 60 for sessions ending that date.
focusSessionCount	integer ≥ 0	derived	—
computedAt	timestamp	yes	—
sourceWatermark	timestamp	yes	Latest source updatedAt folded in; used to decide whether a recompute is needed.

****Invariants****

1. Purely derived and fully recomputable; deleting the table must not change any number the user sees after recompute.
2. A late-arriving change (an external system reports a completion dated yesterday) triggers recompute of the affected dates only, and the dashboard shows a non-blocking "updated" indication rather than an unexplained jump.
3. Rollups never include deleted or canceled tasks as completions, but a task completed and later reopened removes one from the completion count of its original date.
4. There is no future-dated rollup; a zero-activity day exists implicitly and may be materialized on read.

DM-14 — StreakRecord (derived)

Field	Type	Req	Rules
id	opaque id	yes	—
ownerId	ref → DM-22	yes	—
kind	enum current \ longest	yes	FR-80.
definition	enum mirroring DM-12.streakDefinition	yes	Historical rows keep the definition they were computed with.
lengthDays	integer ≥ 0	derived	—
startDateLocal / endDateLocal	date	derived	Inclusive range.
isProvisional	boolean	derived	True for a current streak that includes today, since today can still break it.
computedAt	timestamp	yes	—

****Invariants****

1. longest.lengthDays never decreases for a given definition.

2. A streak of length 0 is representable and is displayed as "no streak yet" with an encouraging empty state, not as a bare zero.
3. At most one ``current`` and one ``longest`` row per ``(ownerId, definition)``.
4. A streak is continuous by local calendar date: a missed local day breaks it, and the timezone used for the boundary is the one recorded in ``DM-13``.

7. F-5 — Integration Hub entities

DM-15 — Connection

Field	Type	Req	Rules
---	---	---	---
<code>`id`</code>	opaque id	yes	—
<code>`ownerId`</code>	ref → <code>`DM-22`</code>	yes	—
<code>`providerType`</code>	enum <code>`calendar`</code> \ <code>`email`</code> \ <code>`issue_tracker`</code> \ <code>`code_host`</code> \ <code>`messaging`</code> \ <code>`file_store`</code> \ <code>`generic_webhook`</code>	yes	Mirrors the catalogue (FR-101).
<code>`providerId`</code>	string	yes	Which concrete provider within the type; opaque to the functional layer.
<code>`label`</code>	string 1–80	yes	User-editable, defaults to the account display name.
<code>`accountIdentityDisplay`</code>	string	yes	Human-readable identity (e.g. an address or handle). Never a secret.
<code>`state`</code>	enum <code>`connected`</code> \ <code>`degraded`</code> \ <code>`expired`</code> \ <code>`error`</code> \ <code>`paused`</code> \ <code>`disconnected`</code>	yes	FR-103.
<code>`scopesGranted`</code>	list of scope strings	yes	What the user actually authorized; drives capability claims.
<code>`capabilities`</code>	map <code>`dataType`</code> → { read, write }	yes	What the catalogue promised **and** what the grant permits; if the two differ, the narrower one wins and the difference is surfaced.
<code>`syncPolicy`</code>	enum <code>`auto`</code> \ <code>`manual`</code> + <code>`frequencyMinutes`</code>	yes	Default <code>`auto`</code> ; user-changeable.
<code>`connectedAt`</code>	timestamp	yes	—
<code>`lastSyncAt`</code>	timestamp \ null	yes	Most recent attempt.
<code>`lastSuccessfulSyncAt`</code>	timestamp \ null	yes	Most recent success.
<code>`consecutiveFailureCount`</code>	integer ≥ 0	yes	Drives <code>`degraded`</code> → <code>`error`</code> promotion.
<code>`lastErrorCode`</code>	enum	no	Machine-readable, so the UI can offer a specific recovery action.
<code>`lastErrorMessage`</code>	string	no	User-safe wording; never raw provider payloads, never credentials.
<code>`pausedAt`</code> / <code>`disconnectedAt`</code>	timestamp \ null	no	Set iff the matching state is active.
<code>`createdAt`</code> / <code>`updatedAt`</code>	timestamp	yes	—
<code>`version`</code>	integer	yes	—

****Allowed state transitions (nothing else is valid):****

From	To	Trigger
---	---	---
—	<code>`connected`</code>	Successful authorization callback (FR-102). A connection only appears after a successful callback; a partially completed handshake leaves no connection row.
<code>`connected`</code>	<code>`degraded`</code>	First transient sync failure.
<code>`degraded`</code>	<code>`connected`</code>	A sync succeeds.
<code>`degraded`</code>	<code>`error`</code>	Failures reach the escalation threshold.
<code>`expired`</code>	<code>`connected`</code>	Successful reauthorization.
<code>`error`</code> / <code>`expired`</code> / <code>`degraded`</code>	<code>`paused`</code>	Explicit user pause.
<code>`paused`</code>	<code>`connected`</code>	Explicit user resume, followed by a verification sync.
any <code>`connected-ish`</code> state	<code>`disconnected`</code>	Explicit user disconnection.
<code>`disconnected`</code>	<code>`connected`</code>	A fresh authorization handshake.

****Invariants****

1. ``paused`` and ``disconnected`` both stop scheduled syncs, but ``paused`` preserves the authorization and queued actions, while ``disconnected`` revokes the secret reference and terminally fails queued actions with a stated reason.
2. Exactly one connection may exist per `(ownerId, providerType, accountIdentityDisplay)``; attempting a duplicate is reported as already connected with a link to the existing one, never as a silent second row.
3. No credential material lives on this entity (convention 8).

DM-16 — ConnectionSecretRef

Field	Type	Req	Rules
<code>`id`</code>	opaque id	yes	—
<code>`connectionId`</code>	ref → <code>`DM-15`</code>	yes	—
<code>`purpose`</code>	enum <code>`access_token`</code> \ <code>`refresh_token`</code> \ <code>`webhook_signing_secret`</code> \ <code>`client_secret`</code>	yes	—
<code>`handle`</code>	opaque string	yes	Reference into whatever custody mechanism the Architect selects. The handle is meaningless outside the app and is never returned to a client.
<code>`issuedAt`</code>	timestamp	yes	—
<code>`expiresAt`</code>	timestamp \ null	no	—
<code>`lastRefreshedAt`</code>	timestamp \ null	no	—
<code>`rotationGeneration`</code>	integer ≥ 1	yes	Increments on rotation so an in-flight action can be attributed to the generation that authorized it.
<code>`status`</code>	enum <code>`active`</code> \ <code>`expired`</code> \ <code>`revoked`</code>	yes	—

Invariants

1. No entity, response, log line or audit entry anywhere in the system contains secret material; only handles do.
2. Revocation is immediate and idempotent, and is recorded in ``DM-24``. Marking a ``DM-15`` as ``disconnected`` revokes all of its handles.
3. Exactly one ``active`` handle per `(connectionId, purpose)``.

DM-17 — SyncRun

Field	Type	Req	Rules
<code>`id`</code>	opaque id	yes	—
<code>`connectionId`</code>	ref → <code>`DM-15`</code>	yes	—
<code>`direction`</code>	enum <code>`inbound`</code> \ <code>`outbound`</code> \ <code>`bidirectional`</code>	yes	—
<code>`trigger`</code>	enum <code>`scheduled`</code> \ <code>`manual`</code> \ <code>`webhook`</code> \ <code>`retry`</code>	yes	—
<code>`state`</code>	enum <code>`running`</code> \ <code>`succeeded`</code> \ <code>`partially_succeeded`</code> \ <code>`failed`</code> \ <code>`rate_limited`</code> \ <code>`canceled`</code>	yes	—
<code>`startedAt`</code> / <code>`finishedAt`</code>	timestamp	yes / nullable	<code>`finishedAt`</code> null iff <code>`state` = <code>`running`</code></code> .
<code>`counters`</code>	map	derived	<code>`itemsRead`</code> , <code>`created`</code> , <code>`updated`</code> , <code>`skipped`</code> , <code>`conflicted`</code> , <code>`failed`</code> .
<code>`errorSummary`</code>	string	no	User-safe. Never raw provider payloads.
<code>`cursorBefore`</code> / <code>`cursorAfter`</code>	opaque string \ null	no	Enables incremental sync without re-reading everything. Only advanced on success or partial success.
<code>`retryOf`</code>	ref → <code>`DM-17`</code> \ null	no	—
<code>`nextRetryAt`</code>	timestamp \ null	no	Exponential backoff; cleared on success.

Invariants

1. At most one ``running`` run per ``(connectionId, direction)``; a second trigger while one is running is coalesced and reported as "already syncing", not queued blindly.
2. Failure isolation: an individual item that cannot be applied increments ``failed`` and the run continues. A healthy run reports ``partially_succeeded``, never a binary success/failure that hides which items were affected.
3. ``cursorAfter`` is never advanced past items that failed, so the next run retries them.
4. Every terminal run writes exactly one user-visible health update on ``DM-15``.

DM-18 — ExternalItemLink

The join between a local task and the external item it mirrors, plus the field-ownership map that makes two-way sync predictable.

Field	Type	Req	Rules
<code>`id`</code>	opaque id	yes	—
<code>`connectionId`</code>	string	yes	Uniqueness on <code>`(connectionId, externalId, externalType)`</code> .
<code>`externalId`</code>	enum mapped from the provider's item kinds	yes	Unknown kinds map to <code>`unknown`</code> (convention 7).
<code>`taskId`</code>	ref → <code>`DM-1`</code>	no	Null when the item is <code>`ignored`</code> or was <code>`orphaned`</code> by a local deletion.
<code>`linkState`</code>	enum <code>`linked`</code> <code>`ignored`</code> <code>`orphaned`</code> <code>`unlinked`</code>	yes	—
<code>`externalVersion`</code>	string	no	ETag / revision / <code>`updatedAt`</code> from the provider, used for conflict detection.
<code>`fieldOwnership`</code>	map <code>`field`</code> → local <code>`external`</code> <code>`newest_wins`</code>	yes	Explicit per field: title, dueAt, importance, status, project, tags. This is the whole conflict rule; it is never implicit.
<code>`firstSeenAt`</code> / <code>`lastSeenAt`</code>	timestamp	yes	—
<code>`lastSyncedAt`</code>	timestamp	no	—
<code>`lastConflict`</code>	structured	no	{ field, localValue, externalValue, resolvedBy, resolvedAt } — retained so a surprising value is explainable.

Invariants

1. A task may be linked to several external items across connections; a local field is only overwritten by an external value if that field's ownership rule permits it.
2. ``orphaned`` is never a silent delete: a locally deleted task leaves its link, and a later re-import of the same external item does not resurrect the task without explicit user action.
3. Because external identities are the join key, storing them is mandatory before any external write is attempted — that is what makes write-back idempotent.

DM-19 — IntegrationAction

Field	Type	Req	Rules
<code>`id`</code>	opaque id	yes	—
<code>`connectionId`</code>	ref → <code>`DM-15`</code>	yes	—
<code>`taskId`</code>	ref → <code>`DM-1`</code>	no	Null for non-task actions (e.g. posting a message).
<code>`externalItemLinkId`</code>	ref → <code>`DM-18`</code>	no	Required for <code>`update`</code> / <code>`complete`</code> / <code>`delete`</code> ; null for <code>`create`</code> until the external id is known.
<code>`actionType`</code>	enum <code>`create`</code> <code>`update`</code> <code>`complete`</code> <code>`delete`</code> <code>`comment`</code> <code>`notify`</code>	yes	Must be permitted by <code>`DM-15.capabilities.write`</code> .

<code>`payloadSummary`</code>	structured, non-secret	yes	Enough to explain the action in an activity list; never a full payload dump, never a credential.
<code>`requestedAt`</code>	timestamp	yes	—
<code>`requestedBy`</code>	enum <code>`user`</code> automation integration	+ actor ref	yes —
<code>`state`</code>	enum <code>`queued`</code> <code>`in_flight`</code> <code>`succeeded`</code> <code>`failed`</code> <code>`retrying`</code> <code>`dead_letter`</code> <code>`canceled`</code>		yes —
<code>`attemptCount`</code>	integer ≥ 0	yes	—
<code>`lastAttemptAt`</code>	timestamp	no	—
<code>`lastErrorCode`</code> / <code>`lastErrorMessage`</code>	enum / string	no	User-safe.
<code>`idempotencyKey`</code>	string	yes	Unique per <code>`(connectionId, idempotencyKey)`</code> .
<code>`resultExternalVersion`</code>	string	no	Written back to <code>`DM-18.externalVersion`</code> on success.
<code>`nextRetryAt`</code>	timestamp	no	—

****Invariants****

1. Retrying an action must never duplicate the external write; ``idempotencyKey`` is the guarantee (convention on at-least-once delivery).
2. Actions queued while a connection is ``paused`` remain ``queued`` and are not silently dropped; they resume when it is resumed, and are terminally failed with a stated reason when it is ``disconnected``.
3. An action the connection is not authorized to perform is rejected at enqueue time with a clear capability message, not failed later after the user believes it worked.
4. ``dead_letter`` is terminal and must be surfaced to the user with a retry option — never swallowed.
5. Any action that changes local state optimistically carries the originating task ``version``, so a stale action cannot clobber a newer local edit.

DM-20 — WebhookDelivery

Field	Type	Req	Rules
			--- --- --- ---
<code>`id`</code>	opaque id	yes	—
<code>`connectionId`</code>	ref	→ <code>`DM-15`</code>	yes —
<code>`eventId`</code>	string	yes	Uniqueness per connection; the dedupe key for at-least-once delivery.
<code>`eventType`</code>	string	yes	—
<code>`signatureKeyRef`</code>	ref	→ <code>`DM-16`</code>	no Reference only; the signing secret value is never stored here.
<code>`attemptedAt`</code>	timestamp	yes	Append-only per attempt.
<code>`attemptCount`</code>	integer ≥ 1	yes	—
<code>`responseStatusClass`</code>	enum <code>`2xx`</code> <code>`3xx`</code> <code>`4xx`</code> <code>`5xx`</code> <code>`timeout`</code> <code>`network_error`</code>		no —
<code>`state`</code>	enum <code>`pending`</code> <code>`delivered`</code> <code>`failed`</code> <code>`dead_letter`</code>		yes —
<code>`nextAttemptAt`</code>	timestamp	no	—
<code>`payloadFingerprint`</code>	string	yes	Proves what was sent without storing the whole payload.

****Invariants****

1. Exactly one terminal state per ``(eventId, connectionId)``; retries update the same logical delivery rather than creating duplicates for the receiver.
2. A ``4xx`` other than a rate-limit code is terminal (``dead_letter``) — retrying a rejected request forever is forbidden.
3. Delivery records are append-only and never contain secrets or full user content beyond the fingerprint.

8. F-6 — Dark Mode entity

DM-21 — ThemePreference

Field	Type	Req	Rules
---	---	---	---
`id`	opaque id	yes	—
`ownerId`	ref → `DM-22`	yes	—
`deviceId`	ref → `DM-23` \ null	no	Null row = the account-level default; per-device rows override it (FR-128).
`preference`	enum `light` \ `dark` \ `system`	yes	The user's choice , and the only value a user can set (FR-126). Exactly one of the three; no per-screen or per-widget value exists anywhere in the model.
`effectiveTheme`	enum `light` \ `dark`	derived	The concrete theme actually rendered.
`resolvedFrom`	enum `user_choice` \ `system_signal` \ `account_default` \ `built_in_default`	derived	Makes an unexpected theme explainable.
`lastSystemSignal`	enum `light` \ `dark` \ `unknown`	derived	Last observed OS/browser signal (FR-127).
`updatedAt`	timestamp	yes	—
`revision`	integer, monotonic	yes	Bumped on every change; lets a client detect a stale preference.

****Invariants****

- Exactly one preference row per `(ownerId, deviceId)` and exactly one per `ownerId` with `deviceId` = null.
- Exactly one preference row per `(ownerId, deviceId)` and exactly one per `ownerId` with `deviceId` = null.
- The user's **choice** is never overwritten by the system signal. Flipping the OS between light and dark while `preference` = `system` changes only `effectiveTheme` and `lastSystemSignal`; `preference` stays `system` (FR-127).
- An `unknown` system signal must still resolve to a concrete `effectiveTheme` through the fallback chain `account\_default` → `built\_in\_default`. An unresolvable theme is not a representable state, because an unthemed flash is a visible defect (FR-129).
- Preference application is local-first: a client may render from its own persisted value before any server round trip, then reconcile using `revision`. A stale-but-valid theme is always preferable to a flash of the wrong theme.
- On sign-in, an existing per-device row for that device takes precedence over the account default; the account default applies only where no per-device row exists. Sign-out deletes neither row (FR-128).
- There is no entity anywhere in the model that stores a per-screen, per-widget, or per-component theme value. Such a value cannot be expressed, so it cannot be introduced by accident.

9. Platform entities

DM-22 — UserProfile

The owner root. Every other user-owned entity resolves its scope through this row.

Field	Type	Req	Rules
---	---	---	---
`id`	opaque id	yes	Also the `ownerId` referenced by every other entity.
`displayName`	string 1–120	yes	—
`timezoneId`	IANA string	yes	The authority for every local-date derivation in the product (convention 3).

| `locale` | BCP-47 string | yes | Drives date, time, number and currency formatting (NFR-16 range consumers). |
 | `defaultFocusDurationMinutes` | integer 1–600 | yes | Built-in default 25. Used only to initialize a ****new**** session (FR-52). |
 | `focusDurationPresets` | list of integers 1–600, ≤ 6 entries | no | Quick choices offered before a session starts (FR-53). |
 | `defaultPriorityProfileId` | ref → `DM-6` | yes | Exactly one active profile (FR-4). |
 | `defaultProjectId` | ref → `DM-4` | yes | The "Inbox" project; must exist before any task is created (DM-1 invariant 5). |
 | `streakDefinitionDefault` | enum mirroring `DM-12.streakDefinition` | yes | Seeds a new `DM-12` row (FR-80). |
 | `createdAt` / `updatedAt` | timestamp | yes | — |
 | `deletedAt` | timestamp \| null | no | Account-level deletion; cascades per §11. |
 | `version` | integer | yes | — |

****Invariants****

1. Exactly one `DM-22` per owner, created atomically with the default project and the default priority profile — the system never has a state in which a user exists without both.
2. Changing `timezoneId` does ****not**** rewrite historical `DM-13` or `DM-14` rows. Those keep the `timezoneUsed` they were computed with, and the dashboard states the zone boundary it used so a changed number is explainable (convention 3).
3. Changing `defaultFocusDurationMinutes` never mutates a `DM-10` row that is already `running` or `paused` (FR-52 applies to new sessions only).
4. `defaultPriorityProfileId` must reference a profile whose `isActive = true`; deactivating a profile requires another to be nominated in the same operation (DM-6 invariant 1).

DM-23 — Device

Field	Type	Req	Rules
`id`	opaque id	yes	Client-minted on first launch; stable for the install.
`ownerId`	ref → `DM-22` \| null	no	Null while unauthenticated.
`label`	string 1–80	yes	User-visible name; defaults to a platform-derived label.
`platformClass`	enum `desktop` \| `tablet` \| `phone` \| `unknown`	yes	Used for layout expectations only; never for authorization.
`firstSeenAt` / `lastSeenAt`	timestamp	yes	—
`clientVersion`	string	no	Diagnostic only; never used to gate behaviour.
`clockSkewSeconds`	integer	derived	Last observed difference from server time; drives the `clock\_corrected` event (see `DM-11`).
`revokedAt`	timestamp \| null	no	Set when a user revokes a device.
`createdAt` / `updatedAt`	timestamp	yes	—

****Invariants****

1. A device row is never hard-deleted while referenced by `DM-10`, `DM-11`, `DM-19`, `DM-21` or `DM-24`; revocation sets `revokedAt` and severs future writes, leaving history intact.
2. A revoked device may still read nothing; its queued `DM-19` actions are terminal-failed with a stated reason rather than silently dropped.
3. `clockSkewSeconds` beyond tolerance makes the server clock authoritative for all timing writes (F-3 sessions, F-1 urgency, F-2 relative dates) and the correction is recorded, so the user is never told a session ran for a duration the records contradict.

DM-24 — AuditEvent

Append-only history. This is what makes the explainability and reversibility requirements answerable after the fact, and what lets a user see that an override expired rather than silently vanishing.

Field	Type	Req	Rules
id	opaque id	yes	—
ownerId	ref → DM-22	yes	—
at	timestamp, server	yes	Client at retained separately when it differed.
actorType	enum user \ system \ integration \ automation	yes	—
actorDeviceId	ref → DM-23	\ null no	Present for user actors.
eventType	closed enum (below)	yes	An unrecognized event type is not representable; new behaviour requires a deliberate extension.
subjectType	enum task \ tag \ project \ dependency \ priority_profile \ rank_override \ capture \ focus_session \ connection \ integration_action \ webhook_delivery \ theme_preference \ user_profile \ export \ purge	yes	—
subjectId	opaque id	yes	—
beforeSummary / afterSummary	structured, non-secret	\ null no	Redacted to field names and safe values; never raw payloads, never credentials.
reason	enum user_action \ expiry \ completion \ policy \ provider_change \ reconciliation \ retention	\ null no	—
correlationId	string	\ null no	Groups one user action that touched several entities.
retentionClass	enum standard \ extended \ until_account_deletion	yes	Drives the purge matrix in §12.

****Required event types (minimum closed set)****

task.created · task.updated · task.status_changed · task.deleted · task.restored ·
task.bulk_reordered · dependency.added · dependency.rejected_cycle · dependency.invalidated ·
priority_profile.updated · priority_profile.rollback · rank_override.created · rank_override.released ·
rank_override.adjusted · capture.applied · capture.discarded · focus_session.started ·
focus_session.paused · focus_session.extended · focus_session.ended ·
focus_session.abandoned · focus_session.auto_expired · focus_session.clock_corrected ·
streak_definition.changed · period.changed · connection.authorized · connection.state_changed ·
connection.secret_rotated · connection.secret_revoked · integration_action.enqueued ·
integration_action.failed · integration_action.dead_lettered · integration_item.orphaned ·
integration_item.reconciled · webhook.delivery_failed · theme.preference_changed ·
theme.resolved_from_signal · data.exported · data.purged.

****Invariants****

1. Append-only. A correction is a new event; no event is ever edited or deleted except by an accounted retention purge (reason = retention), which itself emits data.purged.
2. ****No secrets, ever.**** beforeSummary/afterSummary may name a field and say "changed", but may not carry a token, a signing secret, a raw provider payload, or a full captured line of user text (convention 8).
3. Every requirement in this spec that promises the user an explanation (F-1 FR-5, FR-8) or a visible consequence of an automatic change (F-5 FR-103, F-3 FR-56) has at least one corresponding event type above. An automatic state change with no event type is a modelling bug.
4. Volume is bounded: events carry summaries, not snapshots. An event must not grow with the size of the entity it describes.
5. Correlated events share a correlationId, so one user action that touched four entities reads as one story rather than four unrelated entries.

10. Cross-entity integrity

10.1 Reference and deletion matrix

Deleting	Effect on references
`DM-1` Task	Removes its `DM-3` links and its `DM-7` score; deactivates its `DM-8` overrides and records them as released; `DM-5` edges touching it flip to `invalidated` and the counterpart becomes `dependencyState = ready`; `DM-9` capture keeps `taskId` but is marked `superseded`; `DM-18` links become `orphaned` (**not** deleted); `DM-10` sessions are preserved as orphaned history and their `outcome` is back-filled; `DM-13`/`DM-14` are recomputed for the affected dates; `DM-19` queued actions for it are canceled with a stated reason.
`DM-2` Tag	Removes its `DM-3` links; `usageCount` recomputed. Tasks are untouched.
`DM-4` Project	Its tasks re-home to the default project (`DM-22.defaultProjectId`) in one atomic step; the default project itself cannot be removed.
`DM-5` Dependency edge	Only the edge is removed; neither task changes `status`.
`DM-6` PriorityProfile	Cannot be removed while active; superseding requires a new revision. `DM-7` rows keep their `profileRevision` and remain interpretable.
`DM-8` RankOverride	Releasing it must restore the task to the position its `DM-7.score` implies — the score is deliberately retained while an override is active so this is exact.
`DM-9` CaptureAttempt	Purged only by retention; never removed on task deletion.
`DM-10` FocusSession	Not deletable by the user. Purged only with the account or by retention; `DM-11` events are removed in the same transaction.
`DM-15` Connection	Queued `DM-19` actions terminal-fail with a stated reason; `DM-16` handles are revoked; `DM-18` links become `unlinked` with `taskId` preserved so the local task survives a disconnection intact (disconnecting must never destroy the user's tasks).
`DM-16` SecretRef	Revoking is idempotent and immediate; in-flight `DM-19` actions fail attribution and are retried under the next `rotationGeneration` or fail with a clear message.
`DM-18` ExternalItemLink	The local `DM-1` task is never deleted as a consequence.
`DM-22` UserProfile	Account deletion cascades per §12 with a stated grace period, and emits a terminal purge record.
`DM-23` Device	Never hard-deleted while referenced (see DM-23 invariant 1).

10.2 Consistency units

Unit	Atomic boundary	Why
Capture apply	`DM-9` status change + `DM-1` creation + `DM-3` links + project assignment	A capture must never leave a half-created task visible (DM-9 invariant 4).
Completion	`DM-1.status` + `completedAt` + `completionOrigin` + affected `DM-13`/`DM-14` recompute + `DM-8` auto-release + `DM-24` event	Removes the window where the tick is visible but the metrics have not moved (F-4).
Queue recompute	`DM-6` weight change + `DM-7` invalidation + re-rank + `DM-24` event	Prevents a queue that visibly disagrees with the weights the user just saved (FR-4).
Focus state change	`DM-10.state` + `DM-11` event + single-active-session constraint check	The single-active-session rule is enforced in the same transaction as the state write, not by UI (DM-10 invariant 1).
Connection state change	`DM-15.state` + `DM-16` handle status + `DM-19` queue effects + `DM-24` event	A disconnected connection must not leave a live token or a live queue.
Theme change	`DM-21` row write + `revision` bump; rendering does not wait on it	Local-first application with server reconciliation (DM-21 invariant 4).

11. Derived-data lifecycle

Derived set	Invalidation triggers	Recompute scope	User-visible staleness contract
`DM-7` PriorityScore	task created/updated/completed/deleted; any `DM-5` edge change; `DM-6` revision bump; local-date rollover under `DM-22.timezoneId` (a "due today" task must become due today at the owner's midnight, not the server's)	The affected task; whole queue only on a profile or rollover change	Recompute is not allowed to block the UI. A queue may render from a stale score for at most the freshness window in the NFR-5 range, then must show an updating indicator (NFR-4 range).
`DM-13` DailyActivityRollup	completion, reopen, task deletion, `completionOrigin` change, focus session ended, timezone change (from the change point forward only)	Only the affected local dates, using `sourceWatermark`	A late-arriving external completion updates the affected day and surfaces a non-blocking "updated" cue rather than an unexplained jump (DM-13 invariant 2).
`DM-14` StreakRecord	any change to a relevant `DM-13` date; `streakDefinition` change	Current streak always; longest only when the current exceeds it	`isProvisional = true` while today can still break the streak, so a streak is never shown as final before the local day ends (DM-14 invariant 4).
`DM-1.dependencyState`	`DM-5` edge created/invalidated/satisfied	The single affected task	Never stale: it is computed on read of the queue, so a "blocked" badge cannot linger after its blocker completes.
`DM-2.usageCount`	`DM-3` link added/removed	The single tag	Tolerated staleness; never used for correctness.
`DM-15` health fields	every terminal `DM-17` run	The single connection	Health is pushed, not polled, so the connection list cannot show `connected` for a connection that a sync has already proven broken.
`DM-21.effectiveTheme`	preference change; system signal change; account default change; new device with no row	The single device, resolved locally first	Must resolve with **no** network dependency (DM-21 invariant 3, 4).

****Invariant for the whole section:**** dropping every derived entity and recomputing from sources must produce byte-identical user-visible numbers and orderings. If it does not, the derivation is under-specified — that is the test for whether a derived model is legitimate.

12. Retention and purge

Data	Retention	Purge trigger	Notes
`DM-1` soft-deleted task	Grace period fixed by policy (see OQ-5)	Grace elapsed, or account deletion	Restorable during grace (DM-1 invariant 2); `DM-24` records both delete and restore.
`DM-9.rawText`	Bounded window (see OQ-6)	Retention elapsed	Free text is the most sensitive capture data; after purge the applied `DM-1` task is untouched and the parse is still explainable from `parseResult`.
`DM-10` / `DM-11`	Until account deletion	Account deletion	Sessions feed analytics; shortening this would silently change F-4 history.
`DM-13` / `DM-14`	Until account deletion	Account deletion	Recomputable from `DM-1`, but retained so a purge of tasks cannot retroactively rewrite the metrics the user saw.
`DM-16` revoked handle	Metadata until account deletion; secret material revoked immediately	—	The handle is retained only so an audit trail can say **that** a rotation happened (DM-16 invariant 2).
`DM-17` / `DM-19` / `DM-20`	Rolling window sized by `retentionClass`	Retention elapsed	Terminal failures (dead_letter) are retained longer than successes so a user-reported failure is still diagnosable.

| `DM-18` `orphaned`/`unlinked` links | Until account deletion | Account deletion | Retained deliberately so a re-import cannot silently resurrect a deleted task (DM-18 invariant 2). |
 | `DM-24` by `retentionClass` | `standard` < `extended` < `until\_account\_deletion` | Retention elapsed for that class | Purging emits its own `data.purged` event; an unexplained gap in the audit trail is not permissible. |

13. Data-flow map (feature ↔ models ↔ API ranges)

Model ids are defined here only. API ids in the neighbouring column are defined in the API Contracts section and are referenced as plain text — this table re-defines none of them.

Feature	Reads	Writes	Derived	API surface (plain text)
F-1 Intelligent Task Prioritization	`DM-1`, `DM-2`, `DM-3`, `DM-4`, `DM-5`, `DM-6`, `DM-7`, `DM-8`, `DM-22`	`DM-6`, `DM-8`	`DM-7`, `DM-1.dependencyState`	API-1 ... API-9
F-2 Quick Add	`DM-1`, `DM-2`, `DM-4`, `DM-22`, `DM-23`			API-10 ... API-14
F-3 Focus Mode	`DM-1`, `DM-10`, `DM-11`, `DM-22`, `DM-23`			API-15 ... API-20
F-4 Progress Analytics	`DM-1`, `DM-10`, `DM-12`, `DM-13`, `DM-14`, `DM-22`			API-21 ... API-25
F-5 Integration Hub	`DM-1`, `DM-2`, `DM-4`, `DM-15`, `DM-16`, `DM-18`, `DM-22`			API-26 ... API-35
F-6 Dark Mode	`DM-21`, `DM-22`, `DM-23`			API-36 ... API-39
All	`DM-24` (read-only history)			

14. Modelling constraints and non-goals

1. **Single-owner by construction.** Every scoped entity carries `ownerId` and no model expresses sharing, membership, or cross-owner visibility. Adding collaboration later is a deliberate extension, not an accident of a missing column (see OQ-7).
2. **No vendor, engine, index, partition or replication concept appears above.** Physical shape, indexing for the queue read path, and secret custody are the Architect's decisions. The only storage obligations this model imposes are the invariants: uniqueness rules, the single-active-session constraint, append-only histories, idempotency keys, and monotonic versions.
3. **No denormalized copy of a task outside `DM-1`.** The queue, analytics and integrations all read the same row, so a completion in one place is a completion everywhere.
4. **Every timestamp is UTC; every local date is derived, never stored raw** except in `DM-13`/`DM-14`, where the derivation is frozen deliberately and the timezone used is recorded beside it.
5. **`unknown` is always representable** for externally supplied enumerations (convention 7); no model may fail an entire sync because one unrecognized value arrived.
6. **No model is a store of user-authored HTML or executable content.** `DM-1.title`/`notes`, `DM-15.label`, `DM-8.reason` and `DM-24.reason` are text values treated as data on render.

Open Questions

Id	Question	Owner of the decision	Why it blocks/affects modelling
----	----------	-----------------------	---------------------------------

|---|---|---|---|

| OQ-1 | Should the product support **multiple priority profiles** (e.g. a "work" and a "home" profile, switchable) rather than exactly one active profile? This model enforces one active profile per owner via `DM-6.isActive` and `DM-22.defaultPriorityProfileId`. | USER | Multi-profile changes `DM-6` cardinality, adds a profile-selection concept to F-1's UI, and makes `DM-7.profileId` a user-visible dimension. It cannot be added later without touching saved rankings' meaning. |

| OQ-2 | Should **recurring tasks** be supported? `DM-1.origin` currently has a `recurring` member, but no recurrence-rule entity exists. A recurrence must be modelled explicitly (a rule that *generates* task instances) or the `recurring` member should be removed as unreachable. | USER | Either a new entity plus generation history is required, or the enum member is dead and should be deleted before implementation. Leaving it undefined invites an implementer to invent semantics. |

| OQ-3 | Must a focus session always be **bound to a task**? This model requires it (`DM-10.taskId` is mandatory), so an unbound "just a timer" session is not representable. | USER | If unbound sessions are wanted, `DM-10.taskId` becomes nullable, F-3's entry points widen, and F-4's `tasksCompletedViaFocus` attribution needs a new rule. |

| OQ-4 | Is the **single-active-session rule** product-wide (one per owner across all devices, as modelled) or per device? | USER | Per-device would relax `DM-10` invariant 1, change conflict handling when two devices run timers, and change what "focus time today" means in `DM-13`. |

| OQ-5 | What is the **soft-delete grace period** before a deleted task is purged? No default is assumed here. | USER | Sets the restore window in the UI and the retention row in §12. |

| OQ-6 | How long may `DM-9.rawText` (the user's raw captured line) be retained? | USER | Directly affects data minimisation and what "explain this parse" can show after retention elapses. |

| OQ-7 | Is Progress Analytics ever intended to cover **shared or team** data? This model is strictly single-owner. | USER | Team scope changes every scoped entity's identity and the analytics aggregation unit; retrofitting it is expensive. |

| OQ-8 | Do externally synced items need to be **materialised as tasks** (as modelled in `DM-18`), or should the product show a separate mirrored view that only becomes a task on demand? | USER (with Architect) | Determines whether external items consume `DM-1`'s identity space and whether they appear in the F-1 queue and F-4 metrics by default. |

| OQ-9 | Beyond `DM-24`, is a user-facing **undo history** required (e.g. undo last N actions)? | USER | An undo surface imposes stricter sequencing and retention than an audit log, and would need an explicit operation stack rather than event summaries. |

Generated by PipelineExecutor via opencode-go API